

יובל יוגב - Drive2Game

תיכון אלון לאמנויות ולמדעים רמת השרון



שם התלמיד: יובל יוגב

מספר תעודת זהות: 330780784

שם המנחה: עידן פלדברג, עודי רז, רום רפסון, יעקב שוצמן וליאת סגל

שם החלופה: הגנת סייבר

תאריך ההגשה: 16.05.2025



DRIVE

“YOU’LL NEVER WALK ALONE”

יובל יוגב - Drive2Game

מבוא: 3.....

3..... ייזום
5..... פירוט תיאור המערכת (אפיון)

תיאור תחום הידע - פרק מילולי: 8.....

8..... פירוט מעמיק של היכולות שהוצגו בשלב הקודם ומעבר (ניתוח)

מבנה / ארכיטקטורה: 9.....

9..... תיאור הארכיטקטורה שנלקחו למימוש המערכת (עיצוב)
9..... תיאור הטכנולוגיה הרלוונטית
9..... תיאור זרימת המידע במערכת
11..... תיאור האלגוריתמים מרכזיים בפרויקט
13..... תיאור סביבת הפיתוח
14..... תיאור פרוטוקול התקשורת
17..... תיאור מסכי המערכת
24..... תיאור מבני הנתונים
28..... סקירת החולשות והאיומים

מימוש הפרויקט: 29.....

29..... פירוט מימוש המערכת:

מדריך למשתמש: 38.....

רפלקציה / סיכום אישי: 43.....

43..... תיאור תהליך העבודה
43..... תהליך הלמידה
44..... תובנות מהתהליך ובראייה לאחור
44..... תודות
44..... תשובות הסקר
44..... משפט עם משמעות

ביבליוגרפיה: 45.....

נספחים: 46.....

46..... תיעוד הקוד
46..... main.py
46..... graphics.py
94..... Client_server.py
110..... Encryption.py
113..... DB.py

יובל יוגב - Drive2Game

מבוא:

ייזום

תיאור ראשוני של המערכת והגדרת הלקוח:

- יצירת אפליקציה לארגון הסעות לאוהדי משחקי ספורט ברחבי הארץ.
הסיבה: קיים צורך לאפשר לאוהדים להגיע למשחקים בקלות, במיוחד כשמדובר באזורים מרוחקים.
על פי סקר שנערך בקרב אוהדי ספורט, בו השתתפו 115 נשאלים, נמצאו הנתונים הבאים:
- כ-70% מהמשיבים העידו שלפחות פעם אחת לא היה להם איך להגיע למשחק.
 - 56% מהנשאלים מגיעים למשחקים ברכב פרטי, ו-51% מגיעים בתחבורה ציבורית. (יש כאלה שמשלבים).
 - מתוך 114 נשאלים שענו על השאלה "האם היית מסכים/ה לנסוע בהסעה למשחק עם אנשים שאת/ה לא מכיר/ה?" - נמצא ש-73.7% ענו שהם היו מסכימים. ורק 5.3% מהמשיבים אינם מעוניינים לנסוע למשחק עם אנשים שהם לא מכירים. ו-21% סימנו שהם לא יודעים כרגע.
- הפלטפורמה תאפשר חיבור בין אוהדים שמחפשים הסעה למשחק מסוים לבין נהגים שמציעים הסעות. הפלטפורמה תספק תכונות כמו יצירת נסיעות מותאמות אישית, שיתוף נסיעות עם נוסעים אחרים, אפשרויות דינמיות לשינויים וביטולים, וצ'אט בין משתתפי ההסעה. קהל היעד הוא אוהדי ספורט בארץ המעוניינים להגיע למשחקים של קבוצותיהם.

הגדרת יעדים/מטרות:

- יצירת ממשק שיוצר נסיעות.
- קישור בין נוסעים למסיעים בעזרת מציאת נסיעות לנוסעים.
- יצירת קשר בין הנוסעים למסיע בהסעה בעזרת צאט.
- נתינת עזרה לנוסעים למציאת הסעה על ידי מיקום על המפה.
- לאפשר למשתמשים לבטל או לעדכן את ההסעות שלהם בצורה פשוטה, עם דדליין ברור לביטולים, כדי להבטיח גמישות.
- אינטגרציה עם אפליקציות נוספות כמו מפות, Waze וכודמה, כדי להקל על המשתמשים בתכנון הנסיעה.

בעיות תועלות וחסכונות:

בעיות:

- **קושי בהגעה למשחקים:** אוהדי ספורט מתקשים להגיע למשחקים, במיוחד כאשר הם נערכים באזורים מרוחקים ללא תחבורה ציבורית נוחה.
- **מחסור בפתרונות הסעה נגישים:** אין כיום פלטפורמה ייעודית שמאפשרת לאוהדים לשתף נסיעות באופן פשוט ומסודר.
- **חוסר בקישור ישיר בין נהגים לנוסעים:** קשה למצוא מסיעים אמינים ולקבוע זמני יציאה מדויקים.

תועלות:

- **פלטפורמה מאורגנת ומרכזית:** מקום אחד שבו ניתן למצוא הסעות למשחקים בצורה פשוטה.
- **חיבור בין אוהדים לנהגים בקלות וביעילות.**
- **הגעה למקומות רחוקים:** המערכת תאפשר לאוהדים להגיע גם למשחקים שמתקיימים בערים רחוקות, שלא תמיד קל להגיע אליהן בתחבורה ציבורית.
- **המערכת כוללת דירוגים וביקורות, כך שמשתמשים יוכלו לדעת מראש אם הנהג או הנוסע אמינים ונעימים לנסיעה.**

יובל יוגב - Drive2Game

חסכוניות:

- **חיפוש והתאמת הסעות:** משתמשים יכולים למצוא הסעה שמתאימה להם לפי מיקום, ועוד.
- **מנגנון עדכונים וביטולים:** מאפשר למשתמשים לבצע שינויים בקלות ולתאם מחדש את ההסעות שלהם.
- **מערכת דירוגים:** עוזרת לוודא שהנסיעות מתבצעות בצורה אמינה ובטוחה.
- **מערכת צ'אט:** מאפשרת תקשורת בין נוסעים לנהגים לתיאום פרטים נוספים.
- **זמן:** חיסכון בזמן בחיפוש אחר הסעות.
- **מאמץ:** חיסכון במאמץ הכרוך בארגון הסעה באופן עצמאי.

סקירת פתרונות קיימים:

המצב הקיים הוא שהגעה למשחקים קוראת בעזרת רכבים פרטיים, תחבורה ציבורית, הסעה מאורגנת - אתרים של קבוצות כדורגל/כדורסל שמנהלות בעצמם הסעות של אוהדים למשחקי הקבוצה (לרוב רק למשחקי חוץ), והסעה (=טרמפ) - מתנהלות דרך קבוצות ווצאפ של אוהדים.

סקירת טכנולוגית הפרויקט:

הפרויקט מבוסס על טכנולוגיות קיימות שאינן חדשניות לחלוטין, אך שילובן יחד ליצירת פלטפורמה ייעודית להסעות למשחקים הוא חידוש. קיימות מערכות לשיתוף נסיעות (כגון Uber או BlaBlaCar), אך אין פלטפורמה ייעודית המתמקדת באופן ספציפי באוהדי ספורט, כולל חיבור בין נוסעים למסיעים, צ'אט משולב, ודירוגים.

בעיות ומגבלות שיכולות לנבוע:

- אין מספיק נהגים ולא יהיו מספיק הסעות לכמות האוהדים שמספקת.
- **אמינות נהגים ונסעים** - המערכת תצטרך לכלול מערכת דירוגים כדי לצמצם סיכונים ולהגביר ביטחון.
- **חיבור אינטרנט רציף** - הפלטפורמה היא לא אופליין ונתמך על ידי אינטרנט.

תיחום הפרויקט:

רשתות ותקשורת:

- שילוב API חיצוניים, כגון מפות Google Maps/Waze ואתרי מנהלות בהם נתונים מידע על המשחקים.
- מערכת צ'אט מבוססת WebSockets.

מערכות הפעלה:

- כל מערכת הפעלה אשר מותקנות עליה ספריות הפייטון אשר בהם נוצר שימוש בקוד.

בסיסי נתונים:

- שמירת נתוני משתמשים, הסעות והיסטוריית נסיעות במסד נתונים רלציוני (SQL).

אבטחת מידע:

- אימות משתמשים (הרשמה עם אימייל/טלפון ואימות דו-שלבי אפשרי).

אלגוריתם חיפוש והתאמה:

- התאמת משתמשים למסיעים על בסיס מיקום, ודרישות אישיות (לדוגמה, נסיעה עם ילדים, מספר מקומות פנויים ברכב).

תחומים שהמערכת אינה מטפלת בהם:

- המערכת אינה גובה תשלומים ואינה מנהלת עסקאות בין נהגים לנסעים. כל תיאום תשלום יתבצע ישירות בין המשתמשים, ללא מעורבות מערכתית.

יובל יוגב - Drive2Game

- המערכת אינה אחראית על הגעת הנהג בזמן או עמידה בהתחייבויות בין משתמשים, אלא רק מספקת כלים לדירוג ואמון.
- במקרה של תקלה ברכב או בעיות אחרות במהלך הנסיעה, האחריות היא על המשתמשים. אין תמיכה בשירותי גרירה או חילוץ.
- המערכת נועדה לשיתוף נסיעות בלבד, ולא לניהול קבוצות אוהדים, רכישת כרטיסים למשחקים או ארגון אירועי ספורט.

פירוט תיאור המערכת (אפיון)

תיאור מפורט יותר של המערכת:

Drive2Game היא פלטפורמה שנועדה לחבר בין אוהדי ספורט המחפשים הסעה למשחקים לבין מסיעים פרטיים (נהגים) שמוכנים לקחת אותם. המערכת תספק כלים לתיאום נסיעות משותפות, ניהול מידע, תקשורת בין משתמשים ודירוגי אמינות.

רישום והתחברות:

הרשמה למשתמשים חדשים: כל משתמש יכול להירשם באמצעות כתובת מייל, ובנוסף יוכל גם לחבר את מספר טלפון הנייד.

לכל משתמש יהיה פרופיל אישי שיכלול:

- שם מלא ושם משתמש.
- דירוג אמינות (מספר כוכבים).
- נסיעות קרובות.

מערכת צ'אט מובנית:

- אחרי שאושרה נסיעה, ייפתח צ'אט קבוצתי לכל המשתתפים בה.
- בצ'אט יהיה אפשר לתאם נקודות איסוף, זמני מפגש ופרטים נוספים.
- המערכת תוכל לשלוח התראות אוטומטיות במקרה של שינויים.

דירוגים:

- לאחר כל נסיעה, המשתמשים יוכלו לדרג זה את זה (1-5 כוכבים).
- הדירוג ישפיע על האמינות של המשתמשים במערכת.
- דירוגים נמוכים באופן עקבי עלולים להוביל להגבלות על משתמשים, כגון אי הסעת נוסעים כי לנסעים לא יהיה אמון במסיע או להפך, אי אישור נוסע להסעה עקב אי אמון בנוסע.

סוגי משתמשים - נוסע ומסיע:

1. מסיע:

חיפוש נוסעים:

מסיע יכול לפרסם נסיעה ולציין את הפרטים:

- משחק יעד (יבחר מתוך רשימה של משחקים הקרובים).
- מיקום יציאה.
- מספר מקומות פנויים ברכב.
- האם הנסיעה בתשלום (ואם כן, מחיר).
- תנאים נוספים (למשל, לא מעשנים, מוזיקה שקטה, מינימום נוסעים וכו').
- נוסעים יכולים להגיש בקשות להצטרף לנסיעה, והמסיע יכול לאשר או לדחות בקשות.

יובל יוגב - Drive2Game

ניהול נסיעות ועדכונים:

- המערכת תאפשר לכל מסייע לראות את הנסיעות הקרובות שלו ולבטל אותן.
- יהיו דד-ליינים מוגדרים לביטול (32 שעות לפני המשחק).
- המערכת תשלח התראות על שינויים בנסיעה.

2. נוסע:

חיפוש מסייעים:

- נוסע שרוצה להגיע למשחק יכול לחפש הסעות זמינות לפי מיקום.
- כל תוצאה תוצג עם פרטי המסייע, דירוגו ותנאי הנסיעה.
- לאחר הגשת בקשה, המסייע צריך לאשר או לדחות אותה.

ניהול נסיעות ועדכונים:

- המערכת תאפשר לכל נוסע לראות את הנסיעות הקרובות שלו ולבטל אותן.
- יהיו דד-ליינים מוגדרים לביטול (32 שעות לפני המשחק).
- המערכת תשלח התראות על שינויים בנסיעה.

פירוט הבדיקות:

רישום והתחברות:

מטרת הבדיקה: לוודא שמשתמשים יכולים להירשם ולהתחבר למערכת בצורה תקינה.

תכנון הבדיקה:

1. ניסיון הרשמה עם פרטים תקינים ← ציפייה: הרשמה מוצלחת.
2. ניסיון הרשמה עם אימייל לא תקין ← ציפייה: הודעת שגיאה.
3. ניסיון הרשמה עם סיסמה לא בפורמט ← ציפייה: הודעת שגיאה.
4. ניסיון הרשמה עם שם משתמש לא בפורמט ← ציפייה: הודעת שגיאה.
5. ניסיון הרשמה עם תאריך לידה לא תקין ← ציפייה: הודעת שגיאה.
6. ניסיון התחברות עם שם משתמש וסיסמה נכונים ← ציפייה: התחברות מוצלחת.
7. ניסיון התחברות עם סיסמה שגויה ← ציפייה: הודעת שגיאה.

יצירת נסיעה:

מטרת הבדיקה: לבדוק שמשתמשים יכולים להציע נסיעות עם הפרטים הנדרשים.

תכנון הבדיקה:

1. יצירת נסיעה עם כל השדות תקינים ← ציפייה: הנסיעה נשמרת במערכת.
2. יצירת נסיעה ללא ציון מספר מקומות פנויים ← ציפייה: הודעת שגיאה.
3. יצירת נסיעה ללא כתובת ← ציפייה: הודעת שגיאה.

חיפוש והצטרפות לנסיעה:

מטרת הבדיקה: לוודא שמשתמשים יכולים לחפש ולהצטרף לנסיעות קיימות.

תכנון הבדיקה:

1. חיפוש נסיעות לעיר עם נסיעות זמינות ← ציפייה: הצגת רשימת נסיעות זמינות.
2. חיפוש נסיעות לעיר ללא נסיעות זמינות ← ציפייה: הודעה שאין נסיעות זמינות.
3. הצטרפות לנסיעה עם מקומות פנויים ← ציפייה: בקשת הצטרפות נשלחת למסייע.
4. הצטרפות לנסיעה עם תנאים שלא מתאימים (למשל גיל מוגבל) ← ציפייה: הודעת שגיאה.

יובל יוגב - Drive2Game

תכנון ניהול לו"ז לפיתוח המערכת:

1. בניית שלד המערכת:
 - קלאס של תקשורת
 - קלאס של מסד נתונים
 - קלאס של הצפנה
 - קלאס שמריץ את המערכת (thread)
 - קלאס של UI
2. קלאס של UI - כל בניה של מסך כוללת את הפונקציות שלו:
 - בניית מסך בית
 - בניית מסך של התחברות
 - בניית מסך של רישום
 - בניית מסך של מסיע - הצעת נסיעות
 - בניית מסך של נוסע - חיפוש נסיעות
 - בניית מסך של נסיעות קיימות
 - בניית מסך של צאט
 - בניית מסך של דירוגים
 - בניית מסך של איזור אישי

ניהול הסיכונים בפרויקט והדרכים להתמודדות איתם:

- **הסיכון - חוסר בנהגים:** לא יהיו מספיק נהגים כדי לספק מענה לכל הביקוש.
- **דרך ההתמודדות:** שיתוף פעולה עם קבוצות אוהדים, פרסום נרחב של הפלטפורמה.
- **הסיכון - בעיות אמינות:** קושי באמינות של נהגים ונוסעים.
- **דרך ההתמודדות:** מערכת דירוג משתמשים, אפשרות לתת משוב על נהגים, אימות זהות משתמשים.

יובל יוגב - Drive2Game

תיאור תחום הידע - פרק מילולי:

פירוט מעמיק של היכולות שהוצגו בשלב הקודם ומעבר (ניתוח)

פירוט היכולות בצד שרת:

- קבלת הודעות מלקוחות וריבוי לקוחות.
- רישום והתחברות: ברישום השרת שומר נתונים במסד הנתונים. בהתחברות השרת בודק האם קיים במסד הנתונים אימייל כמו האימייל שהוזן, אם אין השרת שולח הודעה מתואמת אל הלקוח. אם האימייל קיים השרת שולף את הסיסמה של האימייל ושולח הודעה מתואמת עם הסיסמה אל הלקוח על מנת לבדוק אם הסיסמה שהוזנה היא הסיסמה הנכונה.
- יצירת נסיעה: בזמן יצירת הנסיעה והוספתה אל מסד נתונים השרת בודק שאין כבר הסעה כזאת קיימת ואם יש בעיות. אם ההסעה קיימת או יש בעיות אז השרת שולח הודעה מתאימה אל הלקוח. אם אין השרת שומר אצלנו במסד הנתונים את הנסיעה החדשה ושולח הודעה מותאמת אל הלקוח.
- שליחת הנסיעות אל הלקוח: השרת שולף במסד הנתונים הסעות קיימות על פי נתונים ספציפים אם קיימות הסעות השרת שולח אותם אל הלקוח. אם אין אז השרת שולח הודעה מותאמת שלא קיימים הסעות.
- ספירת מספר נוסעים: השרת שולף ממסד הנתונים כמה נוסעים קיימים בהסעה ספציפית ושולח את התשובה אל הלקוח.
- קבלת הדירוג של הלקוח: השרת שולף ממסד הנתונים אך הדירוג הקיים של אחד מהמשתמשים. את הדירוג השרת שולח אל הלקוח.
- הצטרפות לנסיעה: השרת מקבל נתונים של הלקוח לגבי ההסעה ושומר המסד הנתונים את בקשתו להצטרף לנסיעה.
- מוצא את הנסיעות של הלקוח בתור נהג: השרת שולף ממסד הנתונים את כל הנסיעות בהן הוא נהג. אם אין הוא שולח הודעה מתואמת ואם יש אז הוא שולח אל הלקוח את כל הנסיעות.
- מוצא את הנסיעות של הלקוח בתור נוסע: השרת שולף את כל המשחקים שבהן הלקוח הוא נוסע. אם יש השרת שולח הודעה עם כל הנסיעות, אם אין השרת שולח הודעה שאין.
- קבלה של סטטוס הנוסע בנסיעה ספציפית: השרת מקבל נוסע ונסיעה ובודק במסד הנתונים את הסטטוס של הנוסע, הודעה מותאמת תשלח אל הלקוח בין אם הוא אושר, ממתיין או נדחה.
- קבלה של הנהג והנוסעים בנסיעה ספציפית: השרת מקבל נסיעה והוא שולף ממסד הנתונים את הנוסע של ההסעה ואת כל הנוסעים ושולח הודעה מתואמת אל הלקוח.
- שינוי סטטוס של נוסע ופרטים של משתמש: השרת מקבל נתונים ואת מה שהוא צריך לשנות. הוא משנה את הנתונים במסד הנתונים ושולח הודעה מתואמת אל הלקוח.
- שליחת אימייל עם קוד: השרת מקבל את האימייל של הלקוח ושולח לו באימייל קוד אימות על מנת שיהיה אימות דו שלבי. לאחר שליחת הקוד באימייל השרת שולח את הקוד אל הלקוח על מנת שתהיה בדיקה אם הקודים שווים.

פירוט היכולות בצד לקוח:

- הפעלת ממשק גרפי.
- חיבור לשרת.

יובל יוגב - Drive2Game

מבנה / ארכיטקטורה:

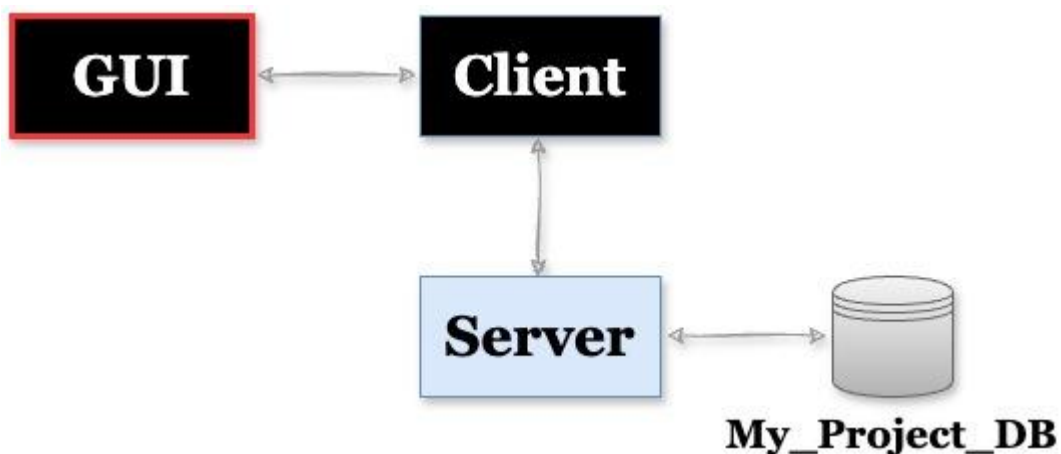
תיאור הארכיטקטורה שנלקחו למימוש המערכת (עיצוב)

המערכת בנויה במודל של שרת-לקוח. הלקוח הוא תוכנת GUI הכתובה בפייתון עם ספריית Tkinter, והשרת הוא תוכנית פייתון המשתמשת בסוקטים לתקשורת. מסד הנתונים הוא SQLite.

תיאור החומרה - רכיבים שונים והקשרים ביניהם:

החומרה הנדרשת היא מחשב סטנדרטי עם חיבור לאינטרנט שמסוגל להריץ את התוכנה הכתובה בשפת Python. אין דרישות חומרה מיוחדות.

שרטוט המציג קשרים אלו:



תיאור הטכנולוגיה הרלוונטית

- שפת תכנות - Python.
- מערכת הפעלה - Mac, windows.
- תקשורת - סוקטים.
- תחומי עניין - פיתוח תוכנה, תקשורת נתונים, ממשקי משתמש.

תיאור זרימת המידע במערכת

1. ניהול משתמשים (הרשמה וחיבור):

• הרשמה:

- ❖ המשתמש מזין פרטים אישיים (שם, תאריך לידה, שם משתמש, סיסמה, אימייל, טלפון).
- ❖ הנתונים נשלחים כהודעה מוצפנת עם פרטוקול אל השרת כסוקט.
- ❖ הנתונים נבדקים (תקינות קלט לדוגמה: ייחודיות שם משתמש ואימייל).
- ❖ במקרה של אימות מוצלח, הנתונים נשמרים במסד הנתונים.
- ❖ השרת שולח הודעה כסוקט - הודעה חיובית אם האימות מוצלח ושלילית אם האימות לא קרה.

• התחברות:

- ❖ המשתמש מזין אימייל וסיסמה.
- ❖ הנתונים נשלחים כהודעה מוצפנת עם פרטוקול אל השרת כסוקט.
- ❖ הנתונים נבדקים מול מסד הנתונים.
- ❖ במקרה של אימות מוצלח, המערכת מאפשרת גישה למשתמש.

יובל יוגב - Drive2Game

❖ במקרה של אימות לא מוצלח, נשלחת הודעה בהתאם כסוקט.

2. ניהול הסעות (יצירה, חיפוש, הצטרפות):

• יצירת הסעה:

- ❖ המשתמש בוחר משחק.
- ❖ נהג מזין פרטי הסעה (כתובת התחלה, מספר נוסעים, הערות).
- ❖ הנתונים נשלחים כהודעה מוצפנת עם פרטוקול אל השרת כסוקט.
- ❖ הנתונים נבדקים.
- ❖ הנתונים נשמרים במסד הנתונים אם הכל תקין, אם לא תשלח הודעה בהתאם.

• חיפוש הסעה:

- ❖ המשתמש בוחר משחק.
- ❖ המשתמש יכול לבחור יישוב או אזור בו תהיה ההסעה.
- ❖ הנתונים נשלחים כהודעה עם פרטוקול אל השרת כסוקט.
- ❖ המערכת שולפת הסעות רלוונטיות ממסד הנתונים.
- ❖ התוצאות מוצגות למשתמש.

• הצטרפות להסעה:

- ❖ המשתמש בוחר הסעה.
- ❖ נוסע מזין פרטי הסעה (כתובת אסיפה, אדם נוסף אם הנוסע מעוניין).
- ❖ הנתונים נשלחים כהודעה מוצפנת עם פרטוקול אל השרת כסוקט.
- ❖ הנתונים נבדקים.
- ❖ אם הכל תקין המערכת מעדכנת את מסד הנתונים (צירוף נוסע להסעה).

3. הצגת ההסעות שאליהם המשתמש קשור וגם ההסעות הקודמות שלו:

• הצגת ההסעות שאליהם המשתמש קשור:

- ❖ המשתמש נכנס אל חלון ההסעות.
- ❖ נתוני המשתמש נשלחים אל השרת בהודעה מוצפנת.
- ❖ השרת בודק איזה הסעות קיימות קשורות אל המשתמש (מסיע או נוסע) ומציג אותם במסך.

• הצגת ההסעות הקודמות של המשתמש:

- ❖ המשתמש נכנס אל חלון ההסעות הקודמות.
- ❖ נתוני המשתמש נשלחים אל השרת בהודעה מוצפנת.
- ❖ השרת בודק באיזה הסעות קודמות המשתמש היה (מסיע או נוסע) ומציג אותם במסך.

4. תצוגת משחקים ומידע (גישה לנתונים חיצוניים, עיבוד תמונות):

• תצוגת המשחקים:

- ❖ המערכת אוספת נתוני משחקים מאתר אינטרנט חיצוני שהורד (HTML).
- ❖ הנתונים מעובדים ומוצגים למשתמש.

• עיבוד תמונות:

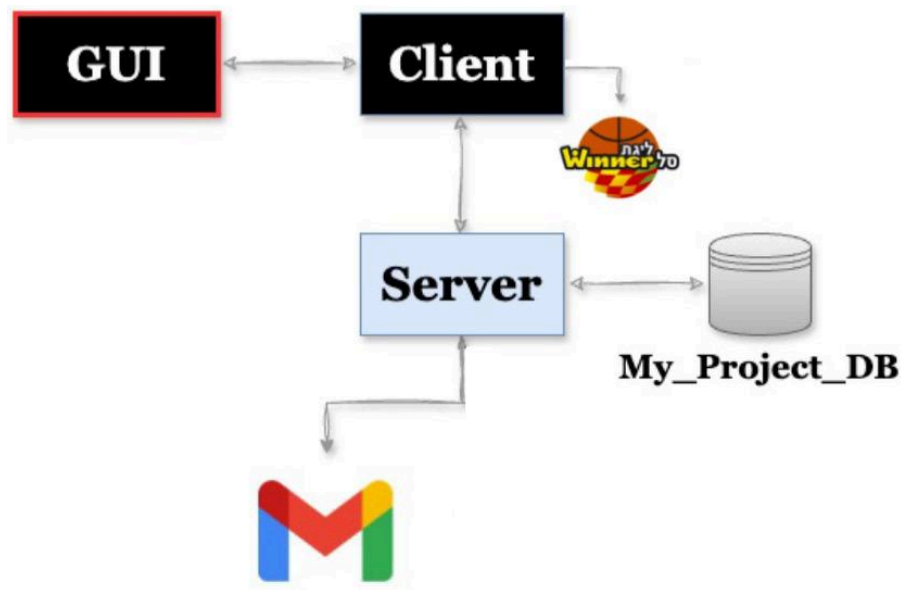
- ❖ המערכת מעבדת תמונות (חיתוך לעיגול, שינוי גודל).
- ❖ התמונות המעובדות מוצגות בממשק המשתמש.
- ❖ שליחת מיילים:
- ❖ המערכת משתמשת בAPI ושולחת אימייל באמצעות gmail.

5. תקשורת בין לקוח לשרת:

- השרת מנהל חיבורים מרובים של לקוחות.
- השרת מנתק לקוחות לא פעילים.

יובל יוגב - Drive2Game

הצגת זרימת המידע בין הרכיבים המרכזיים במערכת:



תיאור האלגוריתמים מרכזיים בפרויקט

ניסוח וניתוח של הבעיה האלגוריתמית:

ניסוח הבעיה האלגוריתמית: מערכת המנהלת תקשורת בין שרת ולקוחות מרובים, תוך ניהול חיבורים במקביל, קריאת בקשות, עיבוד נתונים והצפנה לצורך אבטחה. כמו כן, המערכת צריכה לשלוח ולפרש מידע מתוך קבצי HTML.

ניתוח הבעיה האלגוריתמית:

1. ניהול חיבורים מרובים בשרת: השרת צריך לנהל מספר חיבורים במקביל, לקרוא בקשות, ולעבד אותן.
2. הצפנה ופענוח נתונים: הנתונים שנשלחים ונקלטים עלולים לכלול תווים בעברית, ולכן על המערכת להבטיח שההצפנה לא תפגע בסדר התווים או באפשרות לשחזרם.
3. שלפת משחקים מקובץ HTML: על המערכת לאתר ולחלץ מידע מובנה מקובץ HTML, כגון רשימת משחקים עתידיים. יש לזהות בלוקים רלוונטיים בתוך ה-HTML, לסנן מידע שאינו נחוץ, ולוודא שהנתונים יישארו בפורמט שניתן להשתמש בו בקלות בהמשך.

תיאור אלגוריתמים קיימים לפתרון הבעיה:

- ◀ פתרון לניהול חיבורים מרובים בשרת: שימוש ב- `select.select()`.
- ◀ פתרון להצפנה ופענוח נתונים: שימוש במיפוי מותאם של תווים ושמירת אינדקסים של תווים עבריים כדי להבטיח שחזור תקין.
- ◀ פתרון לשלפת משחקים מקובץ HTML: שימוש ב- `BeautifulSoup` עם ביטויים רגולריים (re) לחיפוש בלוקים רלוונטיים.

הפנייה למקורות רלוונטיים:

1. ניהול חיבורים מרובים בשרת: הקובץ `Client_server.py` מכיל קטע קוד המנהל חיבורים באמצעות `select.select()`, שמאפשר לטפל במספר חיבורים במקביל מבלי לחסום את תהליך הריצה של השרת.
2. הצפנה ופענוח נתונים: הקובץ `Encryption.py` מכיל מחלקה בשם `Encryption` שמשתמשת במיפוי בהצפנת RSA, כולל תמיכה בעברית. הקובץ `Client_server.py` משתמש במחלקה זו

יובל יוגב - Drive2Game

להצפנה ופענוח של נתונים שנשלחים בין השרת ללקוח והפוך.
RSA הוא אלגוריתם הצפנה אסימטרי, כלומר משתמש בזוג מפתחות: מפתח ציבורי להצפנה ומפתח פרטי לפענוח.

יצירת מפתחות - בוחרים שני מספרים ראשוניים p, q . נוסחאה $n = p \cdot q$. מוצאים את e, d .

המפתח הציבורי: (e, n) , המפתח הפרטי: (d, n) .

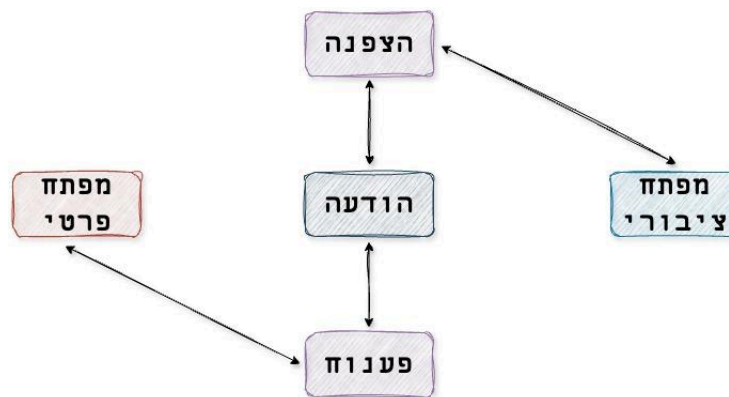
שלב ההצפנה - הטקסט מחולק לבלוקים ומומר למספרים שלמים לפי הערך ASCII שלהם, ולאחר מכן $M^e = C \pmod{n}$. ההודעה המוצפנת - C , המסר - M .

שלב הפענוח - $C^d = M \pmod{n}$ ולאחר מכן ממיר את המספרים מהערך ASCII שלהם לאותיות באנגלית.

התאמה לעברית - בגלל ש-RSA עובד עם מספרים בלבד, נדרש מיפוי:

- אותיות עבריות ← אותיות אנגליות (`hebrew_to_english`)
- אותיות אנגליות ← עברית (`english_to_hebrew`)

תרשים להמחשה:



3. שלילת משחקים מקובץ HTML: בקובץ `graphics.py` יש שימוש ב-`BeautifulSoup` ובביטויים רגולריים (`re`) לחיפוש בלוקים רלוונטיים בקובץ HTML ולשלילת משחקים עתידיים. הפונקציה `games_to_go()` אחראית לחיפוש משחקים בתוך קובץ HTML, זיהוי המבנה של הנתונים, וסינון מידע מיותר.

סקירת הפתרון הנבחר (תוך נימוק הבחירה בו ושליטת הפתרונות האלטרנטיביים, או פיתוח מקורי):

1. ניהול חיבורים מרובים בשרת:
 - הפתרון הנבחר: שימוש ב-`select.select()`.
 - נימוק הבחירה:
 - `select.select()` מאפשר לשרת לטפל במספר חיבורים במקביל בצורה לא חסימתית.
 - מתאים למספר חיבורים נמוך עד בינוני, ללא צורך בהשחלת תהליכים (threads) נוספים או תכנון מבנה מורכב.
2. פתרונות אלטרנטיביים שנשללו:
 - `Thread-per-connection`: יצירת Thread לכל חיבור חדש עלולה להוביל לצריכת זיכרון גבוהה ולבעיות סנכרון.
 - `Multiprocessing`: פתרון זה מיועד בעיקר למקרים שבהם יש צורך לנצל מספר ליבות במעבד, אך לא בהכרח מתאים לשרת רשת עם חיבורים מרובים.
2. הצפנה ופענוח נתונים:
 - הפתרון הנבחר: שימוש במיפוי מותאם אישית של תווים ושמירת אינדקסים של תווים עבריים.
 - נימוק הבחירה:

יובל יוגב - Drive2Game

- מאפשר תמיכה בתווים בעברית בצורה מבוקרת תוך שימור המבנה המקורי של הטקסט.
- אינו דורש שימוש במפתח הצפנה חיצוני, מה שמפשט את ניהול המערכת.
- **פתרונות אלטרנטיביים שנשללו:**
 - Base64 Encoding: לא מספק אבטחה אמיתית אלא רק קידוד, ולכן אינו רלוונטי לצורך הגנה על הנתונים.
 - UTF-8 Encoding בלבד: לא מספק הצפנה אלא רק דרך לאחסן טקסטים בשפות שונות, ולכן לא עונה על הצורך בביטחון מידע.
- 3. **שליפת משחקים מקובץ HTML:**
- **הפתרון הנבחר:** שימוש ב- **BeautifulSoup** וביטויים רגולריים (**re**).
- **נימוק הבחירה:**
 - BeautifulSoup מאפשרת פירוק מבנה ה-HTML בצורה פשוטה ונוחה לקריאה.
 - ביטויים רגולריים (**re**) מאפשרים איתור גמיש של דפוסי טקסט בתוך ה-HTML, מה שמקל על ההתמודדות עם שינויים קלים במבנה הדף.
 - שילוב שתי השיטות מספק פתרון חזק ומדויק לחילוץ נתונים.
- **פתרונות אלטרנטיביים שנשללו:**
 - שימוש ב-PyQuery או Scrapy: פתרונות אלה טובים לסריקת אתרים שלמים, אך מסורבלים יותר במקרה של עיבוד מסמך HTML בודד.
 - Parsing ידני של HTML כטקסט: עלול להיות מסובך מאוד במקרים של מבנה HTML מורכב, ולא ניתן להסתמך עליו לאורך זמן אם האתר משתנה.

תיאור סביבת הפיתוח

פרוט כלי הפיתוח הדרושים לפיתוח:

שפת התכנות היא Python.

ספריות:

- **socket** – לניהול חיבורים בין השרת ללקוחות.
- **select** – לטיפול במספר חיבורים במקביל ללא חסימה.
- **BeautifulSoup** (מתוך **bs4**) – לפירוק ושליפת מידע מקובצי HTML.
- **re** – לביטויים רגולריים לצורך חיפוש וניתוח טקסט.
- **sqlite3** – לניהול מסד הנתונים של המשתמשים והמשחקים.
- **tkinter** – לממשק המשתמש הגרפי.
- **PIL** (מתוך **Pillow**) – לעיבוד תמונות בממשק הגרפי.
- **OpenCageGeocode** (מתוך **opencage.geocoder**) – הפעלת API של מציאת קורדינטות.
- **pyotp** – לטיפול בסיסמאות חד פעמיות.
- **EmailMessage** (מתוך **email.message**) – לבניית הודעת אימייל.
- **ssl** – לוודא שהחיבור לשרת המייל מאובטח ומוצפן.
- **smtplib** – לשלוח מייל דרך רשת SMTP.
- **datetime** – לניהול תאריכים.

ניהול קוד ומעקב אחר גרסאות בפלפורמת PyCharm.

פרוט הסביבה והכלים הנדרשים לבדיקות:

- PyCharm

יובל יוגב - Drive2Game

תיאור פרוטוקול התקשורת

תיאור מילולי של פרוטוקול התקשורת מבנה:

ההודעות מופרדת באמצעות התו '|' אם יש נתונים הנשלחים בהודעה, השדה האחרון בהודעה מייצג את הפרוטוקול.

פירוט כלל ההודעות הזורמות במערכת:

- :SIGNUP
נשלחת מלקוח לשרת.
מבנה השדות בהודעה: שם מלא|תאריך לידה|שם משתמש|סיסמה|אימייל|מספר טלפון|SIGNUP
- :LOGIN
נשלחת מלקוח לשרת.
מבנה השדות בהודעה: אימייל|סיסמה|LOGIN
- :CREATE_RIDE
נשלחת מלקוח לשרת.
מבנה השדות בהודעה: משחק|כתובת התחלה|מספר נוסעים|הערות|שם משתמש
נהג|CREATE_RIDE
- :SEND_THE RIDES
נשלחת מלקוח לשרת.
מבנה השדות בהודעה: משחק|אזור|יישוב|SEND_THE RIDES
- :COUNT_PASSENGER
נשלחת מלקוח לשרת.
מבנה השדות בהודעה: מספר זיהוי נסיעה|COUNT_PASSENGER
- :GET_RATE
נשלחת מלקוח לשרת.
מבנה השדות בהודעה: שם משתמש נהג|GET_RATE
- :JOIN_RIDE
נשלחת מלקוח לשרת.
מבנה השדות בהודעה: פרטי נסיעה|JOIN_RIDE
- :FIND_D_RIDE
נשלחת מלקוח לשרת.
מבנה השדות בהודעה: שם משתמש נהג|FIND_D_RIDE
- :FIND_P_RIDE
נשלחת מלקוח לשרת.
מבנה השדות בהודעה: שם משתמש נוסע|FIND_P_RIDE
- :GET_NUM_OF PASSENGERS
נשלחת מלקוח לשרת.
מבנה השדות בהודעה: מספר זיהוי הנסיעה|סטטוס האישור של הנוסע|GET_NUM_OF PASSENGERS
- :GET_PASSENGER STATUS OF_RIDE
נשלחת מלקוח לשרת.
מבנה השדות בהודעה: מספר זיהוי הנסיעה|שם המשתמש של הנוסע|GET_PASSENGER STATUS OF_RIDE
- :GET_DRIVER AND PASSENGERS
נשלחת מלקוח לשרת.
מבנה השדות בהודעה: מספר זיהוי הנסיעה|האם הוא נהג|אם לא נהג נוסף גם האם הוא נוסע
שאושר|GET_DRIVER AND PASSENGERS
- :CHANGE_PASSENGER STATUS OF_RIDE

יובל יוגב - Drive2Game

- נשלחת מלקוח לשרת.
מבנה השדות בהודעה: מספר זיהוי הנסיעה|שם המשתמש של הנוסע|סטטוס האישור של הנוסע|CHANGE_PASSENGER_STATUS_OF_RIDE
- :CHANE_USER_DETAILS
נשלחת מלקוח לשרת.
מבנה השדות בהודעה: הדבר שרוצים לשנות|השינוי|מה שהיה|שם המשתמש|CHANE_USER_DETAILS
- :SEND_EMAIL_WITH_CODE
נשלחת מלקוח לשרת.
מבנה השדות בהודעה: כתובה האימייל של המקבל|הקוד|SEND_EMAIL_WITH_CODE
- :USERS_ALREADY_RATED_SOME_USER
נשלחת מלקוח לשרת.
מבנה השדות בהודעה: שם המשתמש|USERS_ALREADY_RATED_SOME_USER
- :SET_RATE
נשלחת מלקוח לשרת.
מבנה השדות בהודעה: שם המשתמש|דירוג חדש|מדרג חדש|SET_RATE
- :LOGOUT
נשלחת מלקוח לשרת.
מבנה השדות בהודעה: LOGOUT
- :CONNECT → לאחר הרשמה:
נשלחת משרת ללקוח.
מבנה השדות בהודעה: CONNECT
- :CONNECT → לאחר התחברות:
נשלחת משרת ללקוח.
מבנה השדות בהודעה: מידע משתמש ממסד הנתונים|CONNECT
- :הסיסמה שגויה:
נשלחת משרת ללקוח.
מבנה השדות בהודעה: הסיסמה שגויה
- :האימייל לא קיים:
נשלחת משרת ללקוח.
מבנה השדות בהודעה: האימייל לא קיים
- :SAVE_RIDE
נשלחת משרת ללקוח.
מבנה השדות בהודעה: SAVE_RIDE
- :GET_RIDES
נשלחת משרת ללקוח.
מבנה השדות בהודעה: רשימת נסיעות ממסד הנתונים|GET_RIDES
- :There are no rides
נשלחת משרת ללקוח.
מבנה השדות בהודעה: There_are_no_rides
- :0 → אם אין נוסעים להסעה ספציפית:
נשלחת משרת ללקוח.
מבנה השדות בהודעה: 0
- :[:num_of_passengers[0]][0]
נשלחת משרת ללקוח.
מבנה השדות בהודעה: מספר נוסעים
- :נסיעה ראשונה:

יובל יוגב - Drive2Game

- נשלחת משרת ללקוח.
מבנה השדות בהודעה: נסיעה ראשונה
:rate_stars[0][0]
- נשלחת משרת ללקוח.
מבנה השדות בהודעה: דירוג נהג
:REQUEST_SEND
- נשלחת משרת ללקוח.
מבנה השדות בהודעה: REQUEST_SEND
:THE_PASSENGER_IS_DRIVER
- נשלחת משרת ללקוח.
מבנה השדות בהודעה: THE_PASSENGER_IS_DRIVER
:GET_NUM_OF_PASSENGERS
- נשלחת משרת ללקוח.
מבנה השדות בהודעה: מספר הנוסעים|GET_NUM_OF_PASSENGERS
:There_are_no_passengers
- נשלחת משרת ללקוח.
מבנה השדות בהודעה: There_are_no_passengers
:GET_STATUS
- נשלחת משרת ללקוח.
מבנה השדות בהודעה: סטטוס האישור של הנוסע|GET_STATUS
פרטי הנסיעה:
- נשלחת משרת ללקוח.
מבנה השדות בהודעה: שם המשתמש של הנהג|מיקום המשחק|מאיפה הנהג מתחיל לנסוע|הערות נסיעה|כמות נוסעים|אימייל של הנהג|שאר הנוסעים
:THE_STATUS_HAS_BEEN_CHANGED
- נשלחת משרת ללקוח.
מבנה השדות בהודעה: THE_STATUS_HAS_BEEN_CHANGED
:THE_DETAIL_HAS_BEEN_CHANGED
- נשלחת משרת ללקוח.
מבנה השדות בהודעה: THE_DETAIL_HAS_BEEN_CHANGED
:THE_CODE_HAS_BEEN_SENT
- נשלחת משרת ללקוח.
מבנה השדות בהודעה: THE_CODE_HAS_BEEN_SENT
:users_already_rated[0][0]
- נשלחת משרת ללקוח.
מבנה השדות בהודעה: users_already_rated[0][0]
:HAS_NOT_RATED_YET
- נשלחת משרת ללקוח.
מבנה השדות בהודעה: HAS_NOT_RATED_YET
:THE_RATE_HAS_BEEN_CHANGED
- נשלחת משרת ללקוח.
מבנה השדות בהודעה: THE_RATE_HAS_BEEN_CHANGED
:LOGOUT
- נשלחת משרת ללקוח.
מבנה השדות בהודעה: LOGOUT

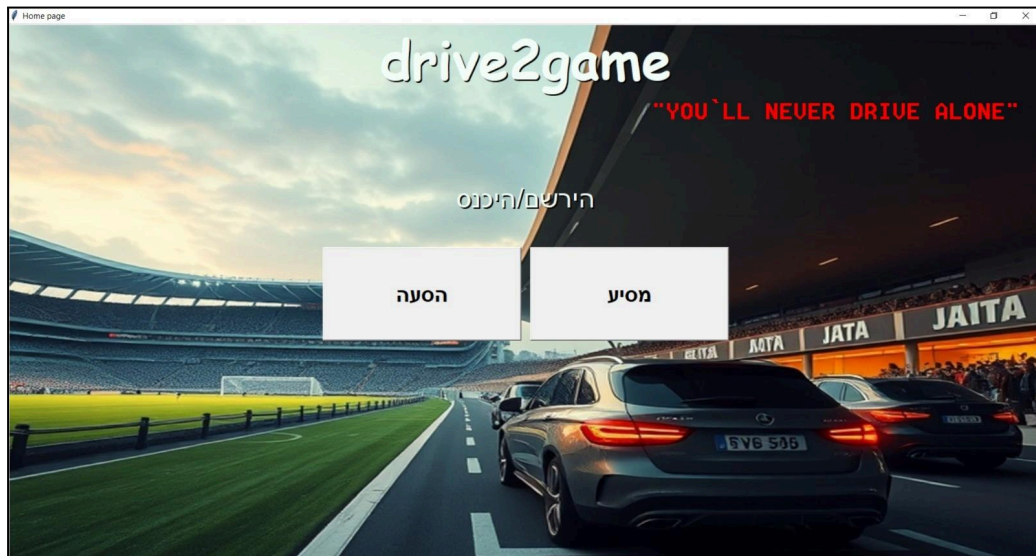
יובל יוגב - Drive2Game

תיאור מסכי המערכת

מסכים:

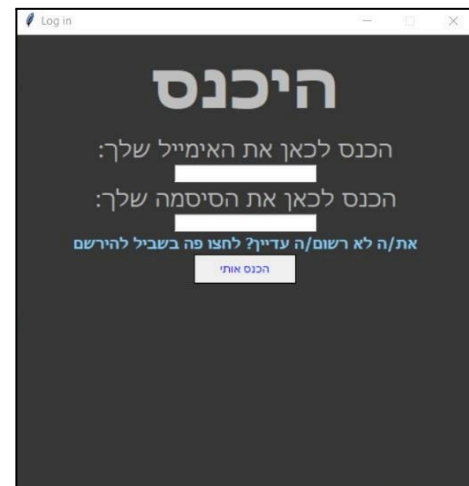
מסך בית לפני התחברות/הרשמה:

- **תיאור המסך:** המסך מציג תמונה, כותרת ושלושה כפתורים (הרשמה, מסיע, והסעה), לא משנה על איזה כפתור לוחצים נפתח רק דבר אחד ← מסך התחברות.
- **תמונת מסך:**



מסך התחברות:

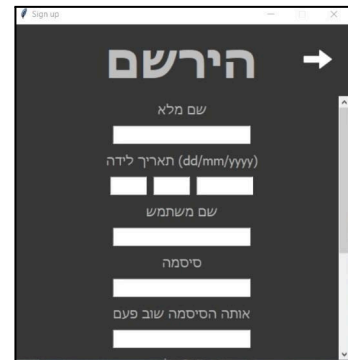
- **תיאור המסך:** המסך מציג שתי מקומות להכניס אליהם את הפרטים (אימייל וסיסמה), ושני כפתורים להתחברות או להרשמה.
- **תמונת מסך:**



יובל יוגב - Drive2Game

מסך הרשמה:

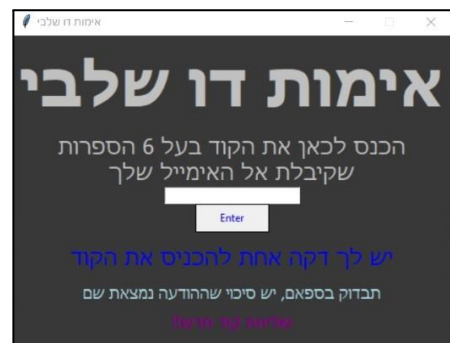
- **תיאור המסך:** המסך מציג שבעה מקומות להכניס אליהם את הפרטים (שם מלא, תאריך לידה, שם משתמש, סיסמה, אימייל, וטלפון אם יש אך זה לא חובה), וכפתור אחד להרשמה.
- **תמונת מסך:**



The screenshot shows a web browser window with the title 'הירשם' (Register). The page has a dark background with white text. It contains several input fields: 'שם מלא' (Full Name), 'תאריך לידה (dd/mm/yyyy)' (Date of Birth), 'שם משתמש' (Username), 'סיסמה' (Password), and 'אותה הסיסמה שוב פעם' (Repeat the same password). There is a right arrow button next to the title and a registration button at the bottom.

מסך אימות דו שלבי:

- **תיאור המסך:** המסך מציג מקום בו להכניס את קוד האימות החד פעמי, יש הסבר של איפה אפשר למצוא את האימייל אם לא רואים אותו.
- **תמונת מסך:**

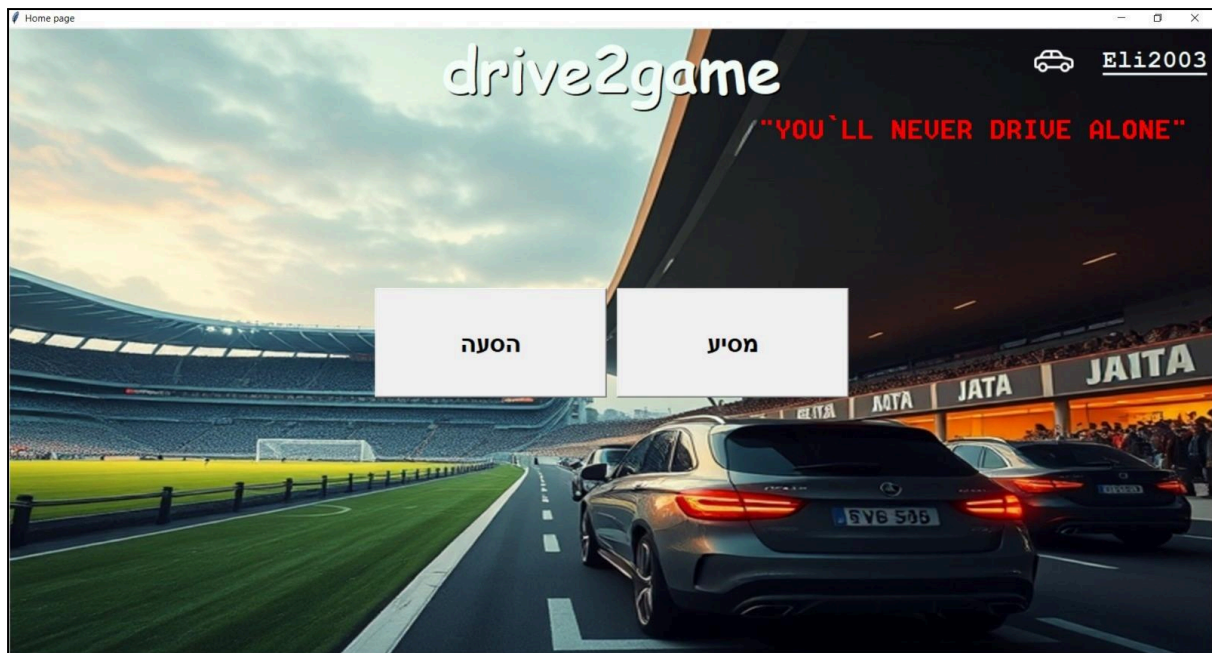


The screenshot shows a web browser window with the title 'אימות דו שלבי' (Two-step verification). The page has a dark background with white text. It contains a text input field for the verification code and an 'Enter' button. Below the input field, there is a blue link that says 'יש לך דקה אחת להכניס את הקוד' (You have one minute to enter the code). At the bottom, there is a note: 'תבדוק בספאם, יש סיכוי שרהודעה נמצאת שם' (Check spam, there is a chance the message is there) and a link 'שלח דואר חוזר' (Resend email).

יובל יוגב - Drive2Game

מסך בית לאחר התחברות/הרשמה:

- **תיאור המסך:** המסך מציג תמונה, כותרת וחמישה כפתורים (מסיע, הסעה, רשימת נסיעות, ציאת, ואזור אישי).
כפתור המסיע ← מסך של נהג.
כפתור הסעה ← מסך של נוסע.
כפתור של רשימת נסיעות ← מסך של רשימת נסיעות.
כפתור שם המשתמש ← תפריט של אזור אישי, דירוג כוכבים והתנתקות.
- **תמונת מסך:**



מסך של נהג #1 (בחירת משחק):

- **תיאור המסך:** המסך מציג listbox של משחקי ספורט, לחיצה על משחק ← מסך של נהג ליצירת נסיעה.
- **תמונת מסך:**



יובל יוגב - Drive2Game

מסך של נהג #2 (יצירת נסיעה):

- **תיאור המסך:** המסך מציג ארבעה מקומות להכניס אליהם את הפרטים (שם רחוב ומספר בית, יישוב, מספר אנשים שאפשר להסיע, והערות אם יש אך זה לא חובה), וכפתור ליצירת הנסיעה.
- **תמונת מסך:**

Driver Page

תיצור את ההסעה שלך

ההסעה למשחק 🎮 מכבי ת"א נגד הפועל עפולה

כתובת בה תתחיל הנסיעה

שם רחוב ומספר בית

יישוב

מספר נוסעים שיכולים להצטרף להסעה:

הערות:

כגון גיל מינימלי או מקסימלי לנוסע, נסיעה כרוכה בתשלום וכו...

תיצור את ההסעה

מסך של נוסע #1 (בחירת משחק):

- **תיאור המסך:** המסך מציג listBox של משחקי ספורט, לחיצה על משחק ← מסך של נוסע לבחירת הסעה.
- **תמונת מסך:**

Passenger Page

אנא בחר משחק:

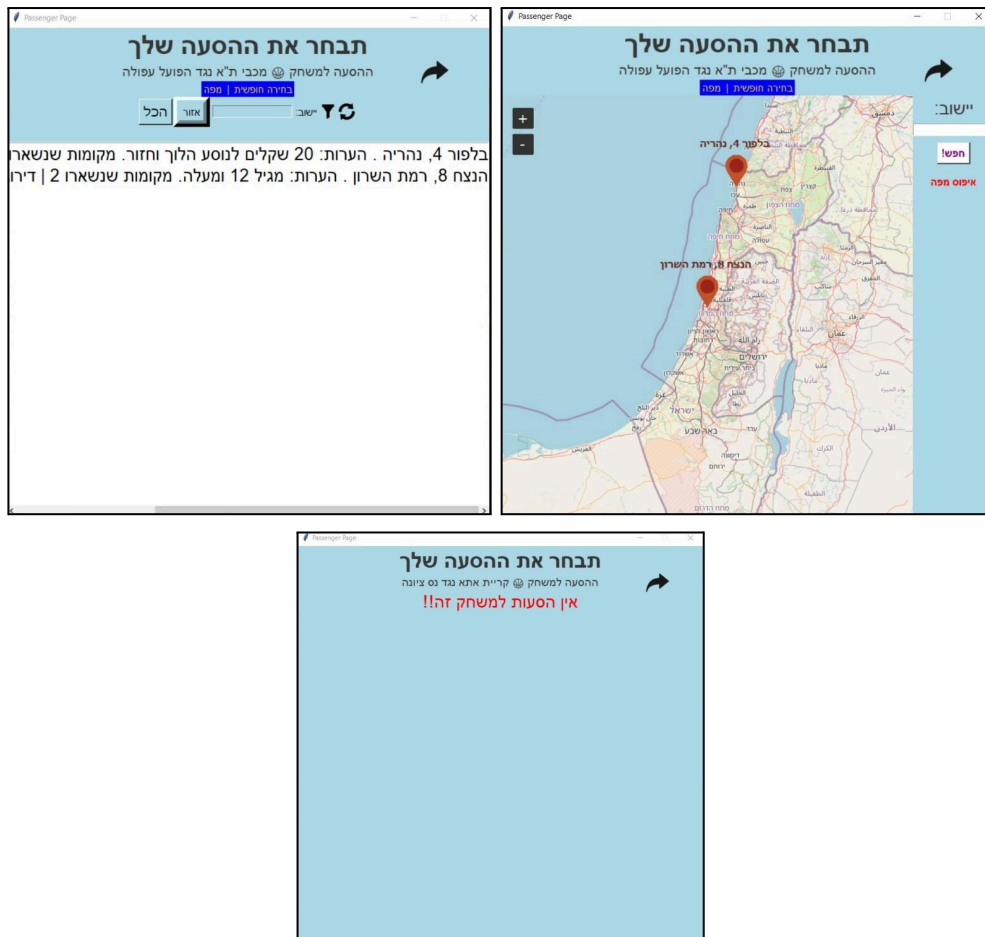
- הפועל ת"א נגד הפועל חולון, בתאריך: 08/03/2025, בשעה: 19:20, אולם היכל מנורה מבטחים, ת"א
- קריית אתא נגד נס ציונה, בתאריך: 09/03/2025, בשעה: 18:30, אולם רמז, קריית אתא
- הפועל חיפה נגד הפועל ב"ש, בתאריך: 09/03/2025, בשעה: 20:20, אולם רוממה
- גלבוע גליל נגד בני הרצליה, בתאריך: 09/03/2025, בשעה: 20:50, אולם גן נר
- אליצור נתניה נגד גליל עליון, בתאריך: 10/03/2025, בשעה: 18:30, אולם ארנה נתניה
- מכבי ת"א נגד הפועל עפולה, בתאריך: 10/03/2025, בשעה: 21:05, אולם היכל מנורה מבטחים
- נגד אליצור נתניה, בתאריך: , בשעה: לא ידוע, אולם לא ידוע
- הפועל חולון נגד גלבוע גליל, בתאריך: 16/03/2025, בשעה: 20:25, אולם היכל טוטו, חולון
- נס ציונה נגד מכבי ת"א, בתאריך: 16/03/2025, בשעה: 20:50, אולם לב המושבה, נס ציונה
- הפועל עפולה נגד מכבי רמת גן, בתאריך: 17/03/2025, בשעה: 18:40, אולם היכל הספורט העירוני, עפולה
- הפועל י-ם נגד הפועל חיפה, בתאריך: 17/03/2025, בשעה: 21:00, אולם פיס ארנה, ירושלים
- הפועל ב"ש נגד בני הרצליה, בתאריך: 18/03/2025, בשעה: 18:30, אולם הקוניה, באר שבע
- גליל עליון נגד קריית אתא, בתאריך: 18/03/2025, בשעה: 20:00, אולם כפר בלום
- נגד גלבוע גליל, בתאריך: , בשעה: לא ידוע, אולם לא ידוע
- מכבי רמת גן נגד גליל עליון, בתאריך: 23/03/2025, בשעה: לא ידוע, אולם לא ידוע
- הפועל עפולה נגד הפועל י-ם, בתאריך: 23/03/2025, בשעה: לא ידוע, אולם היכל הספורט העירוני, עפולה
- הפועל חיפה נגד נס ציונה, בתאריך: 23/03/2025, בשעה: לא ידוע, אולם רוממה
- מכבי ת"א נגד הפועל חולון, בתאריך: 23/03/2025, בשעה: לא ידוע, אולם לא ידוע
- קריית אתא נגד הפועל ב"ש, בתאריך: 23/03/2025, בשעה: לא ידוע, אולם לא ידוע
- בני הרצליה נגד הפועל ת"א, בתאריך: 23/03/2025, בשעה: לא ידוע, אולם לא ידוע

יובל יוגב - Drive2Game

מסך של נוסע #2 (בחירת הסעה):

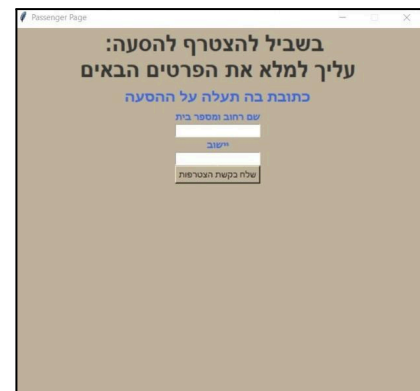
- **תיאור המסך:** אם קיימות הסעות ← בחירה בין רשימה של הסעות או בין מפה לראות את ההסעות. אם רשימה המסך מציג listbox של הסעות למשחק הנבחר + אפשרויות של סינון לאזור/יישוב וריפרש למסך, לחיצה על הסעה ← מסך של נוסע להצטרפות להסעה. אם מפה המסך מציג מפה עם מרקרים (נקודות מסומנות) בהם יש הסעות + אפשרות לבחור יישוב ספציפי ולעשות עליו זום. אם לא קיימות הסעות ← הודעה שאין הסעות קיימות למשחק זה.

• תמונות מסך:



מסך של נוסע #3 (הצטרפות להסעה):

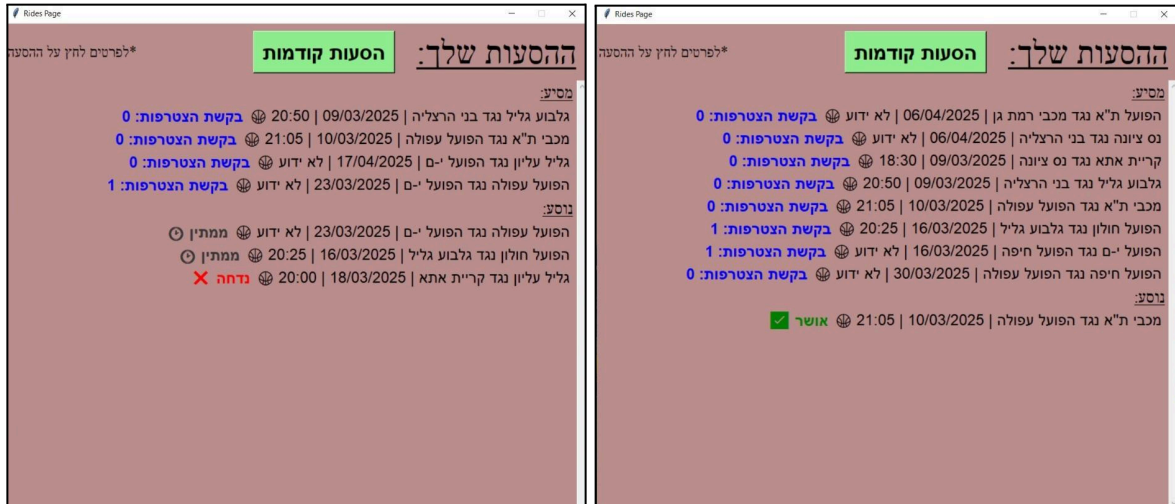
- **תיאור המסך:** המסך מציג שני מקומות להכניס אליהם את הפרטים (שם רחוב ומספר בית, יישוב), וכפתור לשליחת בקשת הצטרפות לנסיעה.
- **תמונות מסך:**



יובל יוגב - Drive2Game

מסך ההסעות של המשתמש:

- **תיאור המסך:** המסך מציג כפתור שממנו אפשר להגיע אל מסך ההסעות הקודמות שלא מהשתמש. ומציג את כל ההסעות העתידיות הזמינות של המשתמש, בתור מסיע ובתור נוסע מאושר/ממתין/נדחה.
- **תמונות מסך:**



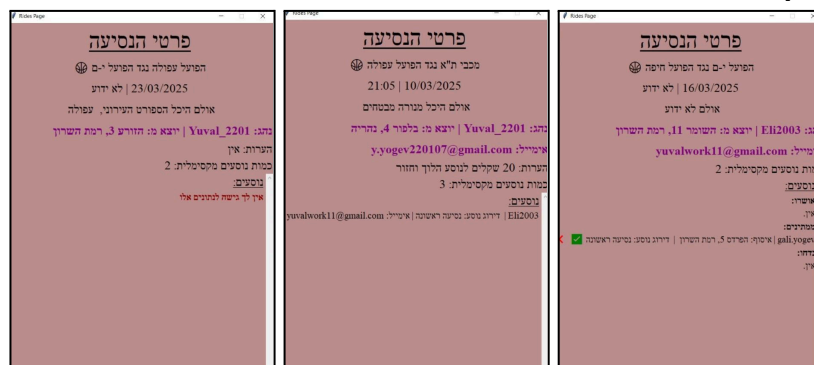
מסך הסעות קודמות של המשתמש:

- **תיאור המסך:** המסך מציג את כל ההסעות הקודמות של המשתמש, בתור מסיע ובתור נוסע.
- **תמונות מסך:**



מסך פרטי ההסעה:

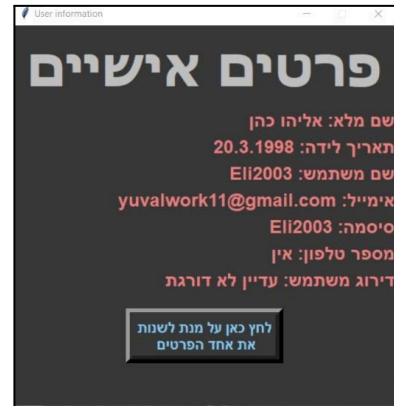
- **תיאור המסך:** המסך מציג את הפרטים של ההסעה, רואים מי זה הנהג ומאיפה הוא יוצא, את האימייל שלו. אותו הדבר גם הנהג יכול לראות על הנוסעים. בנוסף לנהג יש אפשרות לאשר או לדחות נוסע שרוצה להצטרף להסעה.
- **תמונות מסך:**



יובל יוגב - Drive2Game

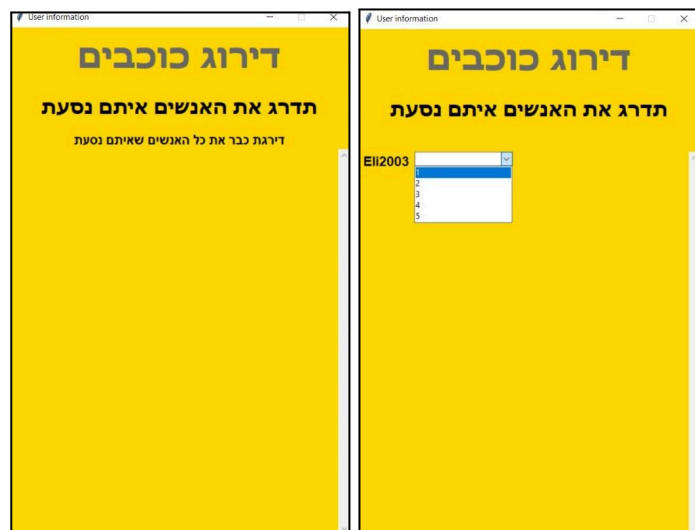
מסך הפרטים האישיים של המשתמש:

- **תיאור המסך:** המסך מציג את הפרטים האישיים של המשתמש, יש אפשרות ללחוץ על כפתור ולבחור לשנות את אחד הפרטים.
- **תמונת מסך:**



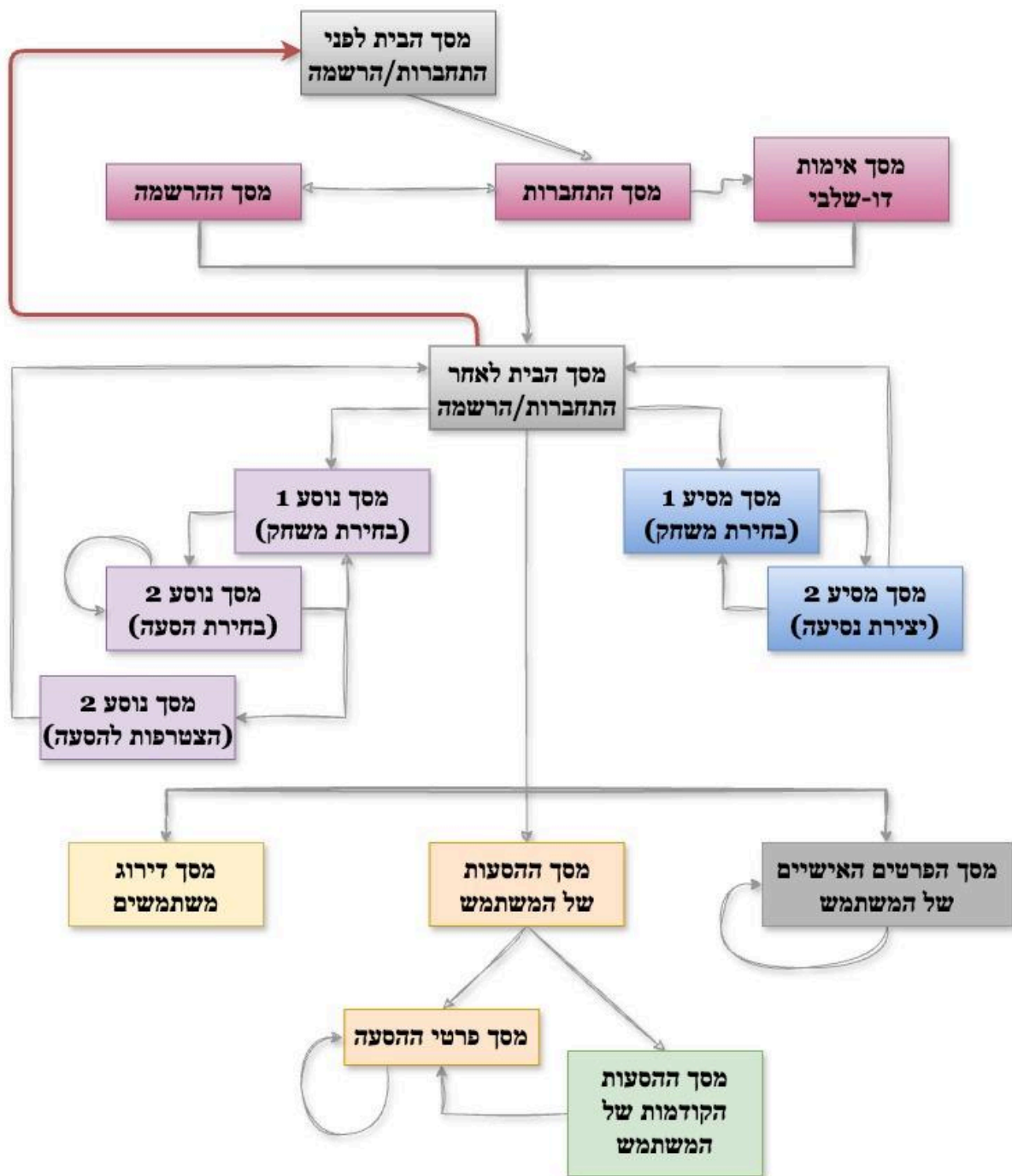
מסך דירוג משתמשים:

- **תיאור המסך:** אם המשתמש עדיין לא דירג את כל המשתמשים שהוא נסע איתם, המסך מציג לו את המשתמשים איתם הוא נסע ויכול לדרג + תיבת קומבו שנותנת לבחור את הדירוג מאחד עד חמש.
- **תמונות מסך:**



יובל יוגב - Drive2Game

תרשים מסכים המתאר את היררכיית המסכים והמעברים ביניהם:



****הערה:** מסך הבית תמיד נשאר פתוח ברקע. סגירת כל חלון אחר תחזיר את המשתמש למסך הבית, אלא אם המשתמש מזער אותו לסרגל המשימות.

תיאור מבני הנתונים

פירוט מבני הנתונים (מסד נתונים, קבצים, מקומיים וכו'):

- קבצי הקוד - graphics.py, Client_server.py, DB.py, Encryption.py, main.py
- קבצי תמונות וטקסט - cities_and_districts.txt, photo1.jpg, chat1.png, car1.png, back, arrow.png, funnel.png, reload.png, reply arrow.png
- קובץ html - מנהלת ליגת העל בכדורסל _ עונת 2024-25 _ משחקים ותוצאות.html

יובל יוגב - Drive2Game

• קובץ מסד נתונים - My_Project_DB.db

מסד נתונים (בסיס נתונים - database):

שם בסיס הנתונים - My_Project_DB

טבלאות:

טבלה מס' 1 - Users - אוגרת מידע של פרטים על המשתמשים במערכת:

מפתח ראשי - Email

שם השדה	תיאור	טיפוס השדה	מאפיינים	דוגמאות
Full_Name	השם המלא של המשתמש.	TEXT	NOT NULL	"Yuval Yogev" "יובל יוגב"
Birth_Date	תאריך הלידה של המשתמש, צריך להיות תאריך תקין.	TEXT	NOT NULL	"22.012007" "1.1.2000"
Username	שם המשתמש של המשתמש, צריך להיות באורך של בין 5-20 תווים ויכול לכלול רק אותיות, מספרים ו _ או . מתוים מיוחדים.	TEXT	NOT NULL UNIQUE	"Eli2003"
Password	הסיסמא של המשתמש, צריכה להיות באורך של 7-25 תווים ויכולה לכלול רק אותיות ומספרים.	TEXT	NOT NULL	"Eli2003"
Email	האימייל של המשתמש, עובר בדיקת תקינות.	TEXT	NOT NULL UNIQUE PRIMARY KEY	"y.yogev220107@gmail.com"
Phone_Number	מספר הטלפון של המשתמש (אם יש), לא עובר בדיקת תקינות רק צריך להיות בעל 10 ספרות.	TEXT	NULL	"0520009116"
Rate_stars	הדירוג הנוכחי של המשתמש נע בין 1-5 אך גם יכול להיות -1 אם זאת נסיעה ראשונה.	INTEGER	NOT NULL	0 -1

טבלה מס' 2 - Rides - אוגרת מידע של פרטים על נסיעות:

יובל יוגב - Drive2Game

מפתח ראשי - Username_driver & Game

שם השדה	תיאור	טיפוס השדה	מאפיינים	דוגמאות
Id_ride	מספר הזהות של הנסיעה.	TEXT	NOT NULL UNIQUE	"0" "18"
Username_driver	שם המשתמש של הנהג.	TEXT	NOT NULL UNIQUE PRIMARY KEY	"Eli2003"
Game	המשחק שאילו מיועדת הנסיעה.	TEXT	NOT NULL UNIQUE PRIMARY KEY	"הפועל י-ם נגד הפועל חיפה"
Date	התאריך של המשחק.	TEXT	NOT NULL	"09/03/2025"
Hour	השעה של המשחק.	TEXT	NOT NULL	"18:30"
Place	המיקום של המשחק.	TEXT	NOT NULL	"אולם זיסמן רמת גן" "אולם לא ידוע"
Start_ride_location	הכתובת בה הנהג מתחיל את הנסיעה.	TEXT	NOT NULL	"משה 1, הרצליה"
Num_passengers	מספר הנוסעים שיכולים להירשם לנסיעה.	INTEGER	NOT NULL	2 1
Comments	הערות (אם יש).	TEXT	NULL	"מעשן ברכב" "10 שקלים לכל נוסע"
Status	הסטטוס של הנסיעה, צריך להיות בין 0-3. 0 - הנסיעה עדיין לא קרתה. 1 - הנסיעה קרתה. 2 - הנסיעה התבטלה. 3 - הנסיעה מלאה.	INTEGER	NOT NULL CHECK(-1<"Status"<3)	0 3

טבלה מס' 3 Passengers - אוגרת מידע של פרטים על נוסעים:

מפתח ראשי - Username_passenger & Id_ride

יובל יוגב - Drive2Game

שם השדה	תיאור	טיפוס השדה	מאפיינים	דוגמאות
Username_passenger	שם המשתמש של הנוסע.	TEXT	NOT NULL UNIQUE PRIMARY KEY	"Eli2003"
Id_ride	מספר הזהות של הנסיעה.	TEXT	NOT NULL UNIQUE PRIMARY KEY	"0" "18"
Pick_up_location	הכתובת של מיקום האיסוף של הנוסע.	TEXT	NOT NULL	"בלפור 12, חיפה"
Approved	הסטטוס של אישור בקשת ההצטרפות להסעה, צריך להיות בין 0-2. 0 - עדיין לא אושר/ממתין. 1 - אושר. 2 - נדחה.	INTEGER	NOT NULL CHECK(-1<"Approved"<2)	0 1 2

טבלה מס' 4: Users_rate - אוגרת מידע של משתמשים שדורגו:
מפתח ראשי - Username

שם השדה	תיאור	טיפוס השדה	מאפיינים	דוגמאות
Username_passenger	שם משתמש.	TEXT	NOT NULL UNIQUE PRIMARY KEY	"Eli2003"
Total_rate	הסכום הכולל שהמשתמש דורג.	TEXT	NOT NULL	"1" "4"
How_many_rate?	כמה אנשים דירגו את המשתמש.	TEXT	NOT NULL	"1"
Who_rate?	איזה משתמשים דירגו את המשתמש.	TEXT	NOT NULL	"Eli2003" "Yuval2201"

יובל יוגב - Drive2Game

סקירת החולשות והאיומים

האיומים האפשריים:

שכבת האפליקציה -

איום: תוקף יכול להחדיר קוד SQL זדוני לשאילתות בסיס הנתונים שלך דרך טופסי קלט או פרמטרים של URL. זה יכול לאפשר לו לדלג על אימות, לקרוא נתונים רגישים, לשנות נתונים או אפילו להריץ פקודות מערכת הפעלה.

פתרון: שימוש בשאילתות פרמטריות במקום ליצור שאילתות SQL באופן דינמי על ידי שרשור מחרוזות, שימוש בשאילתות פרמטריות. זה מפריד בין הקוד לנתונים, כך שהמסד נתונים יודע תמיד להתייחס לקלט המשתמש כאל נתונים ולא כאל קוד SQL. הקוד שלי משתמש ב-`execute_query` עם סימני שאלה `?` כמחזיקי מקום, שזו צורה של שאילתות פרמטריות, ולכן הוא מוגן מפני SQL Injection.

איום: המערכת תלויה באתר חיצוני כדי לקבל מידע על משחקים. אם אתר זה הופך ללא זמין (עקב קריסה, תחזוקה, או חסימה), המערכת לא תוכל להציג את רשימת המשחקים למשתמשים. הדבר עלול לגרום לחוויית משתמש לקויה, לשגיאות באפליקציה, ובמקרים קיצוניים, לחוסר אפשרות להשתמש בחלקים מהמערכת.

פתרון: שימוש באתר שהוא אופליין, הורדתי את האתר לקובץ html על מנת שאני לא אאבד את הנתונים גם אם האתר קורס.

איום: ם תוקף משיג סיסמאות של משתמשים, הוא יכול לנסות להתחבר למערכת באמצעות האימייל והסיסמה הגנובים.

פתרון: בקוד יש שימוש בהגנת סייבר - אימות דו שלבי ולכן גם אם הפורץ ינסה לפרוץ כי הוא יודע את האימייל והסיסמה הוא צריך להיות מחובר לאימייל של המשתמש בשביל לקבל את קוד האימות.

יובל יוגב - Drive2Game

מימוש הפרויקט:

פירוט מימוש המערכת:

חלק א -

סקירת כל המחלקות המרכיבות את המערכת וקשרי הגומלין ביניהם:

• מחלקות מיובאים:

- tkinter: המחלקה מיועדת להפעלת הממשק הגרפי (האפליקציה).
- datetime: המחלקה מיועדת לבדוק אם התאריך הוא חוקי ובדיקה של גיל לפי תאריך.
- tkinter.font: המחלקה מיועדת לשימוש של פונטים מסויימים בממשק הגרפי.
- requests: שולחת בקשת HTTP מסוג GET לאתר לקבלת נתונים מהאינטרנט.
- bs4 - BeautifulSoup: המחלקה מיועדת לשימוש של ניתוח, עיבוד ושליטה מתוך קובץ טקסט בשפת HTML.
- re: המחלקה מאפשרת חיפוש והחלפת טקסט באמצעות ביטויים רגולריים.
- tkinter.messagebox: המחלקה מיועדת לשימוש בהודעות הקופצות למסך בממשק הגרפי.
- scrolledtext: המחלקה מיועדת לתת אפשרות לגלילת טקסט בממשק הגרפי.
- PIL - Image, ImageTk, ImageDraw: המחלקה מיועדת ליצירת תמונות ועריכתם.
- threading
- sqlite3: המחלקה מיועדת לשימוש במסד נתונים באמצעות שפת sql ב-Python.
- socket: המחלקה מיועדת להפעלת תקשורת רשת באמצעות סוקטיים.
- select: המחלקה מיועדת לתת לשרת גישה אל הלקוחות.
- tkintermapview: המחלקה מיועדת להצגת מפות בסגנון גרפי.
- opencage.geocoder: המחלקה מיועדת להפעיל API שמוצא קורדינטות על פי כתובת והפוך.
- tkinter.simpledialog: המחלקה מיועדת להקפצת הודעות למסך של הלקוח בזמן שהוא עושה משהו.
- import pyotp: מחלקה מיועדת להפעיל API שיוצר מספרים רנדומלים שהם סיסמה חד פעמית.

• מחלקות שאני פיתחתי:

- Database Manager: המחלקה מנהלת חיבור למסד נתונים SQLite, מאפשרת ביצוע שאלות, ושליפת נתונים. תומכת בשימוש בתחביר **with** לפתיחה וסגירה אוטומטית של החיבור, ומטפלת בשגיאות בצורה מאובטחת.

★ תכונות המחלקה:

שם	טיפוס
db_path	STRING
connection	sqlite3.Connection
cursor	sqlite3.Cursor

★ פעולות במחלקה:

שם	טענת כניסה	טענת יציאה
__init__	הפונקציה מקבלת את db_path - מחרוזת	הפונקציה מאתחלת את המחלקה ואת תכונות

יובל יוגב - Drive2Game

המחלקה.	שמייצגת את הנתבי לקובץ מסד הנתונים.	
הפונקציה מבטיחה שהאובייקט מחובר למסד נתונים תקין או מדווחת על שגיאה.	הפונקציה לא מקבלת פרמטרים.	connect
הפונקציה מבצעת שאילתת SQL על מסד הנתונים ומדווחת אם יש שגיאה.	הפונקציה מקבלת את query - שאילתא , params - מאפשר הכנסת ערכים דינמיים לשאילתות SQL.	execute_query
הפונקציה מחזירה את כל התוצאות מהשאילתה האחרונה שבוצעה, מדווחת אם יש שגיאה.	הפונקציה לא מקבלת פרמטרים.	fetchall
הפונקציה שומרת שינויים במסד הנתונים.	הפונקציה לא מקבלת פרמטרים.	commit
הפונקציה סוגרת את החיבור למסד הנתונים.	הפונקציה לא מקבלת פרמטרים.	close
הפונקציה מאפשרת שימוש ב-with לניהול החיבור באופן אוטומטי, פותחת את החיבור ומחזירה את האובייקט.	הפונקציה לא מקבלת פרמטרים.	__enter__
הפונקציה מאפשרת שימוש ב-with לניהול החיבור באופן אוטומטי, סוגרת את החיבור.	הפונקציה מקבלת את exc_type, exc_value, traceback הפרמטרים.	__exit__

- Client_server: המחלקה משמשת כמחלקת אם לשתי המחלקות Client ו-Server, על מנת הפעלת תקשורת. ★
תכונות המחלקה:

שם	טיפוס
ip	STRING
port	INT
e_Encryption	ENCRYPTION

★ פעולות במחלקה:

שם	טענת כניסה	טענת יציאה
__init__	הפונקציה מקבלת ip - פרמטר ששומר את כתובת	הפונקציה מאתחלת את המחלקה ואת תכונות

יובל יוגב - Drive2Game

המחלקה.	הip של השרת.	
---------	--------------	--

- Server: המחלקה יורשת מ-Client_server ומנהלת את פעולת השרת. היא מקבלת חיבורים מלקוחות ומנהלת תקשורת איתם באמצעות Sockets. בנוסף היא מאזינה להודעות נכנסות, מפענחת הודעות מוצפנות ושולחת תשובות בהתאם לפרוטוקול.

★ תכונות המחלקה:

שם	טיפוס
ip	STRING
port	INT
e_Encryption	ENCRYPTION
server_socket	SOCKET
rlist, wlist, xlist	LIST - SOCKET
clients	LIST - SOCKET
messages	LIST
date	STRING

★ פעולות במחלקה:

שם	טענת כניסה	טענת יציאה
__init__	הפונקציה מקבלת ip - פרמטר ששומר את כתובת הip של השרת.	הפונקציה מאתחלת את המחלקה ואת תכונות המחלקה.
start	הפונקציה לא מקבלת פרמטרים.	הפונקציה יוצרת סוקט, חיבור לכתובת ולפורט, והתחלת שמיעה על חיבורים נכנסים, ומפעילה את המתודה <u>Running_server</u> . אם יש שגיאה, היא מדפיסה את השגיאה.
running_server	הפונקציה לא מקבלת פרמטרים.	הפונקציה אחראית על ניהול הפעולה של השרת ומאזינה לחיבורים ולשליחת הודעות. והיא אף פעם לא מפסיקה כל עוד הקוד פועל.
send_Message_and_Protocol	הפונקציה מקבלת message - הודעה שהשרת קיבל, client - הלקוח שממנו התקבלה ההודעה.	הפונקציה מתפקדת על פי פרוטוקול שגשג מהלקוח, ומבצעת פעולות שונות בהתאם לסוג ההודעה. הפונקציה מבצעת פעולות

יובל יוגב - Drive2Game

שונות במסד הנתונים באמצעות מחלקת DB.Database_Manag er ומחזירה תשובות ללקוח לאחר ביצוע הפעולה. בנוסף, כל ההודעות הנשלחות מוצפנות.		
הפונקציה בודקת שהתאריכים בסדר ואם יש נסיעה במסד הנתונים שהתאריך שלה עבר, אם כן היא תסמן את הנסיעה כניסעה שכבר הייתה.	הפונקציה לא מקבלת נתונים.	check_the_dates

★ - Client: המחלקה יורשת מ-Client_Server ומנהלת את פעולת הלקוח. המחלקה
מאפשרת לשלוח ולקבל הודעות מהשרת, תוך שימוש בהצפנה.
★ תכונות המחלקה:

שם	טיפוס
ip	STRING
port	INT
e_Encryption	ENCRYPYION
client_socket	SOCKET

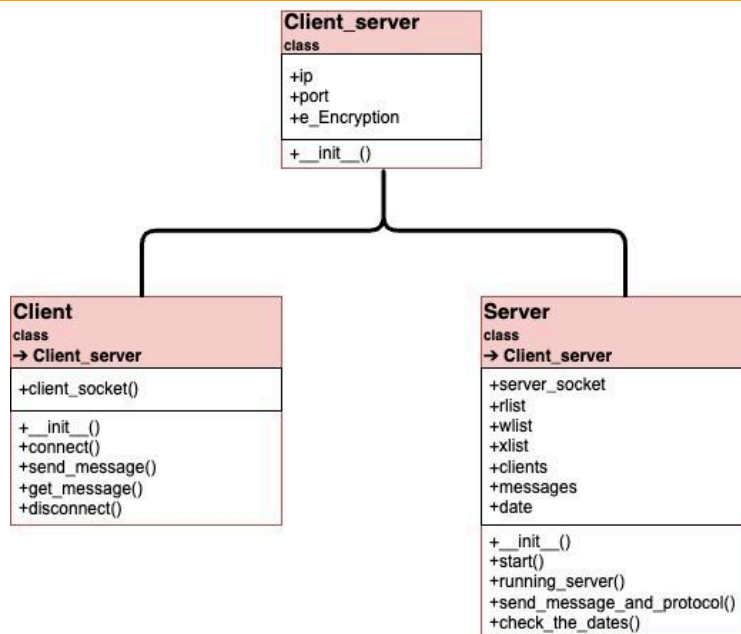
★ פעולות במחלקה:

שם	טענת כניסה	טענת יציאה
__init__	הפונקציה מקבלת ip - פרמטר ששומר את כתובת הip של השרת.	הפונקציה מאתחלת את המחלקה ואת תכונות המחלקה.
connect	הפונקציה לא מקבלת פרמטרים.	הפונקציה אחראית על התחברות הלקוח לשרת באמצעות כתובת ה-IP והפורט של השרת. אם התרחשה שגיאה במהלך החיבור, היא מדפיסה את הודעת השגיאה.
send_message	הפונקציה מקבלת את message - הודעה שהלקוח צריך לשלוח אל השרת.	הפונקציה שולחת הודעה אל השרת לאחר שהיא עוברת הצפנה, אם התרחשה שגיאה במהלך שליחת ההודעה, הפונקציה תדפיס את הודעת השגיאה.

יובל יוגב - Drive2Game

הפונקציה מקבלת הודעה מהשרת, מפענכת אותה ומחזירה אותה. במקרה של שגיאה במהלך קבלת ההודעה, הפונקציה מדפיסה את הודעת השגיאה, אך לא מחזירה שום דבר (None).	הפונקציה לא מקבלת פרמטרים.	get_message
אחראית לסגור את חיבור הסוקט של הלקוח.	הפונקציה לא מקבלת פרמטרים.	disconnect

תרשים UML של הירושה ממחלקת אב Client_server אל המחלקות Client & Server:



- Encryption: המחלקה מצפינה ומאפנכת הודעות באמצעות סוג הצפנה RSA. בנוסף, היא מאפשרת לעבד הודעות שמכילות תווים עבריים, על ידי המרתם לאותיות אנגליות, הצפנתם ופענוחם בעת הצורך.

★ תכונות המחלקה:

שם	טיפוס
pub_key_file	STRING
priv_key_file	STRING
block_size	INT
hebrew_to_english	DICTIONARY
english_to_hebrew	DICTIONARY

★ פעולות במחלקה:

יובל יוגב - Drive2Game

שם	טענת כניסה	טענת יציאה
__init__	הפונקציה לא מקבלת פרמטרים.	הפונקציה מאתחלת את המחלקה ואת תכונות המחלקה.
send_message	הפונקציה מקבלת את message - הודעה שצריך להצפין.	הפונקציה מצפינה הודעה עבור תווים רגילים והמרה של תווים עבריים לאותיות אנגליות. מחזירה מחרוזת חדשה שמייצגת את ההודעה המוצפנת.
get_message	הפונקציה מקבלת את message - הודעה שצריך לפענח, when_hebrew - רשימה שבה יש אינדקסים של תווים בעברית אם יש.	הפונקציה מפענכת את ההודעה המוצפנת ומחזירה מחרוזת מפוענחת שמייצגת את ההודעה המקורית לפני ההצפנה.
find_key	הפונקציה מקבלת את value - ערך מספרי שמציין תו.	הפונקציה מחזירה את המפתח של המילון self.my_dict שהערך שלו תואם לערך שנמסר כפרמטר.
read_key_file	הפונקציה מקבלת את שם הקובץ שממנו היא צריכה לקרוא.	הפונקציה מחזירה את שלושה משתנים בעלי ערך מספרי.
get_blocks_from_text	הפונקציה מקבלת משתנה מסוג מחרוזת שמכילה אך ורק תווים שניתן לקודד מסוג ASCII.	הפונקציה מחזירה רשימה של מספרים שלמים שכל מהם מייצג בלוק של בתים מהמחרוזת המקורית.
get_text_from_blocks	הפונקציה מקבלת את הערך המספרי של הבלוקים שמייצגים בתים מהמחרוזת, את אורך ההודעה, ואת גודל הבלוק.	הפונקציה מחזירה מחרוזת באורך של ההודעה שמתקבלת משחזור ה-ASCII של ההודעה המקורית על סמך הפענוח של המספרים ברשימת הבלוקים.
encrypt_message	הפונקציה מקבלת מחרוזת של תווים שניתנים לקידוד ASCII, ומפתח שאיתו תקרה ההצפנה.	הפונקציה מחזירה רשימה של מספרים שלמים שכל אחד מהם הוא בלוק מוצפן של ההודעה.
decrypt_message	הפונקציה מקבלת את הבלוקים המוצפנים, את אורך ההודעה, את המפתח שאיתו יהיה הפענוח, ואת גודל הבלוקים.	הפונקציה מחזירה מחרוזת באורך של ההודעה המקורית שהיא ההודעה המקורית לפני ההצפנה.

יובל יוגב - Drive2Game

חלק ב -

הבעיה - הצפנה ופענוח נתונים בעברית:

```
def send_message(self, message):
    key_size, n, e = self.read_key_file(self.pub_key_file)
    if key_size < self.block_size * 8:
        sys.exit('ERROR: Block size too large for size.')

    new_message = ""
    when_hebrew = []
    index = 0
    for letter in message:
        if not letter.isascii():
            when_hebrew.append(index)
            new_letter = self.hebrew_to_english[letter]
            new_message += new_letter
        else:
            new_message += letter
            index += 1

    message = new_message

    encrypted_blocks = self.encrypt_message(message, (n, e))
    encrypted_content = ','.join(map(str, encrypted_blocks))
    encrypted_content =
f'{len(message)} {self.block_size} {encrypted_content} {"", ".join(map(str,
when_hebrew))}'

    return encrypted_content
```

הטיפול בבעיה - שימוש במיפוי מותאם של תווים ושמירת אינדקסים של תווים עבריים כדי להבטיח שחזור תקין. המיפוי קורה בעזרת מילונים שיוצרים להעביר תווים מעברית לאנגלית, בכל פעם שיש תו בעברית נוסף ערך לרשימת התווים העבריים עם אינדקס של התו.

הבעיה - שליפת משחקים מקובץ HTML:

```
def Games_To_Go():
    with open("מנהלת ליגת העל בכדורסל - עונת 2024-25 משחקים ותוצאות.html",
              "r", encoding="utf-8") as file:
        soup = BeautifulSoup(file, "html.parser")

        text = soup.text # מטקסט html מעביר משפת

        # מאיפה עד איפה לקחת את המשחקים
        start_marker =
r"תוצאה\s*כרטיסים\s*אורחת\s*מארחת\s*משדר\s*ערוץ\s*אולם\s*שעה\s*תאריך"
        end_marker =
r"מארחת\s*משדר\s*ערוץ\s*אולם\s*שעה\s*תאריך\s*\d+\s*מחזור\s*שהסתיימו\s*משחקים"
r"תוצאה\s*סיקור\s*אורחת"

        pattern = re.compile(f'{start_marker}(.*?) {end_marker}', re.DOTALL)

        match = pattern.search(text) # מחפש את החלק הזה בטקסט
```

Drive2Game – יובל יוגב

```
if match:
    game_schedule = match.group(1).strip() # מנקה את הדברים הלא
קשורים מהטקסט

    # remove all the things I dont want
    for i in range(19, 27):
        new_pattern =
fr"מחזור\s*{i}\s*תאריך\s*שעה\s*אולם\s*ערוץ\s*משדר\s*מארחת\s*אורחת\s*כרטיסים\s*תוצאה"
s*תוצאה"

        game_schedule = re.sub(new_pattern, "", game_schedule)

    games = game_schedule.split("\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n") # בונה
רשימה של משחקים לפי החוקיות של הטקסט הרשום

    list_of_games = []

    for g in games:
        if g[0].isdigit() is False:
            g = g[1:]

        game = g.split('\n')

        if len(game[1].split(':')) == 1: game[1] += "אם # לא ידוע"
השעה עדיין לא רשומה

        if game[2] == ':': game[2] = "אם האולם עדיין לא"
ידוע

        list_of_games.append((game[9], game[-1], game[0][0:10],
game[1], "אולם " + game[2],
'🏀')) # (team home, team guest, date,
time, arena) בונה טאפל של הנתונים למשחק

    return list_of_games
else:
    print("Markers not found in the text.")
```

הטיפול בבעיה - שימוש ב- **BeautifulSoup** עם ביטויים רגולריים (**re**) לחיפוש בלוקים רלוונטיים.

השימוש בספריית **BeautifulSoup** נותן את קבלת העמוד הרצוי מאתר האינטרנט, לאחר מכן להורן אותה לשפה עברית/אנגלית ולא HTML. באמצעות ביטויים רגולריים הקוד מצליח למצוא את המשחקים בתוך הטקסט באתר.

מה שיפה כאן זה שהאתר יכול להמשיך להיות באינטרנט ובכל פעם שמפעילים את האתר אז זה בודק מחדש את המשחקים שנשארו - כיום בגלל בעיה שלא יהיו משחקים בעת הגשת הפרויקט אז הקוד עובד על אתר שהורדתי אותו (offline).

חלק ג - הבדיקות שתכננתי:

רישום והתחברות:
מטרת הבדיקה: לוודא שמשתמשים יכולים להירשם ולהתחבר למערכת בצורה תקינה.
מה בוצע בפועל: כאשר המשתמש ניסה להירשם הייתה בדקה על הפרטים אותם הוא הכניס, לדוגמה: אימייל ← בדקת תקינות של אימות לגבי האימייל, סיסמה ← בדקת תקינות של פורמט הסיסמה, שם

יובל יוגב - Drive2Game

משתמש ← בדיקת תקינות של פורמט שם המשתמש, תאריך לידה ← בדיקת תקינות של תאריך לידה לפי בדיקת תאריך שהתאריך באמת קיים.

כאשר המשתמש ניסה להתחבר הייתה בדיקה על הפרטים אותם הוא הזין, לדוגמה: אימייל ← בדיקה שיש אימייל כזה במערכת, סיסמה ← בדיקה שהסיסמה שהוזנה היא הסיסמה הנכונה שמחוברת במערכת אל האימייל המוזן.

מה היו תוצאות הבדיקה:

- אם המשתמש הכניס פרטים תקינים בנסיון ההרשמה ← הרשמה מוצלחת והמשך גלישה באפליקציה כאשר המשתמש מחובר לחשבון האישי.
- אם המשתמש הכניס פרטים לא תקינים בנסיון ההרשמה ← קופצות למסך בכתב אדום את הבעיות שאותם יש לתקן על מנת לבצע הרשמה.
- אם המשתמש הכניס פרטים תקינים בנסיון ההתחברות ← התחברות מוצלחת והמשך גלישה באפליקציה כאשר המשתמש מחובר לחשבון האישי.
- אם המשתמש הכניס פרטים לא תקינים בנסיון ההרשמה ← קופצות למסך בכתב אדום את הבעיות שאותם יש לתקן על מנת להתחבר.

יצירת נסיעה:

מטרת הבדיקה: לבדוק שמשתמשים יכולים להציע נסיעות עם הפרטים הנדרשים.

מה בוצע בפועל: כאשר המשתמש ניסה ליצור נסיעה הוא היה צריך להזין נתונים, לדוגמה: ציון מספר נוסעים ← בדיקה האם קיים מספר עם ספרות (פורמט תקין), הזמנת כתובת ← אם כתובת הוזנה.

מה היו תוצאות הבדיקה:

- אם המשתמש הכניס את הפרטים הנדרשים ← קופצת הודעה של ההסעה נשמרה ואפשר להמשיך לגלוש באפליקציה.
- אם המשתמש לא הכניסה את כל הפרטים הנדרשים ← קופצות למסך בכתב אדום את הבעיות שאותם יש לתקן או להוסיף על מנת ליצור את ההסעה.

חיפוש והצטרפות לנסיעה:

מטרת הבדיקה: לוודא שמשתמשים יכולים לחפש ולהצטרף לנסיעות קיימות.

מה בוצע בפועל: כאשר המשתמש חיפש נסיעות לעיר עם נסיעות זמינות ← בדיקה אם קיימות נסיעות זמינות לעיר ספציפית.

כאשר המשתמש ניסה להצטרף לנסיעה עם מקומות פנויים הוא היה צריך להזין כתובת שבה הוא עולה ← בדיקה שהמשתמש הכניס כתובת.

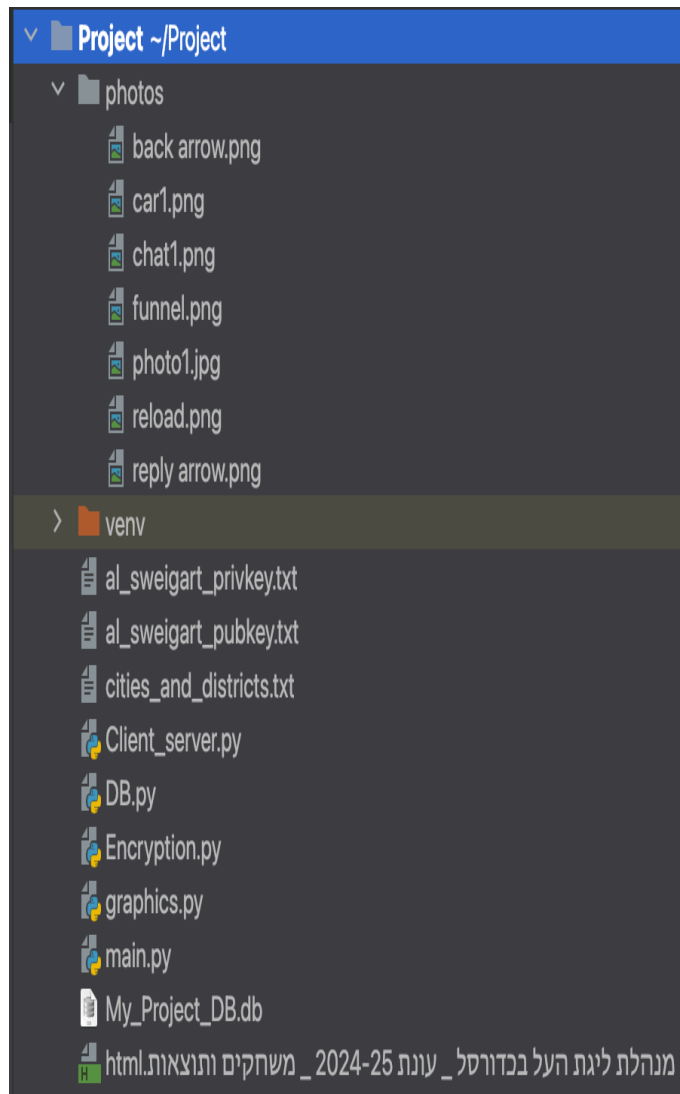
מה היו תוצאות הבדיקה:

- אם קיימות נסיעות זמינות לעיר ספציפית ← תוצג רשימה של הנסיעות הזמינות.
- אם לא קיימות נסיעות זמינות לעיר ספציפית ← תוצג הודעה שאין הסעות מהעיר הזאת למשחק הזה.
- אם המשתמש הכניס את הכתובת שהוא עולה בה ← קופצת הודעה למסך שבקשת ההצטרפות נשלחה.
- אם המשתמש לא הכניס את הכתובת שהוא עולה בה ← קופצת הודעה למסך בכתב אדום שיש להכניס כתובת.

Drive2Game – יובל יוגב

מדריך למשתמש:

עץ קבצים:



התקנת המערכת:

הסביבה הנדרשת: מחשב שישמש כשרת, מחשבים שיהיו הלקוחות ויתחברו לשרת. על כל המחשבים להיות מחוברים לרשת אינטרנט. רצוי שתהיה חזקה ככל האפשר על מנת שתוכל להריץ את הקבצים והתמונות ותקרא מהר נתונים מן האינטרנט.

בשביל שהמערכת תעבוד על המשתמש להוריד גרסת Python 3.11.

על המחשבים צריכים להיות מותקנים כל המודולים שציינתי פה:

PIL

bs4

re

tkintermapview

יובל יוגב - Drive2Game

opencage
pyotp

הסבר למנהל המערכת:

מנהל המערכת יהיה מי שיפעיל את השרת.

על המחשב של מנהל המערכת צריכים להיות הקבצים הבאים:

- main.py ◀
- DB.py ◀
- Encryption.py ◀
- Client_server.py ◀
- My_Project_DB.py ◀
- al_sweigart_privkey.txt ◀
- al_sweigart_pubkey.txt ◀

מנהל המערכת צריך לפתוח פרויקט שבו נמצאים כל הקבצים האלה, ולהיכנס אל הקובץ main.py. לאחר מכן לשנות את הקו של "השרת" לקו של המחשב של מנהל המערכת (מחשב בו נמצא השרת) ולהריץ את הקובץ.

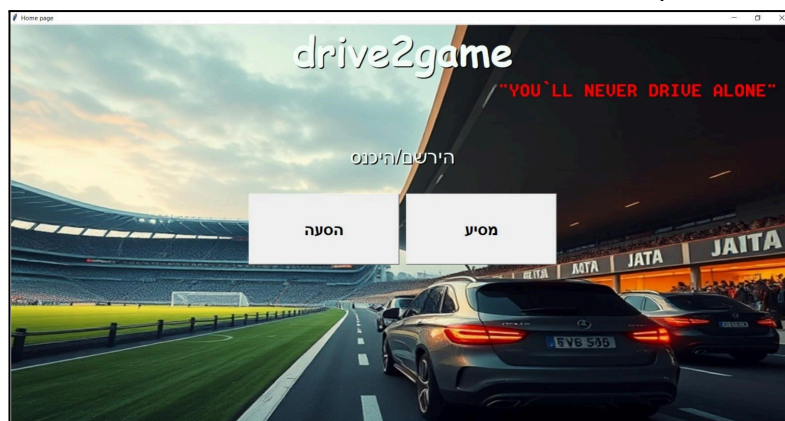
הסבר למשתמש:

על המחשב של מנהל המערכת צריכים להיות הקבצים הבאים:

- graphics.py ◀
- Encryption.py ◀
- Client_server.py ◀
- cities_and_districts.txt ◀
- מנהלת ליגת הכל בכדורסל_עונת 2024-25_משחקים ותוצאות.html ◀
- al_sweigart_privkey.txt ◀
- al_sweigart_pubkey.txt ◀
- photos - "back arrow.png", "car1.png", "chat1.png", "funnel.png", "photo1.png", ◀
- "reload.png", "reply arrow.png" ◀

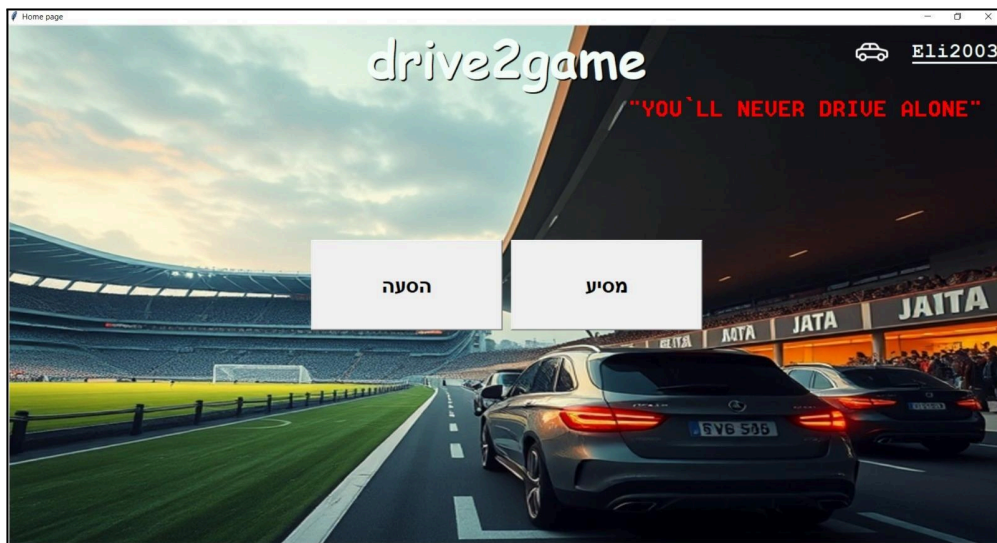
הלקוח צריך לפתוח את הפרויקט שבו נמצאים כל הקבצים האלה, ולהיכנס אל הקובץ graphics.py ולהריץ אותו.

לאחר הרצת התוכנית המסך הראשון שיוצג על המסך יהיה מסך הבית הראשוני. על כל כפתור שהמשתמש ילחץ יוצג לו המסך של להיכנס/להירשם. על המשתמש להיכנס או להירשם ואז הוא יוכל להשתמש בתוכנת האפליקציה.



יובל יוגב - Drive2Game

לאחר שהמשתמש נכנס/נרשם הוא יוכל להשתמש באתר.

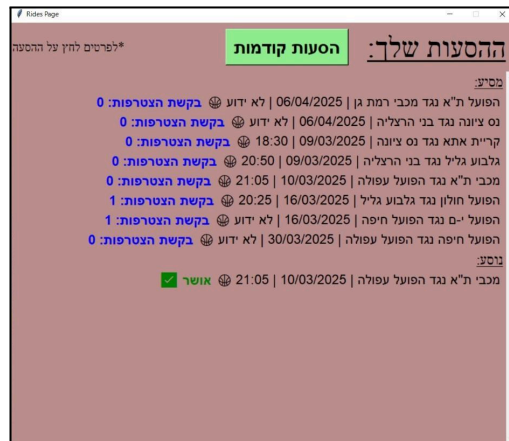


- **כפתור מסיע** - לחיצה על כפתור מסיע תפתח רשימה של משחקים, המשתמש צריך לבחור את המשחק שאליו הוא רוצה לנסוע ולאחר מכן יפתח לו מסך שבו הוא צריך להכניס נתונים בשביל ההסעה.

- **כפתור הסעה** - לחיצה על כפתור הסעה תפתח רשימה של משחקים, המשתמש צריך לבחור את המשחק שאליו הוא רוצה לנסוע ולאחר מכן יפתח לו מסך אם אין הסעות תהיה הודעה על המסך שאין הסעות למשחק הזה. אם יש הסעות אז המסך יפתח עם רשימה של ההסעות שיש למשחק הזה ואופציה לראות את ההסעות במפה. על המשתמש לבחור את ההסעה המתאימה לו, לאחר שהמשתמש יבחר הסעה יפתח לו מסך בו הוא יצטרך להכניס נתונים לגבי נקודת האיסוף שלו.

יובל יוגב - Drive2Game

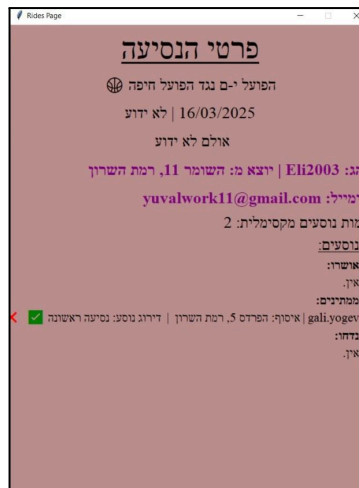
- **כפתור הרכב** - לחיצה על כפתור הרכב תפתח מסך עם כל ההסעות העתידיות שיש למשתמש, לחיצה על ההסעה תראה את פרטי ההסעה.
ליד כל הסעה רשום איזשהו פרט קטן כגון: למסיע יש כמה בקשות הצטרפות יש להסעה. לנוסע יש שלוש אפשרויות - אושר (המסיע אישר את בקשת ההצטרפות של הנוסע), ממתין (המסיע עדיין לא אישר או דחה את בקשת ההצטרפות של הנוסע), נדחה (המסיע דחה את בקשת ההצטרפות של הנוסע).



- **כפתור הסעות קודמות** - לחיצה על כפתור הסעות קודמות תפתח מסך דומה למסך של ההסעות העתידיות שיש למשתמש אבל יראה את ההסעות הקודמות של המשתמש. לחיצה על ההסעה תראה את פרטי ההסעה.

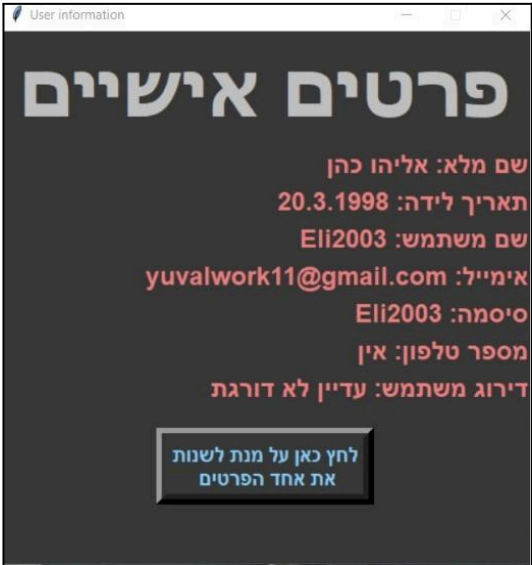


- **במסך פרטי ההסעה** - כאשר נוסע ממתין לחיצה על הכפתור הירוק (הוי) תאשר את הנוסע, לחיצה על הכפתור האדום (האיקס) תדחה את הנוסע.

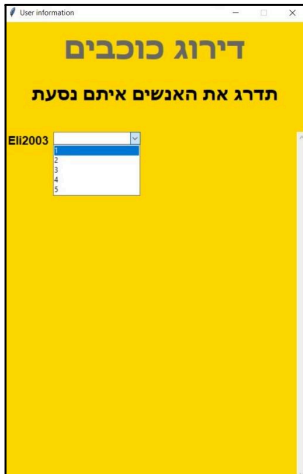


Drive2Game - יובל יוגב

- **כפתור השם משתמש** - לחיצה על כפתור השם משתמש תפתח תפריט בו המשתמש יוכל לבחור אחת משלוש אופציות - פרטי המשתמש, דירוג כוכבים והתנתקות.
- **פרטי המשתמש** - לחיצה על פרטי המשתמש תפתח מסך בו המשתמש יוכל לראות את הפרטים האישיים שלו והוא יוכל גם לשנות חלק מהם.



- **דירוג כוכבים** - לחיצה על דירוג כוכבים תפתח מסך בו המשתמש יוכל לראות את המשתמשים האחרים איתו הוא נסע והוא יוכל לדרג אותם.



- **התנתקות** - לחיצה על התנתקות תנתק את המשתמש ותחזור למסך הבית הראשון שבו צריך להיכנס/להירדם.

יובל יוגב - Drive2Game

רפלקציה / סיכום אישי:

תיאור תהליך העבודה

תחילת העבודה:

תחילה לפני שהתחלתי לעשות את הפרויקט חקרתי קצת וניסיתי להבין מה אני באמת רוצה לעשות בפרויקט שלי. המון מחשבות רצו לי בראש ובסוף לאחר שיחה עם ההורים הבנתי שאני רוצה לעשות משהו שיש בו צורך.

בהתחלה בגלל שאמא שלי דיילת חשבנו אולי על איזה פלטפורמה שתהיה לדיילים שבה הם יוכלו להחליף טיסות. אבל לאחר חשיבה מעמיקה הבנתי שאני לא באמת מכיר את התחום הזה של דיילות ואני צריך משהו שיותר מתאים לי.

באותה התקופה בדיוק קניתי את המנוי הראשון בחיי לקבוצה שאותה אני אוהד - הפועל ירושלים. הנסיעות מרמת השרון לירושלים כל שבוע/שבועיים הם קשות והתחלתי לנסוע מדי פעם בהסעות מאורגנות של אוהדי הקבוצה שנוסעים באוטובוס. ולפתע ברגע אחד הבנתי מה אני באמת רוצה לעשות בפרויקט. פלטפורמה לאוהדים שבה חלק יוכלו להציע הסעות וחלק יוכלו להצטרף להסעות בתור נוסעים. כמה שבועות לאחר שרשמתי לעצמי מה הנושא של הפרויקט התחלתי קצת לרשום למה יש בזה צורך ולמען מה תהיה הפלטפורמה הזאת. בנוסף רשמתי כל מיני שאלות שאני שואל את עצמי על מה יהיה בפרויקט ואז הבנתי גם מה הפיצרים שצריכים להיות בפרויקט והתחלתי לבנות את העבודה שלי.

במהלך העבודה:

במהלך העבודה עבדתי בחלקים, הצבתי לעצמי מטרות שאותם אני רוצה לממש וממש כמו פאזל התחלתי להשלים את הפרויקט. - בשונה מפאזל, אני לא התחלתי מהמסגרת אלא מהאמצע. מהליבה של הפרויקט. הבנתי שהשני דברים החשובים ביותר בפרויקט שלי שבלעדיהם בעצם הפרויקט לא עושה את העבודה שלו זה אחד - נהג יוצר הסעה. שתיים - נוסע מוצא הסעה.

סיום העבודה:

בסיום העבודה על הפרויקט כבר התחיל קושי של הבנה של זה בסוף אז צריך להשקיע והבנה של אין זמן אז צריך להשקיע זמן בשביל להספיק לסיים כמה שיותר. בימים האחרונים אפילו כמעט ולא ישנתי, הלזז היה מאוד צפוף והבנתי שבשביל לתת 100% מעצמי אני צריך לעבוד גם בלילות...

לאחר סיום הפרויקט, הרגשתי מן הקלה אך מלווה בתחושת פספוס קלה. אני מאוד שמח מהתוצר הסופי שלי אבל היו עוד פיצרים ודברים שבהתחלה סימנתי אותם בתור יעד ולא הצלחתי להגיע אליהם, למשל צאט בין הנהג והנוסעים בהסעה מכיוון שלא הספקתי אז הסתפקתי רק בלרשום את האימייל של כל אחד בהסעה בשביל שכן תהיה להם דרך תקשורת. גם תכננתי שתהיה אפשרות ביטול של הסעה ועוד כל מיני דברים שרשומים בהצעה לפרויקט.

תהליך הלמידה

תהליך הלמידה העצמי לא היה חלק גדול במיוחד בפרויקט, התבססתי בעיקר על חומר הלמידה שלמדנו בבית ספר השנה ושנה שעברה. אם זאת בהחלט היו דברים שלמדתי בעצמי גם בתור חומר חדש וגם דברים מהחומר הלימודי אך שלא נלמדו מספיק טוב.

יובל יוגב - Drive2Game

תיאור תכולות שנלמדו באופן עצמי:

- הצפנה ופענוח של תווים בעברית - הצפנת RSA לא תמכה בתווים בעברית שהעברתי בהודעות בין הלקוח לשרת ולכן הייתי צריך ללמוד בעצמי איך לעשות זאת.
- אימות דו שלבי- למדתי איך להשתמש באימות דו שלבי על מנת שיהיה קשה יותר להתחבר למשתמשים בתוכנה. הלמידה הביאה איתה לימוד על שליחת מיילים באמצעות קוד פייטון ויצירת קוד חד פעמי לשליחה.

תובנות מהתהליך ובראייה לאחור

התובנות שאני לוקח איתי להמשך החיים הם לחשוב קדימה ולנסות לנצל את הזמן בדברים חשובים ולא בסתם דברים שנראים טובים. בזבזתי לא מעט שעות על הפרויקט בדברים שבסוף מחקתי כי הם לא היו נראים מספיק טוב או עבדו - אלה דברים שלא הצבתי לעצמי ביעדים. בראייה לאחור הייתי משנה את tkinter שלי לcustom tkinter כי רואים את ההבדל. בנוסף הייתי אולי מנסה ממש לסדר את הזמן שלי ולדבר עם המורים שיעזרו לי בתכנון הזמן - אני חושב שגם עצם בחוסר ידיעה של איך הפרויקט בכלל אמור להראות גרם לזה שלא ידעתי איך, מתי וממה להתחיל.

תודות

אני רוצה להודות קודם כל לצוות המורים בסייבר שנתנו יד ומענה לבעיות הפרויקט שלי, בזכותם הצלחתי להתגבר על בעיות שהיו במהלך השנה. אני אשמח גם להודעות להוריי שנתנו לי תמיכה מלאה בפרויקט וניסו לעזור במעט יכולתם, אמי עזרה לי רבות עם מסד הנתונים ו-sql ובנוסף הם היו שפני הנסיונות שלי שעליהם בדקתי את הפרויקט לאורך הדרך. רוצה להודות גם לחבריי הקרובים וחבריי לכיתה שעזרו לי בחלקים שונים בפרויקט וגם בחשיבה. אני מבקש להודות גם לקהילת האוהדים בארץ, שנטלה חלק בסקר שערכתי במסגרת הפרויקט. במהלך העבודה הבנתי את הצורך בנתונים וסטטיסטיקות מדויקים יותר על הלך הרוח בקרב האוהדים, והתגובות הרבות והכנות שקיבלתי סייעו לי רבות בהעמקת ההבנה על הצורך האמיתי של אוהדים בפלטפורמה כזאת.

תשובות הסקר

מהסקר שנערך בקהילת האוהדים בישראל עולה כי 55.7% מהאוהדים מגיעים למשחקים ברכב פרטי, בעוד 51.3% משתמשים בתחבורה ציבורית, 16.5% מגיעים ב-טרמפ ו-5.2% ב-הסעות מאורגנות. למרות הגידול בשימוש בתחבורה ציבורית, הנתונים מצביעים על כך שחלק גדול מהאוהדים עדיין בוחר ברכב הפרטי — בחירה שתורמת לעומסי תנועה, בעיות חניה ונזק סביבתי. דווקא הסעות משותפות וטרמפים מציעים איזון מיטבי: גם נוחות וגם חיסכון. בנוסף 70.4% מהאוהדים שענו על הסקר ענו שקרה להם אי פעם שלא היה להם איך להגיע למשחק - הסעות משותפות יכולות לפתור בעיה של אוהדים להגיע למשחקים. יחד עם זאת עוד נתון מהסקר הוא ש73.7% מהעונים על הסקר טוענים שהם היו מסכימים לנסוע עם אדם אחר למשחק.

משפט עם משמעות

המשפט שרשום בדף הבית של הפרויקט - **you'll never drive alone** הוא משפט עם משמעות כפולה. המשמעות הראשונה היא שבזכות הפלטפורמה אוהדים אל יצטרכו לנסוע לבד אלא עם עוד אנשים בהסעות. המשמעות השנייה היא שהמשפט מאוד דומה למשפט "you'll never walk alone" מזוהה עם כל מיני אוהדים של קבוצות בעולם, במיוחד עם האוהדים של ליברפול.

יובל יוגב - Drive2Game

ביבליוגרפיה:

- ☒ <https://stackoverflow.com/questions> - עזר לי ברוב הנושאים הנלמדים למעט שימוש במסד נתונים בשפת sql.
- ☒ <https://www.python.org> - עזר לי בנושאים הבאים: מחלקות - ירושה, ממשק גרפי - tkinter, מערכת הפעלה - threads.
- ☒ <https://www.youtube.com> - עזר לי בשימוש של API, נושא למידה חדש כמו שליחת email, אימות דו שלבי וממשק גרפי - tkinter.
- ☒ <https://chatgpt.com> - עזר לי בחלקים קטנים בקוד כמו הצפנה, מערכות הפעלה - threads ובעיקר בממשק גרפי - tkinter.
- ☒ <https://www.lamed-oti.com> האתר של המורה שלימד הצפנה וממשק גרפי ולכן זה עזר לי בהצפנה ובממשק גרפי - tkinter.
- ☒ <https://github.com> עזר לי בממשק גרפי - tkinter.
- ☒ <https://basket.co.il> מהאתר הזה לקחתי את כל המשחקים.

יובל יוגב - Drive2Game

נספחים:

תיעוד הקוד

main.py

```
import Client_server
```

```
if __name__ == "__main__":
```

```
    #main
```

```
    server = Client_server.Server("192.168.1.214") #לקחת את האחד של ה10
```

```
    server.start() # Initializes and starts the server.
```

```
    server.running_server()
```

graphics.py

```
import tkinter as tk
```

```
from PIL import Image, ImageTk, ImageDraw
```

```
from datetime import datetime
```

```
import tkinter.font as tkFont
```

```
from bs4 import BeautifulSoup
```

```
import re
```

```
from tkinter.messagebox import *
```

```
import Client_server
```

```
from tkinter import scrolledtext
```

```
from tkintermapview import TkinterMapView
```

```
from opencage.geocoder import OpenCageGeocode
```

```
import tkinter.simpledialog as simpledialog
```

```
import pyotp
```

```
from tkinter import ttk
```

```
import threading
```

```
def create_client():
```

```
    global client
```

```
    client = Client_server.Client("192.168.1.214") #לקחת את האחד של ה10
```

```
    client.connect() # Initializes and connects the client to the server.
```

```
def crop_to_circle(image_path, size):
```

```
    # פונקציה שמקבלת את נתיב התמונה ואת הגודל שאליו ליצור את התמונה והפונקציה מחזירה תמונה  
    בצורת עיגול
```

```
    circle_image = create_Photos(image_path, size, True, False)
```

```
    # יוצר תמונה חדשה ריקה עם רקע שקוף
```

```
    mask = Image.new("L", size, 0)
```

```
    draw = ImageDraw.Draw(mask)
```

```
    # מצייר עיגול לבן על התמונה הריקה שנוצרה
```

```
    draw.ellipse((0, 0, size[0], size[1]), fill=255)
```

יובל יוגב - Drive2Game

```
# להדביק את התמונה הרצויה על התמונה הריקה עם הציור עיגול
circular_image = Image.new("RGBA", size)
circular_image.paste(circle_image, (0, 0), mask)
return circular_image

def create_Photos(path, size=0, resize=False, tk_format=True):
    # הפונקציה מקבלת נתיב של התמונה, גודל (אם לא נשלח זה 0) ואם לא נשלח גודל אז גם לא נשלח
    שינוי התמונה וגם לא נשלח הפורמט, הפונקציה מחזירה את התמונה שנוצרה
    image = Image.open(path) #Open the image
    if resize:
        image = image.resize(size) #resize it to the user screen size
    if tk_format is False:
        return image
    return ImageTk.PhotoImage(image) #return the image in Tkinter format

def on_enter(event):
    event.widget.config(cursor="hand2")

def on_leave(event):
    event.widget.config(cursor="")

def home_Page():
    # פונקציה של דף הבית שבעצם היא הפונקציה הראשונה שמריצה גרפיקה בקוד והיא קוראת לשאר
    הפונקציות
    global client
    if client is None: # אם עדיין לא קיים לקוח ואין חיבור בין השרת אל הלקוח אז זה יחבר בניהם
        create_client()

    global root

    screen_width = root.winfo_screenwidth() # רוחב המסך של המשתמש
    screen_height = root.winfo_screenheight() # אורך המסך של המשתמש

    root.geometry(f'{screen_width}x{screen_height}') #Set the screen size to match the user's
    display
    root.title("Home page") #title of the window

    # יוצר קנבס
    canvas = tk.Canvas(root, width=screen_width, height=screen_height)
    canvas.pack(fill="both", expand=True)

    # Add the background image
    try:
        background_picture = create_Photos("photos/photo1.jpg", (screen_width,
        screen_height), True)
        # Add the image to the canvas
        canvas.create_image(0, 0, image=background_picture, anchor="nw")
```

Drive2Game - יובל יוגב

```
except FileNotFoundError:
    print(f"The image 'photo1.jpg' has not found")

#Add a shadow effect to the title
canvas.create_text(screen_width / 2 + 2, 50 + 2, text="drive2game", font=("Comic Sans
MS", 60, "bold"), fill="black")
#Add title
canvas.create_text(screen_width / 2, 50, text="drive2game", font=("Comic Sans MS", 60,
"bold"), fill="mint cream")

#Add a label of great sentence
canvas.create_text(screen_width * 0.8, screen_height * 0.15, text="YOU`LL NEVER
DRIVE ALONE", font=(list(tkFont.families())[2], 25, "bold"), fill="Red", justify=tk.RIGHT)

global connect

global list_of_games
list_of_games = None
if connect is not None:
    list_of_games = games_to_go() #Get the list of games, it does not matter if you
passenger or driver, the list is needed

if type(connect) is str:
    # מסדר את הנתונים של המשתמש
    connect = connect.replace('(', ').replace(')', '(').replace('""', '').split(',')
    for i in range(1, 7):
        connect[i] = connect[i].replace(' ', '')
    connect = tuple(connect)

#Add a username label
username_connect_label = canvas.create_text(screen_width * 0.95, screen_height *
0.05, text=connect[2], font=(list(tkFont.families())[7], 24, "bold underline"), fill="mint cream")
canvas.tag_bind(username_connect_label, "<Enter>", on_enter) # אם עומדים על הלייבל
זה מפעיל את הפונקציה שתראה את הסמן שזה כפתור
canvas.tag_bind(username_connect_label, "<Leave>", on_leave) # תבטל את פעולת הסמן
אם לא עומדים על הלייבל
canvas.tag_bind(username_connect_label, "<Button-1>", user_menu()) # אם הלייבל נלחץ
זה יפעיל את הפונקציה שמראה את התפריט של המשתמש

#Add a car icon
try:
    car1 = create_Photos("photos/car1.png", (50, 50), True)
    car1_display = canvas.create_image(screen_width * 0.85, screen_height * 0.05,
image=car1, anchor="w")

    canvas.tag_bind(car1_display, "<Enter>", on_enter) # אם עומדים על התמונה זה מפעיל
את הפונקציה שתראה את הסמן שזה כפתור
```


יובל יוגב - Drive2Game

תבטל את פעולת הסמן אם לא # canvas.tag_bind(car1_display, "<Leave>", on_leave)
עומדים על התמונה

canvas.tag_bind(car1_display, "<Button-1>", lambda event: your_rides_window()) #
אם התמונה נלחצה זה יפעיל את הפונקציה שמראה את כל הנסיעות הפעילות של המשתמש
except FileNotFoundError:
print(f"The image 'car1.png' has not found")

b_passenger = passenger if connect != None else sign_up_or_log_in

#Add "Passenger" / "הסעה" button
passenger_button = tk.Button(root, text="הסעה", font=("Arial", 24, "bold"), width=15,
height=3, cursor="hand2")
passenger_button.place(relx=0.4, rely=0.5, anchor="center")

passenger_button.bind("<Button-1>", lambda event: b_passenger(event, list_of_games))
אם הכפתור נלחץ זה יפעיל את אחת מן הפונקציות או להירשם או לבחור הסעה

b_driver = driver if connect != None else sign_up_or_log_in

#Add "Driver" / "מסיע" button
driver_button = tk.Button(root, text="מסיע", font=("Arial", 24, "bold"), width=15, height=3,
cursor="hand2")
driver_button.place(relx=0.6, rely=0.5, anchor="center")
driver_button.bind("<Button-1>", lambda event: b_driver(event, list_of_games)) # אם
הכפתור נלחץ זה יפעיל את אחת מן הפונקציות או להירשם או ליצור הסעה

signup_login_text = "הירשם/היכנס"
signup_login_text = "" if connect != None else signup_login_text
b_signup_login = None if connect != None else sign_up_or_log_in

#Add a shadow effect to the "Sign up/Log in" label
canvas.create_text(screen_width / 2 + 2, screen_height * 0.3 + 2, text=signup_login_text,
font=("Open Sans", 30), fill="black")
Add "Sign up/Log in" label and place it in the center of the screen
signup_login_label = canvas.create_text(screen_width / 2, screen_height * 0.3,
text=signup_login_text, font=("Open Sans", 30), fill="mint cream")

canvas.tag_bind(signup_login_label, "<Enter>", on_enter) # אם עומדים על הלייבל זה מפעיל
את הפונקציה שתראה את הסמן שזה כפתור
canvas.tag_bind(signup_login_label, "<Leave>", on_leave) # תבטל את פעולת הסמן אם לא
עומדים על הלייבל

Bind the label to the event using tag_bind
canvas.tag_bind(signup_login_label, "<Button-1>", b_signup_login) # אם הלייבל נלחץ זה
יפעיל את הפונקציה של להתחבר או להירשם

יובל יוגב - Drive2Game

פעולה לסגירת החלון — ניתוק # root.protocol("WM_DELETE_WINDOW", user_disconnect)
המשתמש

```
root.mainloop() # Run the main loop
```

```
def sign_up_or_log_in(event=None, not_important=None):
```

```
    # הפונקציה מקבלת אירוע (לא תמיד יש) ועוד איזה פרמטר שאני לא משתמש בו ולכן הוא לא חשוב.  
    הפונקציה מציגה גרפית מסך של כניסה או הרשמה בעת הצורך
```

```
    global top_level
```

```
    top_level = tk.Toplevel() #Open a new window
```

```
    top_level.resizable(False, False) # Disable resizing the window
```

```
    # Create window size 500x500 and centered
```

```
    width, height = 500, 500
```

```
    x = (top_level.winfo_screenwidth() - width) // 2
```

```
    y = (top_level.winfo_screenheight() - height) // 2
```

```
    top_level.geometry(f'{width}x{height}+{x}+{y}')
```

```
    top_level.configure(bg="gray22") #Fill the background in dark gray color
```

```
    top_level.title("Log in") #title of the window
```

```
    label = tk.Label(top_level, text="הכנס", font=("Tahoma", 50, "bold"), bg="gray22",  
fg="silver")
```

```
    label.pack(side="top", pady=10)
```

```
    #Add a box for log in email
```

```
    email_login_label = tk.Label(top_level, text="הכנס לכאן את האימייל שלך:", font=("Tahoma",  
18), bg="gray22", fg="silver")
```

```
    email_login_label.pack()
```

```
    email_login = tk.Entry(top_level, width=25)
```

```
    email_login.pack()
```

```
    #Add a box for log in password
```

```
    password_login_label = tk.Label(top_level, text="הכנס לכאן את הסיסמה שלך:",  
font=("Tahoma", 18), bg="gray22", fg="silver")
```

```
    password_login_label.pack()
```

```
    password_login = tk.Entry(top_level, width=25, show="**")
```

```
    password_login.pack()
```

```
    #Add a label button for sign up
```

```
    signup_label = tk.Label(top_level, text="את/ה לא רשום/ה עדיין? לחצו פה בשביל להירשם",  
font=("Tahoma", 12, "bold"), bg="gray22", fg="light sky blue", cursor="hand2")
```

```
    signup_label.pack()
```

```
    signup_label.bind("<Button-1>", sign_up)
```

יובל יוגב - Drive2Game

```
button = tk.Button(top_level, text="הכנס אותי", fg="blue", relief="solid", borderwidth=1,
padx=25, pady=5, cursor="hand2", command= lambda: check_login(email_login.get(),
password_login.get(), errors_login_label))
button.pack()

global e_login
e_login = ""
# Errors
errors_login_label = tk.Label(top_level, text="", font=("Tahoma", 14), bg="gray22",
fg="red")
errors_login_label.pack(pady=5)

top_level.mainloop() # Run the main loop

def sign_up(event):
    # פונקציה המקבלת אירוע ומציגה גרפית מסך הרשמה
    global top_level
    top_level.destroy() # הורסת את המסך הקודם (סוגרת אותו)
    top_level = tk.Toplevel() # פותחת מסך חדש

    top_level.resizable(False, False) # Disable resizing the window
    # Create window size 500x500 and centered
    width, height = 500, 500
    x = (top_level.winfo_screenwidth() - width) // 2
    y = (top_level.winfo_screenheight() - height) // 2
    top_level.geometry(f'{width}x{height}+{x}+{y}')

    top_level.configure(bg="gray22") # Fill the background in dark gray color
    top_level.title("Sign up") # title of the window

    label = tk.Label(top_level, text="הירשם", font=("Tahoma", 50, "bold"), bg="gray22",
fg="silver")
    label.pack(side="top", pady=10)

    # Add a back arrow icon
    try:
        back_arrow_photo = create_Photos("photos/back arrow.png", (50, 50), True)
        back_arrow = tk.Label(top_level, image=back_arrow_photo, bg="gray22",
cursor="hand2")
        back_arrow.photo = back_arrow_photo
        back_arrow.place(x= width * 0.85, y= height * 0.05)

        back_arrow.bind("<Button-1>", back_Arrow_clicked)
    except FileNotFoundError:
        print(f"The image 'back arrow.png' has not found")

    # יצירת קנבס בשביל סרגל גלילה
```

יובל יוגב - Drive2Game

```
canvas_of_sign_up_page = tk.Canvas(top_level, bg="gray22", highlightthickness=0)
scrollbar = tk.Scrollbar(top_level, orient="vertical",
command=canvas_of_sign_up_page.yview)

scrollbar_frame = tk.Frame(canvas_of_sign_up_page, bg="gray22")

scrollbar_frame.bind("<Configure>", lambda e:
canvas_of_sign_up_page.configure(scrollregion=canvas_of_sign_up_page.bbox("all")))

canvas_of_sign_up_page.create_window((250, 0), window=scrollbar_frame, anchor="n")
canvas_of_sign_up_page.configure(yscrollcommand=scrollbar.set)

canvas_of_sign_up_page.pack(side="left", fill="both", expand=True)
scrollbar.pack(side="right", fill="y")

scrollbar_frame.columnconfigure(0, weight=1)

# full name Label and Entry
name_label = tk.Label(scrollbar_frame, text="שם מלא", font=("Tahoma", 14), bg="gray22",
fg="silver")
name_label.pack(pady=5)
name_entry = tk.Entry(scrollbar_frame, font=("Tahoma", 14))
name_entry.pack(pady=5)

# Birth Date Label and Entry
birth_date_label = tk.Label(scrollbar_frame, text="תאריך לידה (dd/mm/yyyy)",
font=("Tahoma", 14), bg="gray22", fg="silver")
birth_date_label.pack(pady=5)

# Frame for birth date entries
birth_date_frame = tk.Frame(scrollbar_frame, bg="gray22")
birth_date_frame.pack(pady=5)

# Display the birth date entries inside the frame
birth_day = tk.Entry(birth_date_frame, font=("Tahoma", 14), width=5)
birth_day.pack(side="left", padx=5)
birth_month = tk.Entry(birth_date_frame, font=("Tahoma", 14), width=5)
birth_month.pack(side="left", padx=5)
birth_year = tk.Entry(birth_date_frame, font=("Tahoma", 14), width=8)
birth_year.pack(side="left", padx=5)

# Username Label and Entry
username_label = tk.Label(scrollbar_frame, text="שם משתמש", font=("Tahoma", 14),
bg="gray22", fg="silver")
username_label.pack(pady=5)
username_entry = tk.Entry(scrollbar_frame, font=("Tahoma", 14))
username_entry.pack(pady=5)
```

Drive2Game - יובל יוגב

```
# Password Label and Entry
password_label = tk.Label(scrollbar_frame, text="סיסמה", font=("Tahoma", 14),
bg="gray22", fg="silver")
password_label.pack(pady=5)
password_entry = tk.Entry(scrollbar_frame, show='*', font=("Tahoma", 14)) # 'show' hides
the password
password_entry.pack(pady=5)

# Again Password Label and Entry
password2_label = tk.Label(scrollbar_frame, text="אותה הסיסמה שוב פעם", font=("Tahoma",
14), bg="gray22", fg="silver")
password2_label.pack(pady=5)
password2_entry = tk.Entry(scrollbar_frame, show='*', font=("Tahoma", 14)) # 'show'
hides the password
password2_entry.pack(pady=5)

# Email Label and Entry
email_label = tk.Label(scrollbar_frame, text="אימייל", font=("Tahoma", 14), bg="gray22",
fg="silver")
email_label.pack(pady=5)
email_entry = tk.Entry(scrollbar_frame, font=("Tahoma", 14))
email_entry.pack(pady=5)

# Phone number Label and Entry
phone_num_label = tk.Label(scrollbar_frame, text="מספר טלפון", font=("Tahoma", 14),
bg="gray22", fg="silver")
phone_num_label.pack(pady=5)
phone_num_entry = tk.Entry(scrollbar_frame, font=("Tahoma", 14))
phone_num_entry.pack(pady=5)

# Sign Up Button
sign_up_button = tk.Button(scrollbar_frame, text="תרושום אותי", font=("Tahoma", 14),
cursor="hand2", command=lambda: signUp_check_and_saveDB(name_entry.get(),
birth_day.get(), birth_month.get(), birth_year.get(), username_entry.get(),
password_entry.get(), password2_entry.get(), email_entry.get(), phone_num_entry.get(),
errors_signup_label))
sign_up_button.pack(pady=20)

global e_signup
e_signup = ""
# Errors
errors_signup_label = tk.Label(scrollbar_frame, text="", font=("Tahoma", 14), bg="gray22",
fg="red")
errors_signup_label.pack(pady=5)
```

יובל יוגב - Drive2Game

```
def signUp_check_and_saveDB(name_entry, birth_day, birth_month, birth_year,
username_entry, password_entry, password2_entry, email_entry, phone_num_entry,
errors_signup_label):
    # הפונקציה מקבלת נתונים שהם נתוני המשתמש שאותם צריך לבדוק ולהכניס אל מסד הנתונים
    global e_signup
    e_signup = "" #Every time the user try to sign up the errors need to reset

    #Check if the date is valid
    try:
        datetime(int(birth_year), int(birth_month), int(birth_day))
    except ValueError as e:
        e = str(e)
        if e.split(' ')[0] == "day": e = "היום לא נמצא בחודש, תנסה חודש או יום אחר"
        if e.split(' ')[0] == "month": e = "החודש לא נמצא, תנסה חודש אחר בין 1 - 12"
        if e.split(' ')[0] == "invalid": e = "תנסה שוב פעם" #אם התאריך הוא ריק או עם תווים ולא עם"
מספרים
        e_signup += f"תאריך אינו תקין: {e}\n"
    finally:
        birth_date = f"{birth_day}.{birth_month}.{birth_year}" #Save the birth date of the user as
a string of dd.mm.yyyy

    #Check if the username is fine
    if len(username_entry) < 5: e_signup += "חשם המשתמש צריך לכלול לפחות 5 תווים"
    elif len(username_entry) > 20: e_signup += "חשם המשתמש יכול להיות רק עד 20 תווים"
    elif not username_entry.isalnum() and not all(c in '._' for c in username_entry if not
c.isalnum()): e_signup += "חשם המשתמש יכול לכלול רק אותיות, מספרים ו _ או . מתווים מיוחדים"

    #Check if the password is fine
    if len(password_entry) < 7: e_signup += "חלסיסמה צריכה לכלול לפחות 7 תווים"
    elif len(password_entry) > 25: e_signup += "חלסיסמה יכולה להיות רק עד 25 תווים"
    elif not password_entry.isalnum() and not all(c in " for c in password_entry if not
c.isalnum()): e_signup += "חלסיסמה יכולה לכלול רק אותיות ומספרים"
    #Check if the two passwords are same
    elif password_entry != password2_entry: e_signup += "חלסיסמאות צריכות להיות אותו הדבר"

    # Check if the user enter the email
    if '@' not in email_entry: e_signup += "ח'יש להכניס כתובת מייל"

    user_values = (name_entry.replace(' ', '+'), birth_date, username_entry, password_entry,
email_entry, phone_num_entry, "-1")

    #Check if there is no error
    if e_signup == "":
        try:
            global client
            client.send_message(f"{user_values}|SIGNUP") #send message to the server that
Collect user_values and SIGNUP
```

יובל יוגב - Drive2Game

```
e_signup = client.get_message()
except Exception as e:
    print(e)

if e_signup != "CONNECT": errors_signup_label.config(text=e_signup) #Update the label
with error messages
else:
    top_level.destroy()
    global connect
    connect = user_values # Mark user as connected
    global root
    root.destroy() # (סגירת) הבית
    root = tk.Tk() # יצירה חדשה של מסך הבית
    home_Page() # Reload home page

def check_login(email_login, password_login, errors_login_label):
    # הפונקציה נתונים שהם נתוני ההתחברות של המשתמש, היא בודקת אותם ואם הם טובים היא מתחילה
    את תהליך ההכנסה של המשתמש
    global e_login
    e_login = "" #Every time the user try to sign up the errors need to reset

    global client
    # אפשרה של העתק הדבק
    if "\n" in email_login:
        email_login = email_login.replace("\n", "")
    if "\n" in password_login:
        password_login = password_login.replace("\n", "")
    # שליחת הודעה אל השרת שיסתכל על נתוני ההתחברות של המשתמש
    client.send_message(f'{email_login}{password_login}|LOGIN')
    e_login = client.get_message()
    e_login = e_login.split('|')

    global top_level

    if e_login[-1] == "CONNECT":
        if two_way_factor_authentication(email_login) is True:
            top_level.destroy()
            global connect
            connect = e_login[0]
            global root
            root.destroy() # סגירת מסך הבית
            root = tk.Tk() # יצירת מסך הבית
            home_Page() # Reload home page
        else:
            showerror("שגיאה", "הקוד שגוי, תנסה להיכנס שוב פעם")
            top_level.focus_force() # מחזיר את הפוקוס לחלון
```

יובל יוגב - Drive2Game

else:

```
# Update the label with error messages
errors_login_label.config(text=e_login[0])
```

```
def two_way_factor_authentication(email):
```

```
#Page for 2 way factor...
```

```
result = tk.BooleanVar() # משתנה שיחזיק את התוצאה הסופית
```

```
two_way_factor_page = tk.Toplevel() # Open a new window
```

```
two_way_factor_page.resizable(False, False) # Disable resizing the window
```

```
# Create window size 500x350 and centered
```

```
width, height = 500, 350
```

```
x = (two_way_factor_page.winfo_screenwidth() - width) // 2
```

```
y = (two_way_factor_page.winfo_screenheight() - height) // 2
```

```
two_way_factor_page.geometry(f"{width}x{height}+{x}+{y}")
```

```
two_way_factor_page.configure(bg="gray22") # Fill the background in dark gray color
```

```
two_way_factor_page.title("אימות דו שלבי") # title of the window
```

```
key_of_pyotp = "NeuralNineMySuperSecretKey" # מפתח ל-API
```

```
totp = pyotp.TOTP(key_of_pyotp, interval=60) # יצירת אובייקט של סיסמה חד פעמית עם תוקף
```

```
code = tk.StringVar() # שמירה על הקוד
```

```
def send_code(event=None):
```

```
current_code = totp.now() # יצירת קוד עדכני
```

```
code.set(current_code) # שמירת הקוד במשתנה
```

```
def send():
```

```
try:
```

```
client.send_message(f"{email}|{current_code}|SEND_EMAIL_WITH_CODE")
```

```
client.get_message() # אין צורך בפלט
```

```
except Exception as e:
```

```
print(f"שגיאה בשליחת הקוד במייל: {e}")
```

```
threading.Thread(target=send).start()
```

```
send_code()
```

```
def check_2_way():
```

```
input_code = input_code_entry.get() # קבלת הקוד מהמשתמש
```

```
if "\n" in input_code:
```

```
input_code = input_code.replace("\n", "") # הסרת תווי ירידת שורה אם יש בגלל העתק הדבק
```

```
if totp.verify(input_code, valid_window=1):
```

```
result.set(True) # נשמרה תוצאה חיובית
```


יובל יוגב - Drive2Game

```
else:
    result.set(False) # נשמרה תוצאה שלילית
    two_way_factor_page.destroy()

tk.Label(two_way_factor_page, text="אימות דו שלבי", font=("Tahoma", 50, "bold"),
bg="gray22", fg="silver").pack(side="top", pady=10)

tk.Label(two_way_factor_page, text="שקיבלת אל האימייל\הכנס לכאן את הקוד בעל 6 הספרות", font=("Tahoma", 18),
bg="gray22", fg="silver").pack(side="top", pady=10)
input_code_entry = tk.Entry(two_way_factor_page, width=25)
input_code_entry.pack(side="top", pady=10)

tk.Button(two_way_factor_page, text="Enter", fg="blue", relief="solid", borderwidth=1,
padx=25, pady=5, cursor="hand2", command=check_2_way).pack(side="top", pady=10)

tk.Label(two_way_factor_page, text="יש לך דקה אחת להכניס את הקוד", font=("Tahoma", 18),
bg="gray22", fg="blue").pack(side="top", pady=10)
tk.Label(two_way_factor_page, text="תבדוק בספאם, יש סיכוי שההודעה נמצאת שם", font=("Tahoma", 13),
bg="gray22", fg="light blue").pack(side="top", pady=10)

resend_code = tk.Label(two_way_factor_page, text="שליחת קוד חדש!!!", font=("Ariel", 14,
"bold"), bg="gray22", fg="purple", cursor="hand2")
resend_code.pack(pady=5)
resend_code.bind("<Button-1>", send_code)

two_way_factor_page.grab_set()
two_way_factor_page.wait_variable(result) # מחכה לתוצאה
return result.get() # מחזיר אם התוצאה נכונה או לא

def get_cities(region=None):
    #this function put all the cities from the text file, in tuple
    cities = []
    with open("cities_and_districts.txt", "r", encoding="utf-8") as f: #open the text file and
make it only for reading, also encoding the text in the file for reading it
        lines = f.readlines() #save all the lines in the file on the value "lines"
        for line in lines:
            c_r = line.split("|") #split the line to 2 parts, city and region
            city = c_r[0]
            r = c_r[1].replace("\n", "")
            if region is not None:
                if r == region:
                    cities.append(city)
            else:
                cities.append(city)

    return tuple(cities) #return all the cities in tuple
```

יובל יוגב - Drive2Game

```
def checkkey_in_entry_address(event, main_page):
    # של כתובת עיר entry הפונקציה בודקת אם נלחץ כפתור על
    global lb, city_frame, city_driving_frame, city_frame_2

    value = event.widget.get()

    #get data from l
    if value == "":
        data = cities
    else:
        data = []
        for item in cities:
            if value.lower() in item.lower():
                data.append(item)

    #update data in listbox
    update_cities_listbox(data)

    widget = event.widget # משיג את הווידגט של האינבט
    # של הפריים root משיג את ה
    parent = widget
    while True:
        parent = parent.master
        parent_name = parent.winfo_name()
        if parent_name == "passenger_page_1" or parent_name == "passenger_page_2" or
parent_name == "driver_page":
            break
    root = parent

    widgets = [root] #root מהווידגטים
    widgets_name = [root.winfo_name()] #root מהווידגטים
    for child in root.winfo_children():
        try:
            child.pack_info() # אני רוצה רק את הווידגטים שמוצגים
            widgets.append(child)
            widgets_name.append(child.winfo_name())
        except:
            pass

    # Show listbox when typing starts
    if value != "":
        if main_page == "select_drive":
            for i in range(len(widgets_name)):
                if widgets_name[i] == main_page and (i + 1) < len(widgets_name):
                    city_frame.pack(after=widgets[i], before=widgets[i + 1])
                    lb.pack(side="top") # Make Listbox visible when typing
```

יובל יוגב - Drive2Game

```
elif main_page == "join_to_ride":
    for i in range(len(widgets_name)):
        if widgets_name[i] == main_page and (i + 1) < len(widgets_name):
            city_frame_2.pack(after=widgets[i], before=widgets[i + 1])
            lb.pack(side="top") # Make Listbox visible when typing
    else:
        for i in range(len(widgets_name)):
            if widgets_name[i] == main_page and (i + 1) < len(widgets_name):
                city_driving_frame.pack(after=widgets[i], before=widgets[i + 1])
                lb.pack(side="top") # Make Listbox visible when typing
elif value == "":
    lb.pack_forget() # Hide Listbox when Entry is empty
    if main_page == "select_drive": city_frame.pack_forget()
    elif main_page == "join_to_ride": city_frame_2.pack_forget()
    else: city_driving_frame.pack_forget()

def update_cities_listbox(data):
    # פונקציה שמקבלת נתונים של ערים והפונקציה מעדכנת את הליסטבוקס
    global lb

    #clear previous data
    lb.delete(0, 'end')

    #put new data
    for item in data:
        lb.insert('end', item)

def select_region(region, selected_regionCITY_label, listbox_rides, selected_game):
    # הפונקציה מקבלת את האזור שנבחר על ידי המשתמש, את הליסטבוקס של הנסיעות ואת המשחק
    # הנבחר ומוצאת הסעות אפשריות לאותו משחק לאזור ומעדכנת את הליסטבוקס
    global passenger_page

    selected_regionCITY_label.config(text=f"אזור: {region}")
    listbox_rides.delete(0, tk.END) # Delete all items in the listbox

    cities = get_cities(f"{region}") # השגה של גל הערים הנמצאות באזור ספציפי

    # מציג הודעה זמנית
    listbox_rides.insert(tk.END, "טוען הסעות לאזור זה... 🔄")

def fetch_rides():
    all_region_rides = []
    global connect
    for city in cities:
        # שליחת בקשה לשרת לקבלת הסעות ליישוב ולמשחק הנבחר
        rides = get_rides(f"{selected_game.replace('🏀', '')}{connect[2]}|יישוב:{city}|SEND_THE_RIDES")
```

יובל יוגב - Drive2Game

```
if rides != "There_are_no_rides":
    all_region_rides += rides

def update_listbox():
    listbox_rides.delete(0, tk.END) # Delete all items in the listbox
    if all_region_rides != []:
        for ride in all_region_rides:
            username = ride[-1]
            try:
                num_passengers = int(ride[-2])
            except:
                num_passengers = int(ride[-1].split(', ')[0])

            comments = ""
            if ride[2] != "": comments = f". הערות: {ride[2]}"

            # Get how many passenger are in the ride and put in the text how many places
are left
            client.send_message(f"{ride[0]}|COUNT_PASSENGER")
            num_of_passenger = client.get_message()
            place_left = num_passengers - int(num_of_passenger)
            place_left = f". מקומות שנשארו {place_left}"

            # Get how much is the rate of the driver and put it on text
            client.send_message(f"{username}|GET_RATE")
            driver_rate = client.get_message()
            driver_rate = f" דירוג נהג {driver_rate}"

            listbox_rides.insert(tk.END, f"{ride[1]}{comments}{place_left} |{driver_rate}")
        listbox_rides.config()
    else:
        try:
            #Print error message that there is not active ride
            showerror("Error:", "!!זהו מהאזור הזה!!")
            passenger_page.focus_force() # מחזיר את הפוקוס לחלון
        except Exception as e:
            print(e)

listbox_rides.after(0, update_listbox) # הראשי Thread חוזר ל

threading.Thread(target=fetch_rides).start()

def sort_rides_4U(event, city, selected_regionCITY_label, listbox_rides, protocol):
    # הפונקציה מקבלת אירוע שאין בו שימוש ומקבלת את העיר והמשחק הרצויים וליסטבוקס של נסיעות
    ומעדכנת את הרשימה על פי הנסיעות שקיימות רק מעיר מסוימת
```

יובל יוגב - Drive2Game

global passenger_page

if city != "":

selected_regionCITY_label.config(text=f"יישוב: {city}")

listbox_rides.delete(0, tk.END) # Delete all items in the listbox

rides = get_rides(protocol) # get all the rides from the specific city

if rides != "There_are_no_rides":

for ride in rides:

username = ride[-1]

try:

num_passengers = int(ride[-2])

except:

num_passengers = int(ride[-1].split(', ')[0])

comments = ""

if ride[2] != "": comments = f"הערות: {ride[2]}"

Get how many passenger are in the ride and put in the text how many places are

left

client.send_message(f"{ride[0]}|COUNT_PASSENGER")

num_of_passenger = client.get_message()

place_left = num_passengers - int(num_of_passenger)

place_left = f"מקומות שנשארו {place_left}"

Get how much is the rate of the driver and put it on text

client.send_message(f"{username}|GET_RATE")

driver_rate = client.get_message()

driver_rate = f"דירוג נהג {driver_rate}"

listbox_rides.insert(tk.END, f"{ride[1]}{comments}{place_left} |{driver_rate}")

listbox_rides.config()

else:

try:

#Print error message that there is not active ride

showerror("Error:", "אין הסעות למשחק זה מהיישוב הזה!!")

passenger_page.focus_force() # מחזיר את הפוקוס לחלון

except Exception as e:

print(e)

def get_rides(protocol):

global client

client.send_message(protocol)

message = client.get_message()

Drive2Game - יובל יוגב

```
if '|' in message:
    message = message.split("|")

    rides = message[0]
    rides = rides.replace('[', '').replace(']', '')
    if rides.count('(') > 1:
        rides = rides.replace('(', ' ( )|(')
        rides = rides.split('|')
        rides = [tuple(tx.replace('(', '').replace(')', '').replace('"', ' ').replace('""', '').split('|')) for
tx in rides]
    else:
        rides = [tuple(rides.replace('(', '').replace(')', '').replace('"', ' ').replace('""', ' ',
'|').replace('""', '').split('|'))]

    return rides

else: return message

def passenger(event, list_of_games):
    #Page for the passengers and get the list of games
    global passenger_page
    passenger_page = tk.Toplevel() # יצירת חלון

    screen_width = passenger_page.winfo_screenwidth()
    screen_height = passenger_page.winfo_screenheight()

    passenger_page.geometry(f"{screen_width}x{screen_height}") # Set the screen size to
match the user's display
    passenger_page.title("Passenger Page") # title of the window

    passenger_page.configure(bg="lavender") # Fill the background in lavender color

    label = tk.Label(passenger_page, text="אנא בחר משחק:", font=("Ariel", 50, "bold"),
bg="lavender", fg="gray22")
    label.pack(side="top", pady=10)

    # Display the games in a listbox
    listbox = tk.Listbox(passenger_page, font=("Arial", 20), height=screen_height,
width=screen_width, selectmode=tk.SINGLE, justify=tk.RIGHT)
    for game in list_of_games:
        listbox.insert(tk.END, f'{game[5]} {game[0]} נגד {game[1]}, בתאריך, {game[2]}, בשעה:
{game[3]}, {game[4]}')
    listbox.pack(pady=20)

    # Bind the selection event
    # הפונקציה בתוך הפונקציה בשביל שלא יהיה צורך בהעברת משתנים
    def on_select(event):
```

יובל יוגב - Drive2Game

```
selected_index = listbox.curselection()
if selected_index: # Ensure something is selected
    selected_game = listbox.get(selected_index)
    select_drive(selected_game)

listbox.bind("<<ListboxSelect>>", on_select)

# Start the main event loop
passenger_page.mainloop()

def select_drive(selected_game, choose_style=0): #0 means a choose and sort style, 1
means choose map style
    # פונקציה שמקבלת את המשחק הנבחר ואת הסטייל שבו המשתמש רוצה לראות את ההסעות, הפונקציה
מראה גרפית את ההסעות הקיימות לאותו משחק
    global passenger_page
    passenger_page.destroy() # סגירתו (הריסת חלון)

    passenger_page = tk.Toplevel(name="passenger_page_1") # יצירת חלון

    passenger_page.resizable(False, False) # Disable resizing the window
    # Create window size 750x750 and centered
    width, height = 750, 750
    x = (passenger_page.winfo_screenwidth() - width) // 2
    y = ((passenger_page.winfo_screenheight() - height) // 2) - 12
    passenger_page.geometry(f"{width}x{height}+{x}+{y}")

    passenger_page.title("Passenger Page") # title of the window

    passenger_page.configure(bg="light blue") # Fill the background in light blue color

    tk.Label(passenger_page, text="תבחר את ההסעה שלך", font=("Ariel", 30, "bold"), bg="light
blue", fg="gray22").pack(side="top")

    # Add a back arrow icon
    try:
        back_arrow_photo = create_Photos("photos/reply arrow.png", (50, 50), True)
        back_arrow = tk.Label(passenger_page, image=back_arrow_photo, bg="light blue",
cursor="hand2")
        back_arrow.photo = back_arrow_photo
        back_arrow.place(x=width * 0.85, y=height * 0.05)

        back_arrow.bind("<Button-1>", back_Arrow_clicked)
    except FileNotFoundError:
        print(f"The image 'back arrow.png' has not found")

    tk.Label(passenger_page, text=f"ההסעה למשחק {selected_game.split(',')[0]}", font=("Ariel",
16), bg="light blue", fg="gray22").pack()
```

יובל יוגב - Drive2Game

```
listbox_rides = tk.Listbox(passenger_page)
global connect
rides = get_rides(f"{selected_game.replace('🏀', '')}|{connect[2]}|SEND_THE RIDES") #
משיגה את כל ההסעות הקיימות למשחק הספציפי

if rides != "There_are_no_rides":
    def Change_to_map(event): #Change the choosing style to map style
        choose_style = 1
        select_drive(selected_game, choose_style)
    def Cahnge_to_free(event): #Change the choosing style to free style
        choose_style = 0
        select_drive(selected_game, choose_style)

    frame = tk.Frame(passenger_page, bg="blue") # Create a Frame for containing the
labels
    frame.pack(side="top")

    c_map = tk.Label(frame, text="מפה", font=("Ariel", 12), bg="blue", fg="gold",
cursor="hand2")
    c_map.pack(side="left")
    c_map.bind("<Button-1>", Change_to_map)

    tk.Label(frame, text="|", font=("Ariel", 12), bg="blue", fg="gold").pack(side="left",
padx=2)

    c_free = tk.Label(frame, text="בחירה חופשית", font=("Ariel", 12), bg="blue", fg="gold",
cursor="hand2")
    c_free.pack(side="left")
    c_free.bind("<Button-1>", Cahnge_to_free)

    if choose_style == 0:
        # בחירת הסעות על ידי בחירה חופשית (רשימה)
        frame = tk.Frame(passenger_page, bg="light blue", name="select_drive") # Create a
Frame for containing the labels
        frame.pack(side="top")

        # All ride Button
        tk.Button(frame, text="הכל", font=("Tahoma", 14), bg="light blue", cursor="hand2",
command=lambda: select_drive(selected_game, 0)).pack(side="left")

        # Create a Menubutton
        mbutton = tk.Menubutton(frame, text="אזור", font=("Ariel", 12), bg="light blue", bd=10,
cursor="hand2", relief=tk.RAISED)
        mbutton.pack(side="left", padx=2)
```


Drive2Game - יובל יוגב

```
selected_regionCITY_label = tk.Label(passenger_page, text="", font=("Ariel", 10),
bg="light blue", fg="black")
selected_regionCITY_label.pack()

# Create a Menu
items = tk.Menu(mbutton, tearoff=0)
items.add_command(label="צפון", command=lambda: select_region("צפון",
selected_regionCITY_label, listbox_rides, selected_game))
items.add_command(label="מרכז", command=lambda: select_region("מרכז",
selected_regionCITY_label, listbox_rides, selected_game))
items.add_command(label="ירושלים", command=lambda: select_region("ירושלים",
selected_regionCITY_label, listbox_rides, selected_game))
items.add_command(label="דרום", command=lambda: select_region("דרום",
selected_regionCITY_label, listbox_rides, selected_game))

# Attach the menu to the Menubutton
mbutton.config(menu=items)

global cities
cities = get_cities()

city = tk.Entry(frame,bg="light blue", justify='right')
city.pack(side="left", padx=2)
city.bind('<KeyRelease>', lambda e: checkkey_in_entry_address(e, "select_drive"))
tk.Label(frame, text="ישוב:", font=("Ariel", 12), bg="light blue",
justify='right').pack(side="left")

def Reload_rides_4U(event):
    #refresh the rides on the page
    t_label = selected_regionCITY_label['text'] #Get the text in the label
selected_regionCITY_label
    if 'ישוב:' in t_label: sort_rides_4U(None, city.get(), selected_regionCITY_label,
listbox_rides, f"{selected_game.replace('🏀', '')}{connect[2]}ישוב:
{city.get()}|SEND_THE_RIDES")
    elif 'אזור' in t_label: select_region(t_label.split(" ")[1], selected_regionCITY_label,
listbox_rides, selected_game)
    else: select_drive(selected_game, 0)

try:
    reload_image = create_Photos("photos/reload.png", (25, 25), True)
    reload_image_label = tk.Label(frame, image=reload_image, bg="light blue",
cursor="hand2")
    reload_image_label.pack(side="right")
    reload_image_label.bind("<Button-1>", lambda event: Reload_rides_4U(event))
except Exception as e:
    print(e)
```

Drive2Game - יובל יוגב

```
try:
    sort_image = create_Photos("photos/funnel.png", (25, 25), True)
    sort_image_label = tk.Label(frame, image=sort_image, bg="light blue",
cursor="hand2")
    sort_image_label.pack(side="right")
    sort_image_label.bind("<Button-1>", lambda event: sort_rides_4U(event, city.get(),
selected_regionCITY_label, listbox_rides, f"{selected_game.replace('🏀',
"))}{connect[2]}יישוב: {city.get()}SEND_THE_RIDES"))
except Exception as e:
    print(e)

# creating list box
global city_frame
city_frame = tk.Frame(passenger_page, bg="light blue")
city_frame.pack(side="top")
global lb
lb = tk.Listbox(city_frame)
lb.pack()
lb.pack_forget() # Hide Listbox initially

update_cities_listbox(cities)

# Bind the selection event
# הפונקציה בתוך הפונקציה בשביל שלא יהיה צורך בהעברת משתנים
def on_select_city(event):
    selected_index = lb.curselection()
    if selected_index: # Ensure something is selected
        selected_city = lb.get(selected_index)
        lb.pack_forget()
        city_frame.pack_forget()
        city.delete(0, tk.END)
        city.insert(0, selected_city)

lb.bind("<<ListboxSelect>>", on_select_city)

# Display the rides in a listbox
listbox_rides = tk.Listbox(passenger_page, font=("Arial", 20), height=300, width=500,
selectmode=tk.SINGLE, justify=tk.RIGHT)

# Create a horizontal scrollbar
scrollbar_x = tk.Scrollbar(passenger_page, orient=tk.HORIZONTAL)
scrollbar_x.pack(side=tk.BOTTOM, fill=tk.X)

# Configure the Listbox to use the horizontal scrollbar
listbox_rides.config(xscrollcommand=scrollbar_x.set)
scrollbar_x.config(command=listbox_rides.xview)
```

Drive2Game - יובל יוגב

```
for ride in rides:
    username = ride[-1]
    try:
        num_passengers = int(ride[-2])
    except:
        num_passengers = int(ride[-1].split(', ')[0])

    comments = ""
    if ride[2] != "": comments = f". הערות: {ride[2]}"

    #Get how many passenger are in the ride and put in the text how many places are
left    client.send_message(f"{ride[0]}|COUNT_PASSENGER")
    num_of_passenger = client.get_message()
    place_left = num_passengers - int(num_of_passenger)

    if place_left > 0:
        place_left = f". מקומות שנשארו {place_left}"

    client.send_message(f"{username}|GET_RATE")
    driver_rate = client.get_message()
    driver_rate = f" דירוג נהג {driver_rate}"

    listbox_rides.insert(tk.END, f'{ride[1]}{comments}{place_left} |{driver_rate}')

# Ensure the Listbox view starts from the right
listbox_rides.xview_moveto(1.0) # Move the horizontal view to the rightmost position

listbox_rides.pack()

# Bind the selection event
# הפונקציה בתוך הפונקציה בשביל שלא יהיה צורך בהעברת משתנים
def on_select_ride(event):
    selected_index = listbox_rides.curselection()
    if selected_index: # Ensure something is selected
        selected_ride = listbox_rides.get(selected_index)
        join_to_ride(selected_ride, selected_game.split(',')[0][2:])

listbox_rides.bind("<<ListboxSelect>>", on_select_ride)

else:
    # שימוש בבחירה של הסעה על ידי מפה
    OCG = OpenCageGeocode('5ffad6bdfdd24065aae0518433557ea5') # מפתח ל-API

    map_frame = tk.Frame(passenger_page, bg="light blue")
    map_frame.pack(fill="both", expand=True)
```

יובל יוגב - Drive2Game

```
map_widget = TkinterMapView(map_frame, width=600, height=650)
map_widget.pack(fill="both", expand=True, side="left")

def rest_map(event=None):
    # קבע מיקום התצוגה הראשוני + זום
    map_widget.set_position(32.143128, 34.838085)
    map_widget.set_zoom(8)

rest_map() # מציג את המפה

# חיפוש הסעות בעיר מסוימת. עושה זום על עיר ספציפית
tk.Label(map_frame, text="יישוב:", font=("Ariel", 20), bg="light blue",
fg="gray22").pack(side="top")
city = tk.Entry(map_frame)
city.pack(pady=5)

tk.Button(map_frame, text="חפש!", font=("Ariel", 12, "bold"), bg="white", fg="purple",
command=lambda: search_coordinates_of_city(city.get())).pack(pady=5)

rest_map_label = tk.Label(map_frame, text="איפוס מפה", font=("Ariel", 12, "bold"),
bg="light blue", fg="red")
rest_map_label.pack(pady=10)
rest_map_label.bind("<Button-1>", rest_map)

def search_coordinates_of_city(city):
    # חיפוש קורדינטות של עיר ספציפית
    if city != "":
        try:
            # get the coordinates by the address:
            results = OCG.geocode(f'{city}')
            coordinates = '%f;%f' % (results[0]['geometry']['lat'],
results[0]['geometry']['lng'])
            coordinates = coordinates.split(';')

            map_widget.set_position(float(coordinates[0]), float(coordinates[1]))
            map_widget.set_zoom(14)
        except:
            try:
                # Print message that the city name has a problem
                showerror("יש בעיה עם השם של העיר, תנסה שם אחר או תבדוק אם יש",
("שגיאות כתיב"))

                passenger_page.focus_force() # מחזיר את הפוקוס לחלון
            except Exception as e:
                print(e)

def show_ride_info(marker, ride_info):
    # פונקציה שמקפיצה למסך הודעה של פרטי הנסיעה
```

יובל יוגב - Drive2Game

```
result = askokcancel("פרטי נסיעה", f"כתובת: {ride_info}")

if result:
    join_to_ride(ride_info, selected_game.split(',')[0][2:])
else:
    passenger_page.focus_force() # מחזיר את הפוקוס לחלון

for ride in rides:
    # פרטי הנסיעה:
    address = ride[1]

    username = ride[-1]
    try:
        num_passengers = int(ride[-2])
    except:
        num_passengers = int(ride[-1].split(',')[0])

    comments = ""
    if ride[2] != "":
        comments = f"הערות: {ride[2]}"

    # Get how many passenger are in the ride and put in the text how many places are
left
    client.send_message(f"{ride[0]}|COUNT_PASSENGER")
    num_of_passenger = client.get_message()
    place_left = num_passengers - int(num_of_passenger)

    if place_left > 0:
        place_left = f"מקומות שנשארו {place_left}"

    client.send_message(f"{username}|GET_RATE")
    driver_rate = client.get_message()
    driver_rate = f"דירוג נהג {driver_rate}"

    # get the coordinates by the address:
    results = OCG.geocode(f'{address}')
    coordinates = '%f;%f' % (results[0]['geometry']['lat'], results[0]['geometry']['lng'])
    coordinates = coordinates.split(';')

    map_widget.set_marker(float(coordinates[0]), float(coordinates[1]),
text=address, command= lambda marker, ride_info=f'{address}{comments}{place_left}
|{driver_rate}': show_ride_info(marker, ride_info))
    else:
        tk.Label(passenger_page, text="!!אין הסעות למשחק זה", font=("Ariel", 24), bg="light blue",
fg="red").pack()

passenger_page.mainloop()
```

יובל יוגב - Drive2Game

```
def join_to_ride(selected_ride, selected_game):
    # פונקציה המקבלת את ההסעה שנבחרה טאת המשחק הנבחר, מבקשת מהמשתמש פרטים על כתובת
    # האיסוף ושולחת בקשה אל הנהג שיאשר אותו להסעה
    global passenger_page
    passenger_page.destroy() # סגירתו (הריסת המסך)

    passenger_page = tk.Toplevel(name="passenger_page_2") # יצירת מסך

    passenger_page.resizable(False, False) # Disable resizing the window
    # Create window size 600x550 and centered
    width, height = 600, 550
    x = (passenger_page.winfo_screenwidth() - width) // 2
    y = (passenger_page.winfo_screenheight() - height) // 2
    passenger_page.geometry(f'{width}x{height}+{x}+{y}')

    passenger_page.title("Passenger Page") # title of the window

    passenger_page.configure(bg="#BFB39B")

    tk.Label(passenger_page, text="עליך למלא את הפרטים הבאים\ בשביל להצטרף להסעה",
    font=("Ariel", 25, "bold"), bg="#BFB39B", fg="gray22").pack(side="top")

    # pick up address label and entry
    tk.Label(passenger_page, text="כתובת בה תעלה על ההסעה", font=("Tahoma", 14, "bold"),
    bg="#BFB39B", fg="royal blue").pack(pady=4)
    tk.Label(passenger_page, text="שם רחוב ומספר בית", font=("Tahoma", 10, "bold"),
    bg="#BFB39B", fg="royal blue").pack()
    street_driving_address = tk.Entry(passenger_page, justify='right')
    street_driving_address.pack(side="top")

    global cities
    cities = get_cities() # מקבלת את כל היישובים בישראל

    # הצגה של כל היישובים בישראל בליסטבוקס
    tk.Label(passenger_page, text="יישוב", font=("Tahoma", 10, "bold"), bg="#BFB39B",
    fg="royal blue").pack()
    city_driving_address = tk.Entry(passenger_page, justify='right', name="join_to_ride")
    city_driving_address.pack(side="top")
    city_driving_address.bind('<KeyRelease>', lambda e: checkkey_in_entry_address(e,
    "join_to_ride"))

    # creating list box
    global city_frame_2
    city_frame_2 = tk.Frame(passenger_page, bg="light blue")
    city_frame_2.pack(side="top")
    global lb
```

יובל יוגב - Drive2Game

```
lb = tk.Listbox(city_frame_2)
lb.pack()
lb.pack_forget() # Hide Listbox initially

update_cities_listbox(cities)

# Bind the selection event
# הפונקציה בתוך הפונקציה בשביל שלא יהיה צורך בהעברת משתנים
def on_select(event):
    selected_index = lb.curselection()
    if selected_index: # Ensure something is selected
        selected_city = lb.get(selected_index)
        lb.pack_forget()
        city_frame_2.pack_forget()
        city_driving_address.delete(0, tk.END)
        city_driving_address.insert(0, selected_city)

lb.bind("<<ListboxSelect>>", on_select)

tk.Button(passenger_page, text="שלח בקשת הצטרפות", bg="#BFB39B", cursor="hand2",
command=lambda: save_passenger(id_ride, street_driving_address.get(),
city_driving_address.get())).pack(side="top")

# קבלת פרטים על הנסיעה הנבחרת
client.send_message(f"{{selected_game}}|{{connect[2]}}|יישוב:
{{selected_ride}}|SEND_THE_RIDES")
the_ride = client.get_message()
if the_ride != "There_are_no_rides":
    the_ride = the_ride.split('|')[0].replace('(', ' ').replace(')', ' ').split(", ")
    id_ride = the_ride[0].replace("","")

passenger_page.mainloop()

def save_passenger(id_ride, street_driving_address, city_driving_address):
    # פונקציה שמקבלת את פרטי הנסוע ומוסיפה אותו למסד הנתונים בתור נוסע שרוצה להצטרף להעסה
    וצריך לאשר אותו
    global passenger_page
    global connect
    username_passenger = connect[2] # Collect the username of the passenger

    pick_up_location = f"{{street_driving_address}}, {{city_driving_address}}"

    global client
    client.send_message(f"{{username_passenger}}|{{id_ride}}|{{pick_up_location}}|JOIN_RIDE")
    m = client.get_message()
    if m == "REQUEST_SEND":
        try:
```

יובל יוגב - Drive2Game

```
# Print message that the request ride was sended
showinfo("הבקשה נשלחה בהצלחה. הישאר/י מעודכן/ת לגבי סטטוס ההסעה")

global passenger_page
passenger_page.destroy()
except Exception as e:
    print(e)
else:
    if m == "THE_PASSENGER_IS_DRIVER":
        message_error = "אתה לא יכול להצטרף להסעה בה אתה הנהג" # זה לא אמור בכלל להראות לו"
        שאתה לא יכול להצטרף להסעה הזאת אבל זה בדיקה למקרה ש...
    else:
        message_error = "יש בעיה עם שליחת הבקשה! נרשמת כבר להסעה זו... תבדוק את זה"
        "ברשימת ההסעות שלך"
    try:
        # Print message that the request ride has a problem
        showerror("מצב ההסעה", f"{message_error}")
        passenger_page.focus_force() # מחזיר את הפוקוס לחלון הראשי
    except Exception as e:
        print(e)

def driver(event, list_of_games):
    #Page for the drivers
    global driver_page
    driver_page = tk.Toplevel()

    screen_width = driver_page.winfo_screenwidth()
    screen_height = driver_page.winfo_screenheight()

    driver_page.geometry(f"{screen_width}x{screen_height}") # Set the screen size to match
    the user's display
    driver_page.title("Driver Page") # title of the window

    driver_page.configure(bg="lavender") # Fill the background in lavender color

    label = tk.Label(driver_page, text="אנא בחר משחק:", font=("Arial", 50, "bold"),
        bg="lavender", fg="gray22")
    label.pack(side="top", pady=10)

    # Display the games in a listbox
    listbox = tk.Listbox(driver_page, font=("Arial", 20), height=screen_height,
        width=screen_width, selectmode=tk.SINGLE, justify=tk.RIGHT)
    for game in list_of_games:
        listbox.insert(tk.END, f'{game[5]} {game[0]} נגד {game[1]}, בתאריך: {game[2]}, בשעה: {game[3]}, {game[4]}')
    listbox.pack(pady=20)
```


Drive2Game – יובל יוגב

```
# Bind the selection event
# הפונקציה בתוך הפונקציה בשביל שלא יהיה צורך בהעברת משתנים
def on_select(event):
    selected_index = listbox.curselection()
    if selected_index: # Ensure something is selected
        selected_game = listbox.get(selected_index)
        start_drive(selected_game)

listbox.bind("<<ListboxSelect>>", on_select)

# Start the main event loop
driver_page.mainloop()

def games_to_go():
    # - (מנהלת הליגה בכדורסל) של ליגת ווינר סל
    אתר שהורדתי אז הוא אופליין
    with open("מנהלת ליגת העל בכדורסל _ עונת 2024-25 _ משחקים ותוצאות.html", "r",
encoding="utf-8") as file:
        soup = BeautifulSoup(file, "html.parser") # יצירת אובייקט BeautifulSoup

        text = soup.text # מעביר משפט html בטקסט

        מאיפה עד איפה לקחת את המשחקים
        start_marker = r"תוצאה*סל*רטיסים*לאורחת*לארחת*משדר*ערוע*לאולם*שעה*לתאריך"r
        end_marker =
r"סל*תוצאה*לאורחת*לארחת*משדר*ערוע*לאולם*שעה*לתאריך*סל*ד*לאולם*שעה*לתאריך*סל*תוצאה*לאורחת*לארחת*משחקים"r

        pattern = re.compile(f"{start_marker}{.*?}{end_marker}", re.DOTALL) # יצירת תבנית
לחיפוש טווח המשחקים

        match = pattern.search(text) # מחפש את החלק הזה בטקסט
        if match:
            game_schedule = match.group(1).strip() # מנקה את הדברים הלא קשורים מהטקסט

            # reomove all the things I dont want
            for i in range(19, 27):
                new_pattern =
fr"תוצאה*סל*רטיסים*לאורחת*לארחת*משדר*ערוע*לאולם*שעה*לתאריך*סל*{i}*לאולם*שעה*לתאריך*סל*תוצאה*לאורחת*לארחת*משחקים"r
                game_schedule = re.sub(new_pattern, "", game_schedule)

            games = game_schedule.split("\n") # בונה רשימה של משחקים
לפי החוקיות של הטקסט הרשום

            list_of_games = []

            for q in games:
```

יובל יוגב - Drive2Game

```
if g[0].isdigit() is False:
    g = g[1:]

game = g.split("\n")

if len(game[1].split(':')) == 1: game[1] += " לא ידוע" # אם השעה עדיין לא רשומה
if game[2] == "": game[2] = " לא ידוע" # אם האולם עדיין לא ידוע

list_of_games.append((game[9], game[-1], game[0][0:10], game[1], "אולם" +
game[2], '🏀')) # (team_home, team_gust, date, time, arena) למשחק
בונה טאפל של הנתונים

return list_of_games
else:
    print("Markers not found in the text.")

def start_drive(selected_game):
    # פונקציה שמקבלת את המשחק אליו ההסעה תהיה מציגה מסך לנהג בו המשתמש צריך להכניס פרטים
    על הנסיעה
    global driver_page
    driver_page.destroy() # סגירת מסך

    driver_page = tk.Toplevel(name="driver_page") # יצירת מסך

    driver_page.resizable(False, False) # Disable resizing the window
    # Create window size 600x500 and centered
    width, height = 600, 500
    x = (driver_page.winfo_screenwidth() - width) // 2
    y = (driver_page.winfo_screenheight() - height) // 2
    driver_page.geometry(f'{width}x{height}+{x}+{y}')

    driver_page.title("Driver Page") # title of the window

    driver_page.configure(bg="lavender") # Fill the background in lavender color

    tk.Label(driver_page, text="תיצור את ההסעה שלך", font=("Ariel", 30, "bold"), bg="lavender",
fg="gray22").pack(side="top")

    # Add a back arrow icon
    try:
        back_arrow_photo = create_Photos("photos/reply arrow.png", (50, 50), True)
        back_arrow = tk.Label(driver_page, image=back_arrow_photo, bg="lavender",
cursor="hand2")
        back_arrow.photo = back_arrow_photo
        back_arrow.place(x=width * 0.85, y=height * 0.05)

        back_arrow.bind("<Button-1>", back_Arrow_clicked)
    except FileNotFoundError:
```

יובל יוגב - Drive2Game

```
print(f"The image 'back arrow.png' has not found")

tk.Label(driver_page, text=f"ההסעה למשחק {selected_game.split(',')[0]}", font=("Ariel", 16),
bg="lavender", fg="gray22").pack()

#start driving address label and entry
tk.Label(driver_page, text="כתובת בה תתחיל הנסיעה", font=("Tahoma", 14, "bold"),
bg="lavender", fg="royal blue").pack(pady=4)
tk.Label(driver_page, text="שם רחוב ומספר בית", font=("Tahoma", 8, "bold"), bg="lavender",
fg="royal blue").pack()
street_driving_address = tk.Entry(driver_page, justify='right')
street_driving_address.pack(side="top")

global cities
cities = get_cities() # השגה של כל היישובים בישראל

tk.Label(driver_page, text="יישוב", font=("Tahoma", 8, "bold"), bg="lavender", fg="royal
blue").pack()
city_driving_address = tk.Entry(driver_page, justify='right', name="start_drive")
city_driving_address.pack(side="top")
city_driving_address.bind('<KeyRelease>', lambda e:checkkey_in_entry_address(e,
"start_drive"))

# creating list box
global city_driving_frame
city_driving_frame = tk.Frame(driver_page, bg="light blue")
city_driving_frame.pack(side="top")
global lb
lb = tk.Listbox(city_driving_frame)
lb.pack()
lb.pack_forget() # Hide Listbox initially

update_cities_listbox(cities)

# Bind the selection event
# הפונקציה בתוך הפונקציה בשביל שלא יהיה צורך בהעברת משתנים
def on_select(event):
    selected_index = lb.curselection()
    if selected_index: # Ensure something is selected
        selected_city = lb.get(selected_index)
        lb.pack_forget()
        city_driving_frame.pack_forget()
        city_driving_address.delete(0, tk.END)
        city_driving_address.insert(0, selected_city)

lb.bind("<<ListboxSelect>>", on_select)
```

יובל יוגב - Drive2Game

```
#number of passengers label and entry
tk.Label(driver_page, text="מספר נוסעים שיכולים להצטרף להסעה", font=("Tahoma", 14,
"bold"), bg="lavender", fg="royal blue").pack(pady=10)

num_passenger = tk.Entry(driver_page, justify='right')
num_passenger.pack()

#text_comments label and entry
tk.Label(driver_page, text="כגון גיל מינימלי או מקסימלי לנוסע, נסיעה כרוכה בתשלום... הערות:", font=("Tahoma", 13, "bold"), bg="lavender", fg="royal blue").pack(pady=25)

# scrolled text
text_comments = scrolledtext.ScrolledText(driver_page, width=30, height=1)
text_comments.pack()

e_save_ride_label = tk.Label(driver_page, text="", font=("Tahoma", 14, "bold"),
bg="lavender", fg="red")
e_save_ride_label.pack(side="bottom")

start_driving_address = ""
#Create ride Button
create_ride_button = tk.Button(driver_page, text="תיצור את ההסעה", font=("Tahoma", 14),
cursor="hand2", command=lambda: saveDB_ride(selected_game,
start_driving_address.join(f"{street_driving_address.get()}, {city_driving_address.get()}"),
num_passenger.get(), text_comments.get("1.0", tk.END).strip(), e_save_ride_label))
create_ride_button.pack(pady=10)

driver_page.mainloop()

def saveDB_ride(selected_game, start_driving_address, num_passenger, text_comments,
e_save_ride_label):
    # פונקציה המקבלת את נתוני ההסעה ויוצרת נסיעה בעזרת הוספה למסד הנתונים
    global connect
    username_driver = connect[2] # Collect the username of the driver

    e_save_ride = ""
    if num_passenger.isdigit() is False:
        e_save_ride += "-מספר הנוסעים צריך להיות מבוטא באמצעות מספרים"
    if start_driving_address == "":
        e_save_ride += "-לא הכנסת את הכתובת בה תתחיל את הנסיעה"

    if e_save_ride == "":
        global client
        client.send_message(f"{selected_game.replace('🏠',
'')}{start_driving_address}{num_passenger}{text_comments}{username_driver}|CREATE_
RIDE")
        m = client.get_message()
```

יובל יוגב - Drive2Game

```
global driver_page
if m == "SAVE_RIDE":
    try:
        #Print meassage that the ride is complete and save
        showinfo("ההסעה נשמרה בהצלחה", "מצב ההסעה")

        driver_page.destroy()
    except Exception as e:
        print(e)
    else:
        try:
            # Print meassage that the user already has a ride to this game
            showerror("מצב ההסעה", "כבר קיימת לך הסעה למשחק הזה")

            driver_page.destroy()
        except Exception as e:
            print(e)
    else:
        e_save_ride_label.config(text=e_save_ride)

def change_the_request_ride(event, id_ride, username_passenger, the_event):
    # פונקציה שמשנה את בקשת ההצטרפות אל נסיעה
    label = event.widget #get the label
    text_of_label = label.cget("text") #get the text of the label

    global client
    if '✅' in text_of_label:
        # שולח לשרת הודעה שיאשר את הבקשה של הנוסע

    client.send_message(f"{id_ride}|{username_passenger}|1|CHANGE_PASSENGER_STATUS_OF_RIDE")
    message = client.get_message()

    # לאחר אישור בקשת הנסיעה של הנוסע, נוצרת בדיקה אל האם נשארו עוד מקומות בהסעה, אם לא
    # נשארו נשלחת הודעה אל השרת לדחות את כל השאר הבקשות של הנוסעים
    client.send_message(f"{id_ride}|0|GET_NUM_OF_PASSENGERS")
    message = client.get_message()
    if message == "There_are_no_passengers":
        num_pass = 0
    else:
        num_pass = int(message.split("|")[0])

    client.send_message(f"{id_ride}|COUNT_PASSENGER")
    message = client.get_message()
    num_of_pass = int(message)

    if num_of_pass == num_pass: #נגמרו המקומות בהסעה
```

יובל יוגב - Drive2Game

```
client.send_message(f'{id_ride}||{2}|CHANGE_PASSENGER_STATUS_OF_RIDE")
message = client.get_message()

try:
    #Print meessage that there is no more space for more passengers
    showinfo("מצב נוסעים בהסעה", "נגמרו המקומות בהסעה, כל הנוסעים שהמתינו נדחו")
except Exception as e:
    print(e)
elif '✗' in text_of_label:
    # שולח הודעה לשרת על דחיית הבקשה של הצטרפות לנסיעה

client.send_message(f'{id_ride}|{username_passenger}|{2}|CHANGE_PASSENGER_STATU
S_OF_RIDE")
message = client.get_message()

#ריפרש למסך
global info_ride_page
info_ride_page.destroy()
get_information_of_ride(the_event)

def get_information_of_ride(event):
    # פונקציה שמציגה למשתמש בצורה גרפית מידע על הנסיעה
    global the_event
    the_event = event
    frame = the_event.widget.master #get the frame of the label

    labels_in_frame = [child for child in frame.winfo_children() if isinstance(child, tk.Label)]
    #get all the labels in the frame

    text_labels = [label.cget("text") for label in labels_in_frame] #get the text of all the labels in
    the frame

    information = text_labels[0] #get the information from label number 1

    id__ride = text_labels[2] #get the id ride from label number 3

    global client

    email_of_driver = ""

    if "בקשת הצטרפות" in text_labels[1]: #אם זה קיים אז המשתמש הוא הנהג
        client.send_message(f'{id__ride}|T|GET_DRIVER_AND_PASSENGERS")
        message = client.get_message()

        s = message.split("$")
        information = information.split(" | ") + s[0].split("|")
        information.pop(-1) #הערך האחרון נופל בגלל ה split
```

יובל יוגב - Drive2Game

```
ps_approved = s[1].split("|")[1::]
ps_waiting = s[2].split("|")[1::]
ps_reject = s[3].split("|")[1::]

email_of_driver += f"אימייל202B\u: {information[-1]}"
else:
    if "אושר" in text_labels[1]: # אם זה קיים אז המשתמש הוא נוסע שאישורו אותו להסעה
        client.send_message(f"{id__ride}|F|T|GET_DRIVER_AND_PASSENGERS")
        message = client.get_message()

        s = message.split("$")
        information = information.split(" | ") + s[0].split("|")
        information.pop(-1) # הסרת האחרון נופל בגלל ה split

        ps_approved = s[1].split("|")[1::]

        email_of_driver += f"אימייל202B\u: {information[-1]}"
    else:
        client.send_message(f"{id__ride}|F|F|GET_DRIVER_AND_PASSENGERS")
        message = client.get_message()

        s = message.split("$")
        information = information.split(" | ") + s[0].split("|")

# Page for the info of the ride
global info_ride_page
info_ride_page = tk.Toplevel()

info_ride_page.resizable(False, False) # Disable resizing the window
# Create window size 600x800 and centered
width, height = 600, 800
x = (info_ride_page.winfo_screenwidth() - width) // 2
y = ((info_ride_page.winfo_screenheight() - height) // 2) - 12
info_ride_page.geometry(f"{width}x{height}+{x}+{y}")

info_ride_page.title("Rides Page") # title of the window

info_ride_page.configure(bg="rosy brown") # Fill the background in rosy brown color

title_frame = tk.Frame(info_ride_page, bg="rosy brown")
title_frame.pack(side="top")
tk.Label(title_frame, text="פרטי הנוסיעה", font=("Times New Roman", 40, "underline"),
bg="rosy brown", fg="black").pack(pady=5)

tk.Label(info_ride_page, text=f"{information[0]}", font=("Times New Roman", 20), bg="rosy
brown", fg="black").pack(side="top", pady=5)
```

Drive2Game - יובל יוגב

```
tk.Label(info_ride_page, text=f"\u202B{information[1]} | {information[2]}", font=("Times New Roman", 20), bg="rosy brown", fg="black").pack(side="top", pady=5)
tk.Label(info_ride_page, text=f"{information[4]}", font=("Times New Roman", 20), bg="rosy brown", fg="black").pack(side="top", pady=5)

tk.Label(info_ride_page, text=f"\u202Bנהג: {information[3]} | יוצא מ: {information[5]}", font=("Times New Roman", 20, "bold"), bg="rosy brown", fg="purple").pack(side="top", anchor="ne", pady=5)
if email_of_driver != "": tk.Label(info_ride_page, text=f"\u202B{email_of_driver}", font=("Times New Roman", 20, "bold"), bg="rosy brown", fg="purple").pack(side="top", anchor="ne", pady=5)
if information[-3] != "": tk.Label(info_ride_page, text=f"הערות: {information[-3]}", font=("Times New Roman", 20), bg="rosy brown", fg="black").pack(side="top", anchor="ne")
tk.Label(info_ride_page, text=f"\u202Bכמות נוסעים מקסימלית: {information[-2]}", font=("Times New Roman", 20), bg="rosy brown", fg="black").pack(side="top", anchor="ne")

canvas_of_info_ride_page = tk.Canvas(info_ride_page, bg="rosy brown", highlightthickness=0, width=570, height=770)
v_scrollbar = tk.Scrollbar(info_ride_page, orient="vertical", command=canvas_of_info_ride_page.yview)

h_scrollbar = tk.Scrollbar(info_ride_page, orient="horizontal", command=canvas_of_info_ride_page.xview)

scrollbar_frame = tk.Frame(canvas_of_info_ride_page, bg="rosy brown")

scrollbar_frame.bind("<Configure>", lambda e:
canvas_of_info_ride_page.configure(scrollregion=canvas_of_info_ride_page.bbox("all")))

canvas_of_info_ride_page.create_window((580, 0), window=scrollbar_frame, anchor="ne")
canvas_of_info_ride_page.configure(yscrollcommand=v_scrollbar.set, xscrollcommand=h_scrollbar.set)

h_scrollbar.pack(side="bottom", fill="x")
v_scrollbar.pack(side="right", fill="y")
canvas_of_info_ride_page.pack(side="left", fill="both", expand=True)

scrollbar_frame.columnconfigure(0, weight=1)

tk.Label(scrollbar_frame, text=f"נוסעים:", font=("Times New Roman", 20, "underline"), bg="rosy brown", fg="black").pack(side="top", anchor="ne")

if "בקשת הצטרפות" in text_labels[1]:
    tk.Label(scrollbar_frame, text=f"אושרו:", font=("Times New Roman", 16, "bold"), bg="rosy brown", fg="black").pack(side="top", anchor="ne")
    if ps_approved[0] == "":
```


Drive2Game - יובל יוגב

```
tk.Label(scrollbar_frame, text=f"א.י", font=("Times New Roman", 16), bg="rosy brown", fg="black").pack(side="top", anchor="ne")
else:
    for passenger in ps_approved:
        p = passenger.split("&")
        username_passenger, pickup_loc, email_of_passenger = p[0], p[1], p[2]
        client.send_message(f"{username_passenger}|GET_RATE")
        passenger_rate = client.get_message()
        passenger_rate = f"דירוג נוסע: {passenger_rate}"
        tk.Label(scrollbar_frame, text=f"א.י: {username_passenger} | מיקום: {pickup_loc} | {passenger_rate} | א.י: {email_of_passenger}", font=("Times New Roman", 16), bg="rosy brown", fg="black").pack(side="top", anchor="ne")

tk.Label(scrollbar_frame, text=f"ממתנים", font=("Times New Roman", 16, "bold"), bg="rosy brown", fg="black").pack(side="top", anchor="ne")
if ps_waiting[0] == "":
    tk.Label(scrollbar_frame, text=f"א.י", font=("Times New Roman", 16), bg="rosy brown", fg="black").pack(side="top", anchor="ne")
else:
    for passenger in ps_waiting:
        p_wait_frame = tk.Frame(scrollbar_frame, bg="rosy brown")
        p_wait_frame.pack(side="top", anchor="ne", fill="x")
        p = passenger.split("&")
        username_passenger, pickup_loc = p[0], p[1]
        client.send_message(f"{username_passenger}|GET_RATE")
        passenger_rate = client.get_message()
        passenger_rate = f"דירוג נוסע: {passenger_rate}"
        tk.Label(p_wait_frame, text=f"א.י: {username_passenger} | מיקום: {pickup_loc} | {passenger_rate}", font=("Times New Roman", 16), bg="rosy brown", fg="black").pack(side="right")
        accept_label = tk.Label(p_wait_frame, text=f"✔", font=("Times New Roman", 16), bg="rosy brown", fg="green", cursor="hand2")
        accept_label.pack(side="right")
        accept_label.bind("<Button-1>", lambda event: change_the_request_ride(event, id__ride, username_passenger, the_event))
        reject_label = tk.Label(p_wait_frame, text=f"✖", font=("Times New Roman", 16), bg="rosy brown", fg="red", cursor="hand2")
        reject_label.pack(side="right")
        reject_label.bind("<Button-1>", lambda event: change_the_request_ride(event, id__ride, username_passenger, the_event))

tk.Label(scrollbar_frame, text=f"נדון", font=("Times New Roman", 16, "bold"), bg="rosy brown", fg="black").pack(side="top", anchor="ne")
if ps_reject[0] == "":
    tk.Label(scrollbar_frame, text=f"א.י", font=("Times New Roman", 16), bg="rosy brown", fg="black").pack(side="top", anchor="ne")
else:
```

יובל יוגב - Drive2Game

```
for passenger in ps_reject:
    p = passenger.split("&")
    username_passenger, pickup_loc = p[0], p[1]
    client.send_message(f"{username_passenger}|GET_RATE")
    passenger_rate = client.get_message()
    passenger_rate = f"דירוג נוסע: {passenger_rate}"
    tk.Label(scrollbar_frame, text=f"u202B{username_passenger} | איסוף: {pickup_loc} | {passenger_rate}", font=("Times New Roman", 16), bg="rosy brown", fg="black").pack(side="top", anchor="ne")

elif "אושר" in text_labels[1]:
    if ps_approved[0] == "":
        tk.Label(scrollbar_frame, text=f"אין", font=("Times New Roman", 16), bg="rosy brown", fg="black").pack(side="top", anchor="ne")
    else:
        for passenger in ps_approved:
            p = passenger.split("&")
            username_passenger, email_of_passenger = p[0], p[1]
            client.send_message(f"{username_passenger}|GET_RATE")
            passenger_rate = client.get_message()
            passenger_rate = f"דירוג נוסע: {passenger_rate}"
            tk.Label(scrollbar_frame, text=f"u202B{username_passenger} | {passenger_rate} | אימייל: {email_of_passenger}", font=("Times New Roman", 16), bg="rosy brown", fg="black").pack(side="top", anchor="ne")

        else:
            tk.Label(scrollbar_frame, text=f"אין לך גישה לנתונים אלו", font=("Times New Roman", 16, "bold"), bg="rosy brown", fg="dark red").pack(side="top", anchor="ne")

info_ride_page.mainloop()

def your_rides_window():
    # Page for the RIDES
    rides_page = tk.Toplevel() # יצירת חלון

    rides_page.resizable(False, False) # Disable resizing the window
    # Create window size 900x750 and centered
    width, height = 900, 750
    x = (rides_page.winfo_screenwidth() - width) // 2
    y = ((rides_page.winfo_screenheight() - height) // 2) - 12
    rides_page.geometry(f"{width}x{height}+{x}+{y}")

    rides_page.title("Rides Page") # title of the window

    rides_page.configure(bg="rosy brown") # Fill the background in rosy brown color

    top_frame = tk.Frame(rides_page, bg="rosy brown")
```

יובל יוגב - Drive2Game

```
top_frame.pack(side="top", fill="x")

tk.Label(top_frame, text="ההסעות שלך", font=("Times New Roman", 40, "underline"),
bg="rosy brown", fg="black").pack(side="right", pady=10, padx=15)

tk.Button(top_frame, text="הסעות קודמות", font=("Arial", 24, "bold"), bg="light green",
cursor="hand2", command=previous_rides).pack(side="right", padx=15)

tk.Label(top_frame, text="לפרטים לחץ על ההסעה", font=("Times New Roman", 18),
bg="rosy brown", fg="black").pack(side="left")

global client

canvas_of_your_rides_page = tk.Canvas(rides_page, bg="rosy brown",
highlightthickness=0)
scrollbar = tk.Scrollbar(rides_page, orient="vertical",
command=canvas_of_your_rides_page.yview)

scrollbar_frame = tk.Frame(canvas_of_your_rides_page, bg="rosy brown")

scrollbar_frame.bind("<Configure>", lambda e:
canvas_of_your_rides_page.configure(scrollregion=canvas_of_your_rides_page.bbox("all")))

canvas_of_your_rides_page.create_window((880, 0), window=scrollbar_frame,
anchor="ne")
canvas_of_your_rides_page.configure(yscrollcommand=scrollbar.set)

canvas_of_your_rides_page.pack(side="left", fill="both", expand=True)
scrollbar.pack(side="right", fill="y")

scrollbar_frame.columnconfigure(0, weight=1)

tk.Label(scrollbar_frame, text="סיוע", font=("Times New Roman", 20, "underline"),
bg="rosy brown", fg="black").pack(side="top", anchor="ne")

rides = get_rides(f"{connect[2]}|0|FIND_D_RIDES")
if rides != "There_are_no_rides":
    new_rides = []
    for ride in rides:
        new_ride = (f"🏀{ride[0]}", ride[1], ride[2], ride[3])
        new_rides.append(new_ride)

    for ride in new_rides:
        frame_label_ride = tk.Frame(scrollbar_frame, bg="rosy brown")
        frame_label_ride.pack(side="top", fill="x")
        label_of_ride = tk.Label(frame_label_ride, text=f"{ride[0]} | {ride[1]} | {ride[2]}",
font=("Arial", 18), bg="rosy brown", fg="black", cursor="hand2")
```

Drive2Game - יובל יוגב

```
label_of_ride.pack(side="right", padx=5)
label_of_ride.bind("<Button-1>", get_information_of_ride)

client.send_message(f"{ride[3]}|0|GET_NUM_OF_PASSENGERS")
message = client.get_message()
if message == "There_are_no_passengers":
    num_of_requests = 0
else:
    message = message.split('|')
    num_of_requests = int(message[0])

label_of_status = tk.Label(frame_label_ride, text=f"בקשת הצטרפות:
{num_of_requests}", font=("Arial", 18, "bold"), bg="rosy brown", fg="blue")
label_of_status.pack(side="right")

tk.Label(frame_label_ride, text=f"{ride[3]}", bg="rosy brown", fg="rosy
brown").pack(side="right") #label of id ride
else:
    tk.Label(scrollbar_frame, text="אין", font=("Arial", 16, "bold"), bg="rosy brown",
fg="red").pack(side="top", anchor="ne")

tk.Label(scrollbar_frame, text="נוסע", font=("Times New Roman", 20, "underline"),
bg="rosy brown", fg="black").pack(side="top", anchor="ne")

rides = get_rides(f"{connect[2]}|0|FIND_P_RIDES")
if rides != "There_are_no_rides":
    new_rides = []
    for ride in rides:
        new_ride = (f"🚗 {ride[0]}", ride[1], ride[2], ride[3])
        new_rides.append(new_ride)

    for ride in new_rides:
        frame_label_ride = tk.Frame(scrollbar_frame, bg="rosy brown")
        frame_label_ride.pack(side="top", fill="x")
        label_of_ride = tk.Label(frame_label_ride, text=f"{ride[0]} | {ride[1]} | {ride[2]}",
font=("Arial", 18), bg="rosy brown", fg="black", cursor="hand2")
        label_of_ride.pack(side="right", padx=5)
        label_of_ride.bind("<Button-1>", get_information_of_ride)

client.send_message(f"{ride[3]}|{connect[2]}|GET_PASSENGER_STATUS_OF_RIDE")
status = int(client.get_message().split('|')[0][2])
if status == 0:
    label_of_status = tk.Label(frame_label_ride, text="ממתין 🕒", font=("Arial", 18,
"bold"), bg="rosy brown", fg="gray22")
    elif status == 1:
```

יובל יוגב - Drive2Game

```
label_of_status = tk.Label(frame_label_ride, text="✅ אושר", font=("Arial", 18,
"bold"), bg="rosy brown", fg="green")
elif status == 2:
    label_of_status = tk.Label(frame_label_ride, text="❌ נדחה", font=("Arial", 18,
"bold"), bg="rosy brown", fg="red")

label_of_status.pack(side="right")

tk.Label(frame_label_ride, text=f"{ride[3]}", bg="rosy brown", fg="rosy
brown").pack(side="right") #label of id ride
else:
    tk.Label(scrollbar_frame, text="אין", font=("Arial", 16, "bold"), bg="rosy brown",
fg="red").pack(side="top", anchor="ne")

rides_page.mainloop()

def previous_rides():
    # Page for the PREVIOUS RIDES
    previous_rides_page = tk.Toplevel() # יצירת מסך

    previous_rides_page.resizable(False, False) # Disable resizing the window
    # Create window size 600x600 and centered
    width, height = 600, 600
    x = (previous_rides_page.winfo_screenwidth() - width) // 2
    y = ((previous_rides_page.winfo_screenheight() - height) // 2) - 12
    previous_rides_page.geometry(f'{width}x{height}+{x}+{y}')

    previous_rides_page.title("Previous Rides Page") # title of the window

    previous_rides_page.configure(bg="light green") # Fill the background in light green color

    tk.Label(previous_rides_page, text="ההסעות הקודמות שלך:", font=("Times New Roman", 40,
"underline"), bg="light green", fg="black").pack(side="top", pady=10)

    global connect

    rides_D = get_rides(f'{connect[2]}|1|FIND_D_RIDES')
    rides_P = get_rides(f'{connect[2]}|1|FIND_P_RIDES')

    canvas_of_your_previous_rides_page = tk.Canvas(previous_rides_page, bg="light
green", highlightthickness=0)
    scrollbar = tk.Scrollbar(previous_rides_page, orient="vertical",
command=canvas_of_your_previous_rides_page.yview)

    scrollbar_frame = tk.Frame(canvas_of_your_previous_rides_page, bg="light green")
```

Drive2Game - יובל יוגב

```
scrollbar_frame.bind("<Configure>", lambda e:
canvas_of_your_previous_rides_page.configure(scrollregion=canvas_of_your_previous_rides_page.bbox("all")))
```

```
canvas_of_your_previous_rides_page.create_window((580, 0), window=scrollbar_frame,
anchor="ne")
canvas_of_your_previous_rides_page.configure(yscrollcommand=scrollbar.set)
```

```
canvas_of_your_previous_rides_page.pack(side="left", fill="both", expand=True)
scrollbar.pack(side="right", fill="y")
```

```
scrollbar_frame.columnconfigure(0, weight=1)
```

```
tk.Label(scrollbar_frame, text="בתור מסייע", font=("Times New Roman", 20, "underline"),
bg="light green", fg="black").pack(side="top", anchor="ne")
```

```
if rides_D != "There_are_no_rides":
    new_rides = []
    for ride in rides_D:
        new_ride = (f"🏎️{ride[0]}", ride[1], ride[2], ride[3])
        new_rides.append(new_ride)
```

```
for ride in new_rides:
    previous_rides_frame_4_label = tk.Frame(scrollbar_frame, bg="light green")
    previous_rides_frame_4_label.pack(side="top", fill="x")
```

```
ride_label = tk.Label(previous_rides_frame_4_label, text=f"{ride[0]} | {ride[1]} | {ride[2]}", font=("Arial", 18), bg="light green", fg="black", cursor="hand2")
ride_label.pack(side="right")
ride_label.bind("<Button-1>", get_information_of_ride)
```

```
tk.Label(previous_rides_frame_4_label, text=f"בקשת הצטרפות ", bg="light green", fg="light green").pack(side="right") #FAKE LABEL
tk.Label(previous_rides_frame_4_label, text=f"{ride[3]}", bg="light green", fg="light green").pack(side="right") # label of id ride
else:
    tk.Label(scrollbar_frame, text="אין", font=("Arial", 16, "bold"), bg="light green", fg="red").pack(side="top", anchor="ne")
```

```
tk.Label(scrollbar_frame, text="בתור נוסע", font=("Times New Roman", 20, "underline"), bg="light green", fg="black").pack(side="top", anchor="ne")
```

```
if rides_P != "There_are_no_rides":
    new_rides = []
    for ride in rides_P:
        new_ride = (f"🏎️{ride[0]}", ride[1], ride[2], ride[3])
        new_rides.append(new_ride)
```

Drive2Game - יובל יוגב

```
for ride in new_rides:
    previous_rides_frame_4_label = tk.Frame(scrollbar_frame, bg="light green")
    previous_rides_frame_4_label.pack(side="top", fill="x")

    ride_label = tk.Label(previous_rides_frame_4_label, text=f"{ride[0]} | {ride[1]} | {ride[2]}", font=("Arial", 18), bg="light green", fg="black", cursor="hand2")
    ride_label.pack(side="right")
    ride_label.bind("<Button-1>", get_information_of_ride)

    tk.Label(previous_rides_frame_4_label, text=f"אושר", bg="light green", fg="light green").pack(side="right") # FAKE LABEL
    tk.Label(previous_rides_frame_4_label, text=f"{ride[3]}", bg="light green", fg="light green").pack(side="right") # label of id ride
else:
    tk.Label(scrollbar_frame, text="אין", font=("Arial", 16, "bold"), bg="light green", fg="red").pack(side="top", anchor="ne")

previous_rides_page.mainloop()

def user_menu():
    # פונקציה שמציגה למשתמש ומפעילה את התפריט של המשתמש במסך הבית
    global root
    # יצירת תפריט
    menu = tk.Menu(root, tearoff=0, font=("Segoe UI", 11), background="#2c3e50", foreground="mint cream", activebackground="#34495e", activeforeground="white", borderwidth=2)

    menu.add_command(label="👤 פרופיל משתמש", command=user_information)
    menu.add_command(label="⭐ דירוג כוכבים", command=rate_stars)
    menu.add_separator()
    menu.add_command(label="🔌 התנתק", command=lambda: user_disconnect("התנתק"))

def show_menu(event):
    try:
        menu.tk_popup(event.x_root, event.y_root + 5)
    finally:
        menu.grab_release()

return show_menu

def user_information():
    # Page for user information
    user_information_page = tk.Toplevel() # Open a new window

    user_information_page.resizable(False, False) # Disable resizing the window
    # Create window size 500x500 and centered
```

יובל יוגב - Drive2Game

```
width, height = 500, 500
x = (user_information_page.winfo_screenwidth() - width) // 2
y = (user_information_page.winfo_screenheight() - height) // 2
user_information_page.geometry(f"{width}x{height}+{x}+{y}")

user_information_page.configure(bg="gray22") # Fill the background in dark gray color
user_information_page.title("User information") # title of the window

tk.Label(user_information_page, text="פרטים אישיים", font=("Tahoma", 50, "bold"),
bg="gray22", fg="silver").pack(side="top", pady=10)

global connect
#פרטי המשתמש
full_name = connect[0]
birth_date = connect[1]
username = connect[2]
password = connect[3]
email = connect[4]
phone_number = connect[5] if connect[5] != "" else "אין"
user_rate = connect[6] if connect[6] != "-1" else "עדיין לא דורגת"

tk.Label(user_information_page, text=f"שם מלא: {full_name}", font=("Ariel", 18,
"bold"), bg="gray22", fg="light coral").pack(side="top", anchor="ne")
tk.Label(user_information_page, text=f"תאריך לידה: {birth_date}", font=("Ariel", 18,
"bold"), bg="gray22", fg="light coral").pack(side="top", anchor="ne")
tk.Label(user_information_page, text=f"שם משתמש: {username}", font=("Ariel", 18,
"bold"), bg="gray22", fg="light coral").pack(side="top", anchor="ne")
tk.Label(user_information_page, text=f"אימייל: {email}", font=("Ariel", 18, "bold"),
bg="gray22", fg="light coral").pack(side="top", anchor="ne")
tk.Label(user_information_page, text=f"סיסמה: {password}", font=("Ariel", 18,
"bold"), bg="gray22", fg="light coral").pack(side="top", anchor="ne")
tk.Label(user_information_page, text=f"מספר טלפון: {phone_number}", font=("Ariel",
18, "bold"), bg="gray22", fg="light coral").pack(side="top", anchor="ne")
tk.Label(user_information_page, text=f"דירוג משתמש: {user_rate}", font=("Ariel", 18,
"bold"), bg="gray22", fg="light coral").pack(side="top", anchor="ne")

def change_user_information(field_name):
    global client
    global connect
    if field_name == "שם מלא":
        value = "Full_Name"
        old_detail = connect[0]
    elif field_name == "אימייל":
        value = "Email"
        old_detail = connect[4]
    elif field_name == "סיסמה":
        value = "Password"
```


יובל יוגב - Drive2Game

```
old_detail = connect[3]
else:
    value = "Phone_Number"
    old_detail = connect[5]

new_detail = simplifiedialog.askstring(f"X", f"ט202B/יזן {field_name} חדש:",
initialvalue=old_detail)

if new_detail != None:
    #Check if the new details written by the rules
    error_chnage = ""
    if field_name == "סיסמה":
        # Check if the password is fine
        if len(new_detail) < 7:
            error_chnage += "ח'הסיסמה צריכה לכלול לפחות 7 תווים"
        elif len(new_detail) > 25:
            error_chnage += "ח'הסיסמה יכולה להיות רק עד 25 תווים"
        elif not new_detail.isalnum() and not all(c in " for c in new_detail if not c.isalnum()):
            error_chnage += "ח'הסיסמה יכולה לכלול רק אותיות ומספרים"
    elif field_name == "מספר טלפון":
        # Check if the phone number is fine
        if len(new_detail) != 10:
            error_chnage += "ח'המספר טלפון צריך לכלול 10 תווים בדיוק"
        elif not new_detail.isalnum():
            error_chnage += "ח'ההמספר טלפון יכול לכלול רק מספרים"

    if error_chnage == "":

client.send_message(f"{{value}}|{{new_detail}}|{{old_detail}}|{{connect[2]}}|CHANGE_USER_DETA
ILS")

message = client.get_message()

if message == "THE_DETAIL_HAS_BEEN_CHANGED":
    showinfo("עודכן בהצלחה", f"{{field_name}}", f"{{field_name}}")

new_connect = []
for item in connect:
    new_connect.append(item)
if value == "Full_Name":
    new_connect[0] = new_detail
elif value == "Email":
    new_connect[4] = new_detail
elif value == "Password":
    new_connect[3] = new_detail
else:
    new_connect[5] = new_detail
```

יובל יוגב - Drive2Game

```
connect = tuple(new_connect)

user_information_page.destroy()
user_information()
else:
    showwarning("שגיאה", error_chnage)

# Create a Menubutton
mbutton = tk.Menubutton(user_information_page, text="את אחד חלחץ כאן על מנת לשנות  
הפרטים", font=("Ariel", 14, "bold"), bg="gray22", fg="light sky blue", bd=10, relief=tk.RAISED)
mbutton.pack(side="top", pady=20)

# Create a Menu
items = tk.Menu(mbutton, tearoff=0)
items.add_command(label="שם מלא", command=lambda: change_user_information("שם  
מלא"))
items.add_command(label="אימייל", command=lambda:
change_user_information("אימייל"))
items.add_command(label="סיסמה", command=lambda:
change_user_information("סיסמה"))
items.add_command(label="מספר טלפון", command=lambda:
change_user_information("מספר טלפון"))

# Attach the menu to the Menubutton
mbutton.config(menu=items)

user_information_page.mainloop()

def user_disconnect(disconnect=None):
    # פונקציה המנתקת את הלקוח
    global client
    client.send_message(f"LOGOUT")
    message = client.get_message()
    if message == "LOGOUT":
        client.disconnect() # אני לא בטוח שצריך לשחרר את הסוקט
        global root
        root.destroy()

    if disconnect is not None:
        client = None
        global connect
        connect = None # עושה שאף אחד לא מחובר
        root = tk.Tk()
        home_Page()
    else:
        showwarning("שגיאה", "יש בעיה, תנסו/ה שוב פעם")
```

יובל יוגב - Drive2Game

```
def rate_stars():
    # Page for stars rate
    stars_rate_page = tk.Toplevel() # Open a new window

    stars_rate_page.resizable(False, False) # Disable resizing the window
    # Create window size 500x750 and centered
    width, height = 500, 750
    x = (stars_rate_page.winfo_screenwidth() - width) // 2
    y = ((stars_rate_page.winfo_screenheight() - height) // 2) - 10
    stars_rate_page.geometry(f'{width}x{height}+{x}+{y}')

    stars_rate_page.configure(bg="#FFD700") # Fill the background in dark gray color
    stars_rate_page.title("User Rate") # title of the window

    stars_rate_frame = tk.Frame(stars_rate_page, bg="#FFD700")
    stars_rate_frame.pack(fill="both", expand=True)

    tk.Label(stars_rate_frame, text="דירוג כוכבים", font=("Tahoma", 35, "bold"), bg="#FFD700",
fg="Dim Gray").pack(side="top", pady=10)
    tk.Label(stars_rate_frame, text="תדרג את האנשים איתם נסעת", font=("Tahoma", 20, "bold"),
bg="#FFD700", fg="black").pack(side="top", pady=15)

    global connect
    global client

    rides_D = get_rides(f'{connect[2]}|1|FIND_D_RIDES')
    rides_P = get_rides(f'{connect[2]}|1|1|FIND_P_RIDES')
    users_to_rate = []

    def rate_the_user(event):
        new_rate = combo_box_rate.get()

        frame = event.widget.master # get the frame of the label
        labels_in_frame = [child for child in frame.winfo_children() if isinstance(child, tk.Label)]
# get all the labels in the frame
        text_labels = [label.cget("text") for label in labels_in_frame] # get the text of all the
labels in the frame
        username_of_user = text_labels[0].replace("\u202B", "")

        client.send_message(f'{username_of_user}|{new_rate}|{connect[2]}|SET_RATE')
        client.get_message()

    if rides_D != "There_are_no_rides":
        new_rides = []
        for ride in rides_D:
            id_ride = ride[3]
            new_rides.append(id_ride)
```

Drive2Game - יובל יוגב

```
for id_ride in new_rides:
    client.send_message(f'{id_ride}|F|T|GET_DRIVER_AND_PASSENGERS')
    message = client.get_message()

    s = message.split("$")
    ps_approved = [item.split("&")[0] for item in s[1].split("|")[1:]]
    username_driver = s[0].split("|")[0]

    if username_driver != connect[2]:
        if f'{username_driver}' not in users_to_rate:
            users_to_rate.append(f'{username_driver}')

    for p in ps_approved:
        if p != connect[2]:
            if f'{p}' not in users_to_rate:
                users_to_rate.append(f'{p}')

if rides_P != "There_are_no_rides":
    new_rides = []
    for ride in rides_P:
        id_ride = ride[3]
        new_rides.append(id_ride)

for id_ride in new_rides:
    client.send_message(f'{id_ride}|F|T|GET_DRIVER_AND_PASSENGERS')
    message = client.get_message()

    s = message.split("$")
    ps_approved = [item.split("&")[0] for item in s[1].split("|")[1:]]
    username_driver = s[0].split("|")[0].split("&")[0]

    if username_driver != connect[2]:
        if f'{username_driver}' not in users_to_rate:
            users_to_rate.append(f'{username_driver}')

    for p in ps_approved:
        if p != connect[2]:
            if f'{p}' not in users_to_rate:
                users_to_rate.append(f'{p}')

error_label = tk.Label(stars_rate_frame, text="", font=("Ariel", 14, "bold"), bg="#FFD700",
fg="#FFD700")
error_label.pack(side="top", padx=5)

if users_to_rate == []:
```

Drive2Game - יובל יוגב

```
error_label.config(text="לדרג מי לדרג", fg="black") #Update  
the label with error message
```

```
else:
```

```
    canvas_of_rate_page = tk.Canvas(stars_rate_frame, bg="#FFD700",  
highlightthickness=0)  
    scrollbar = tk.Scrollbar(stars_rate_frame, orient="vertical",  
command=canvas_of_rate_page.yview)
```

```
    scrollbar_frame = tk.Frame(canvas_of_rate_page, bg="#FFD700")
```

```
    scrollbar_frame.bind("<Configure>", lambda e:  
canvas_of_rate_page.configure(scrollregion=canvas_of_rate_page.bbox("all")))
```

```
    canvas_of_rate_page.create_window((0, 0), window=scrollbar_frame, anchor="nw")  
    canvas_of_rate_page.configure(yscrollcommand=scrollbar.set)
```

```
    canvas_of_rate_page.pack(side="left", fill="both", expand=True)  
    scrollbar.pack(side="right", fill="y")
```

```
    if "" in users_to_rate:  
        users_to_rate.remove("")
```

```
    count_rate = 0  
    for username in users_to_rate:  
        client.send_message(f'{username}|USERS_ALREADY_RATED_SOME_USER')  
        users_already_rated = client.get_message()
```

```
    if users_already_rated == "HAS_NOT_RATED_YET" or connect[2] not in  
users_already_rated:  
        count_rate += 1
```

```
    frame_passenger_and_driver = tk.Frame(scrollbar_frame, bg="#FFD700")  
    frame_passenger_and_driver.pack(side="top", fill='x')
```

```
    tk.Label(frame_passenger_and_driver, text=f"\u202B{username}", font=("Ariel",  
14, "bold"), bg="#FFD700", fg="black").pack(side="left", padx=5)
```

```
    combo_box_rate = ttk.Combobox(frame_passenger_and_driver, values=["1", "2",  
"3", "4", "5"])
```

```
    combo_box_rate.pack()  
    combo_box_rate.bind("<<ComboboxSelected>>", rate_the_user)
```

```
    if count_rate == 0:  
        error_label.config(text="דירגת כבר את כל האנשים שאיתם נסעת", fg="black") #Update the  
label with error message
```

```
stars_rate_page.mainloop()
```

יובל יוגב - Drive2Game

```
def back_Arrow_clicked(event):
    # פונקציה שנקראת לאחר לחיצה על כפתור החץ של לחזור אחורה והיא מחזירה אחורה
    root_of_event_isFrom = event.widget.winfo_toplevel() # widget של ה-root - מקבל את ה-
    title_root = root_of_event_isFrom.title() # root של ה-
    root_of_event_isFrom.destroy()
    global list_of_games
    if title_root == "Sign up":
        # אז סימן שהמשתמש היה בחלון של הרשמה והוא רוצה לחזור Sign up היא root האם הכותרת של ה-
        לחלון ההתחברות
        sign_up_or_log_in()
    elif title_root == "Driver Page":
        # אז סימן שהמשתמש היה בחלון של יצירת נסיעה והוא Driver Page היא root האם הכותרת של ה-
        רוצה לחזור לחלון של בחירת משחק
        driver(event, list_of_games)
    elif title_root == "Passenger Page":
        # אז סימן שהמשתמש היה בחלון של בחירת הסעה Passenger Page היא root האם הכותרת של ה-
        והוא רוצה לחזור לחלון של בחיר משחק
        passenger(event, list_of_games)
    if __name__ == "__main__":
        #main
        create_client() #create a client and connect him to the server
        root = tk.Tk() # To initialize Tkinter, we have to create a Tk root widget.
        connect = None #מחובר או לא מחובר
        home_Page()
```

Client_server.py

```
import socket
import select
import DB
import Encryption as e
from email.message import EmailMessage
import ssl
import smtplib
import datetime

class Client_Server():
    def __init__(self, ip):
        self.ip = ip # Server's IP address.
        self.port = 1729 # Port to bind the server socket.
        self.e_Encryption = e.Encryption('al_sweigart_pubkey.txt', 'al_sweigart_privkey.txt')
```

יובל יוגב - Drive2Game

```
class Server(Client_Server):
    def __init__(self, ip):
        Client_Server.__init__(self, ip)
        self.server_socket = None # The main server socket.
        self.rlist, self.wlist, self.xlist = None, None, None
        self.clients = [] # Socket list for communication with the connected clients.
        self.messages = []
        self.date = "08/03/2025" #datetime.date.today().strftime("%d/%m/%Y")

    def start(self):
        # Initializes and starts the server.
        try:
            self.server_socket = socket.socket() # Creates a new socket object.
            self.server_socket.bind((self.ip, self.port)) # Binds the socket to a specific IP address
            and port.
            print(f"Server started on {self.ip}:{self.port}")
            self.server_socket.listen() # Puts the socket into listening mode, meaning it's ready
            to accept incoming connection requests.

            self.check_the_dates() #Check if there are rides unavailable because the date has
            passed.

            self.running_server() #if the server is connected the server will start running
        except Exception as e:
            print(f"Error starting server: {e}")

    def running_server(self):
        while True:
            #זה בהערה בגלל שהאתר ממנו המשחקים נשלפים הוא אופליין ולכן יש הסעות ומשחקים שהתאריך
            שלהם כבר עבר מלפני כמה חודשים
            ""#always checking if a day has passed
            if datetime.datetime.strptime(self.date, "%d/%m/%Y").date() < datetime.date.today():
                self.date = datetime.date.today().strftime("%d/%m/%Y") #new date
                self.check_the_dates()

            self.rlist, self.wlist, self.xlist = select.select([self.server_socket] + self.clients,
            self.clients, [])
            for client in self.rlist:
                if client is self.server_socket:
                    print('new client joined')
                    c, address = client.accept()
                    self.clients.append(c)
                else:
                    message = client.recv(4096).decode()
                    message = self.e_Encryption.get_message(message) # Decrypt the message
                    after get it
```

Drive2Game - יובל יוגב

```
if message != "":
    self.send_Message_and_Protocol(message, client)

def send_Message_and_Protocol(self, message, client):
    #The message template is values between the symbol | and the last value is the
protocol
    m = message.split('|')
    p = m[-1]

    if p == "SIGNUP":
        #sign up the client and save in database & send to the client a message about it
        try:
            user_values = m[0].replace('(', '').replace(')', '').replace("'", '').replace(' ',
            ").replace('+', ' ').split(',')
            user_values[-1] = int(user_values[-1])
            if user_values[-2] == "": user_values[-2] = None # מספר טלפון יכול להיות None
            user_values = tuple(user_values)

            with DB.Database_Manager("My_Project_DB.db") as db_users: # Connect to the
data base
                e = db_users.execute_query("INSERT INTO Users VALUES (?, ?, ?, ?, ?, ?, ?)",
user_values) # Insert to the table Users the values
                db_users.commit() # Save the db file after the changes
            finally:
                # Check if there is an Error of usage values like username and email
                e = str(e)
                if e.split(':')[0].split(' ')[0] == "UNIQUE":
                    if e.split(':')[1].split(' ')[-1] == "Email": e = "האימיל"
                    elif e.split(':')[1].split(' ')[-1] == "Username": e = "שם המשתמש"

                message = f"{e} חללקח כבר על ידי מישהו אחר, אנא בחר אחר"
                message = self.e_Encryption.send_message(message) # encrypt the message
before sending it
                client.send(message.encode()) #send to the client the errors
                print(e)
            else:
                message = "CONNECT"
                message = self.e_Encryption.send_message(message) # encrypt the message
before sending it
                client.send(message.encode()) #send to the client that now he is connected
                print("Connect...")

        elif p == "LOGIN":
            #log in the client and serach in database & send to the client a message about it
            try:
                email_login = m[0]
                password_login = m[1]
```


יובל יוגב - Drive2Game

```
with DB.Database_Manager("My_Project_DB.db") as db_users: # Connect to the
database
    # Execute a query to check if the email and password match
    db_users.execute_query("SELECT Password FROM Users WHERE Email = ?",
(email_login,))
    result = db_users.fetchall()

    if result:
        # Extract the password from the query result
        stored_password = result[0][0]
        if stored_password == password_login:
            db_users.execute_query("SELECT * FROM Users WHERE Email = ?",
(email_login,))
            message = f"{db_users.fetchall()}|CONNECT"
            message = self.e_Encryption.send_message(message) # encrypt the
message before sending it
            client.send(message.encode()) #Mark user as connected
        else:
            message = "הסיסמה שגויה"
            message = self.e_Encryption.send_message(message) # encrypt the
message before sending it
            client.send(message.encode())
        else:
            message = "האימייל לא קיים"
            message = self.e_Encryption.send_message(message) # encrypt the
message before sending it
            client.send(message.encode())
    except Exception as e:
        print(f"An error occurred: {e}")

elif p == "CREATE_RIDE":
    #create a ride and save in database & send to the client a message about it
    selected_game = f"🏀{m[0]}"
    username_driver = m[4]
    start_driving_address = m[1]
    num_passenger = m[2]
    text_comments = m[3]

    try:
        with DB.Database_Manager("My_Project_DB.db") as db_rides: # Connect to the
data base
            db_rides.execute_query("SELECT COUNT (*) FROM Rides")
            id_ride = db_rides.fetchall() # Collect how many rides there are in the table
            id_ride = id_ride[0][0] + 1 # Add 1 to the rides
            game_info = selected_game.split(',')
            game = game_info[0]
```

Drive2Game - יובל יוגב

```

date = game_info[1].split(' ')[-1]
hour = game_info[2].split(' ')[-1] if len(game_info[2].split(' ')) == 3 else "לא ידוע"
place = game_info[3] if len(game_info) == 4 else f"{game_info[3]},
{game_info[4]}"

# check if the same user hasn't already added a ride for the same game
db_rides.execute_query("SELECT * FROM Rides WHERE game = ? AND date
= ? AND hour = ? AND place = ? AND username_driver = ?", (game, date, hour, place,
username_driver))
is_already_exist = db_rides.fetchall()

if is_already_exist != []: message = "THE_USER_ALREADY_HAS_A_RIDE"

else:
    ride = (id_ride, username_driver, game, date, hour, place,
start_driving_address, num_passenger, text_comments, 0)
    # Insert to the table Rides the values
    db_rides.execute_query("INSERT INTO Rides VALUES (?, ?, ?, ?, ?, ?, ?, ?,
?, ?)", ride)
    db_rides.commit() # Save the db file after the changes
    message = "SAVE_RIDE"

finally:
    message = self.e_Encryption.send_message(message) # encrypt the message
before sending it
    client.send(message.encode())

elif p == "SEND_THE_RIDES":
    #search all the rides to the specific game and send to the client a message about it.
It will also search about city or address
    selected_game = f"🏀{m[0].split(',')[0]}"
    username = m[1]
    city = ""
    if len(m) == 4:
        city = m[2].split(": ")[1].split(". ")[0]
        print(city)
        print(m)

    try:
        city = f"%{city}%"
        with DB.Database_Manager("My_Project_DB.db") as db_rides: # Connect to the
data base
            db_rides.execute_query("SELECT Id_ride, Start_ride_location, Comments,
Num_passengers, Username_driver FROM Rides WHERE Game = ? AND Status = ? And
Start_ride_location like ? AND Username_driver != ?", (selected_game, 0, city, username))
            rides = db_rides.fetchall() # Collect the rides

        finally:
            if len(rides) > 0:

```

יובל יוגב - Drive2Game

```
message = f"{rides}|GET RIDES"
message = self.e_Encryption.send_message(message) # encrypt the message
before sending it
client.send(message.encode())
else:
    message = "There_are_no_rides"
    message = self.e_Encryption.send_message(message) # encrypt the message
before sending it
client.send(message.encode())

elif p == "COUNT_PASSENGER":
    #search how many passengers are in the ride
    id_ride = m[0]
    try:
        with DB.Database_Manager("My_Project_DB.db") as db_rides: # Connect to the
data base
            db_rides.execute_query("SELECT COUNT (*) FROM Passengers WHERE
Id_ride = ? AND Approved = ?;", (id_ride, 1))
            num_of_passengers = db_rides.fetchall() # Collect the num of passengers
    finally:
        if num_of_passengers is None:
            message = "0"
            message = self.e_Encryption.send_message(message) # encrypt the message
before sending it
            client.send(message.encode())
        else:
            message = f"{num_of_passengers[0][0]}"
            message = self.e_Encryption.send_message(message) # encrypt the message
before sending it
            client.send(f"{message}".encode())

elif p == "GET_RATE":
    # send how many rae stars the user has
    username = m[0].split(',')[0]
    try:
        with DB.Database_Manager("My_Project_DB.db") as db_rides: # Connect to the
data base
            db_rides.execute_query(f"SELECT Rate_stars FROM Users WHERE
Username = ?;", (username,))
            rate_stars = db_rides.fetchall() # Collect the rate stars of the user
    finally:
        if rate_stars[0][0] == -1:
            message = "נסיעה ראשונה"
            message = self.e_Encryption.send_message(message) # encrypt the message
before sending it
            client.send(message.encode())
        else:
```

Drive2Game - יובל יוגב

```
message = f"{rate_stars[0][0]} כוכבים"
message = self.e_Encryption.send_message(message) # encrypt the message
before sending it
client.send(message.encode())

elif p == "JOIN_RIDE":
    #attach the client to a ride and save in database & send to the client a message
    about it
    username_passenger = m[0]
    id_ride = m[1]
    pick_up_location = m[2]
    error = ""

    try:
        with DB.Database_Manager("My_Project_DB.db") as db_rides: # Connect to the
data base
            db_rides.execute_query("SELECT Username_driver FROM Rides WHERE
Id_ride = ?", (id_ride,))
            username_driver = db_rides.fetchall()[0][0] # Collect the rides
            if username_passenger != username_driver:
                passenger_ride = (username_passenger, id_ride, pick_up_location, 0)
                error = db_rides.execute_query("INSERT INTO Passengers VALUES (?, ?, ?,
?)", passenger_ride) # Insert to the table Passengers the values
                db_rides.commit() # Save the db file after the changes
            else:
                error = "THE_PASSENGER_IS_DRIVER"

    finally:
        if error is None:
            message = "REQUEST_SEND"
            message = self.e_Encryption.send_message(message) # encrypt the message
before sending it
            client.send(message.encode())
        else:
            message = f"{error}"
            message = self.e_Encryption.send_message(message) # encrypt the message
before sending it
            client.send(message.encode())

elif p == "FIND_D_RIDES":
    #find rides that the client is a driver and send to the client a message about it
    username = m[0]
    status = m[1]
    try:
        with DB.Database_Manager("My_Project_DB.db") as db_rides: # Connect to the
data base
            db_rides.execute_query("SELECT Game, Date, Hour, Id_ride FROM Rides
WHERE Username_driver = ? AND Status = ? ORDER BY Date, Hour;", (username, status))
```

Drive2Game - יובל יוגב

```
rides = db_rides.fetchall() # Collect the rides of the user
finally:
    if len(rides) > 0:
        new_rides = []
        for ride in rides:
            new_ride = (ride[0].replace('🏀', ''), ride[1], ride[2], ride[3])
            new_rides.append(new_ride)

        message = f'{new_rides}|GET RIDES'
        message = self.e_Encryption.send_message(message) # encrypt the message
before sending it
        client.send(message.encode())
    else:
        message = "There_are_no_rides"
        message = self.e_Encryption.send_message(message) # encrypt the message
before sending it
        client.send(message.encode())

elif p == "FIND_P_RIDES":
    #find rides that the client is a passenger and send to the client a message about it
    username = m[0]
    status = int(m[1])
    approved = m[2]
    try:
        query = "SELECT Id_ride FROM Passengers WHERE Username_passenger = ?"
        values = (username,)
        if approved != "":
            query += " AND Approved = ?"
            values += (int(approved),)
        with DB.Database_Manager("My_Project_DB.db") as db_rides: # Connect to the
data base
            db_rides.execute_query(query, values)
            id_rides = db_rides.fetchall() # Collect the id rides of the user
            rides = []
            if len(id_rides) > 0:
                for id_ride in id_rides:
                    id_ride = id_ride[0]
                    db_rides.execute_query("SELECT Game, Date, Hour, Id_ride FROM Rides
WHERE Id_ride = ? AND Status = ? ORDER BY Date, Hour;", (id_ride, status))
                    ride = db_rides.fetchall()
                    if ride != []:
                        rides.append(ride[0]) # Collect the rides of the id rides
            finally:
                if len(rides) > 0:
                    new_rides = []
                    for ride in rides:
                        new_ride = (ride[0].replace('🏀', ''), ride[1], ride[2], ride[3])
```

Drive2Game - יובל יוגב

```
new_rides.append(new_ride)

message = f"{new_rides}|GET RIDES"
message = self.e_Encryption.send_message(message) # encrypt the message
before sending it
client.send(message.encode())
else:
    message = "There_are_no_rides"
    message = self.e_Encryption.send_message(message) # encrypt the message
before sending it
client.send(message.encode())

elif p == "GET_NUM_OF_PASSENGERS":
    #get the num of passengers of specific ride
    id_ride = m[0]
    status = int(m[1])
    try:
        with DB.Database_Manager("My_Project_DB.db") as db_rides: # Connect to the
data base
            db_rides.execute_query(f"SELECT COUNT(*) FROM Passengers WHERE
Id_ride = ? AND Approved = ?;", (id_ride, status))
            num_of_passengers = db_rides.fetchall() # Collect the num of passengers for
the specific ride by the id ride
    finally:
        num_of_passengers = num_of_passengers[0][0]
        if num_of_passengers > 0:
            message = f"{num_of_passengers}|GET_NUM_OF_PASSENGERS"
            message = self.e_Encryption.send_message(message) # encrypt the message
before sending it
            client.send(message.encode())
        else:
            message = "There_are_no_passengers"
            message = self.e_Encryption.send_message(message) # encrypt the message
before sending it
            client.send(message.encode())

elif p == "GET_PASSENGER_STATUS_OF_RIDE":
    # get the status of specific ride for specific passenger
    id_ride = m[0]
    username_passenger = m[1]
    try:
        with DB.Database_Manager("My_Project_DB.db") as db_rides: # Connect to the
data base
            db_rides.execute_query(f"SELECT Approved FROM Passengers WHERE
Username_passenger = ? AND Id_ride = ?;", (username_passenger, id_ride))
            status = db_rides.fetchall() # Collect the status of the specific ride and
passenger
```

Drive2Game - יובל יוגב

```
finally:
    message = f"{status}|GET_STATUS"
    message = self.e_Encryption.send_message(message) # encrypt the message
before sending it
    client.send(message.encode())

elif p == "GET_DRIVER_AND_PASSENGERS":
    #get the driver and the passengers of the ride
    id_ride = m[0]
    is_driver = m[1]
    if is_driver != "T":
        is_p_approved = m[2]

    #get the driver
    try:
        with DB.Database_Manager("My_Project_DB.db") as db_rides: # Connect to the
data base
            db_rides.execute_query("SELECT Username_driver, Place, Start_ride_location,
Comments, Num_passengers FROM Rides WHERE Id_ride = ?;", (id_ride,))
            results = db_rides.fetchall()

        with DB.Database_Manager("My_Project_DB.db") as db_rides: # Connect to the
data base
            db_rides.execute_query("SELECT Email FROM Users WHERE Username =
?;", (results[0][0],))
            email = db_rides.fetchall()
    finally:
        username_driver = results[0][0]
        place = results[0][1]
        start_ride_loc = results[0][2]
        comments = results[0][3]
        num_passengers = results[0][4]

        email = email[0][0]

    message =
f"{username_driver}|{place}|{start_ride_loc}|{comments}|{num_passengers}|{email}"

    #get the passengers
    if is_driver == "T": #if the user is the driver
        try:
            with DB.Database_Manager("My_Project_DB.db") as db_rides: # Connect to
the data base
                db_rides.execute_query("SELECT P.Username_passenger,
P.Pick_up_location, U.Email FROM Passengers P JOIN Users U ON
P.Username_passenger = U.Username WHERE Id_ride = ? AND Approved = ?;", (id_ride,
1))
```

Drive2Game – יובל יוגב

```
passengers = db_rides.fetchall()
finally:
    if passengers != []:
        passengers = "|".join("&".join(passenger) for passenger in passengers)
    else:
        passengers = ""
    message += f"$APPROVED|{passengers}"
try:
    with DB.Database_Manager("My_Project_DB.db") as db_rides: # Connect to
the data base
        db_rides.execute_query("SELECT Username_passenger, Pick_up_location
FROM Passengers WHERE Id_ride = ? AND Approved = ?;", (id_ride, 0))
        passengers = db_rides.fetchall()
    finally:
        if passengers != []:
            passengers = "|".join("&".join(passenger) for passenger in passengers)
        else:
            passengers = ""
        message += f"$WAITING|{passengers}"
try:
    with DB.Database_Manager("My_Project_DB.db") as db_rides: # Connect to
the data base
        db_rides.execute_query("SELECT Username_passenger, Pick_up_location
FROM Passengers WHERE Id_ride = ? AND Approved = ?;", (id_ride, 2))
        passengers = db_rides.fetchall()
    finally:
        if passengers != []:
            passengers = "|".join("&".join(passenger) for passenger in passengers)
        else:
            passengers = ""
        message += f"$REJECT|{passengers}"

elif is_p_approved == "T": #if the user is passenger on approved
try:
    with DB.Database_Manager("My_Project_DB.db") as db_rides: # Connect to
the data base
        db_rides.execute_query("SELECT P.Username_passenger, U.Email FROM
Passengers P JOIN Users U ON P.Username_passenger = U.Username WHERE Id_ride =
? AND Approved = ?;", (id_ride, 1))
        passengers = db_rides.fetchall()
    finally:
        if passengers != []:
            passengers = "|".join("&".join(passenger) for passenger in passengers)
        else:
            passengers = ""
        message += f"$APPROVED|{passengers}"
```


Drive2Game - יובל יוגב

```
message = self.e_Encryption.send_message(message) # encrypt the message
before sending it
client.send(message.encode())

elif p == "CHANGE_PASSENGER_STATUS_OF_RIDE":
    # change the status of specific ride for specific passenger
    id_ride = m[0]
    username_passenger = m[1]
    status = int(m[2])

    query = f"UPDATE Passengers SET Approved = ? WHERE Id_ride = ? AND "
    values = (status, id_ride)
    if username_passenger != "":
        query += f"Username_passenger = ?" # to approved or reject the passenger
        values += (username_passenger,)
    else:
        query += f"Approved = ?" # to reject all the passengers
        values += (0,)

    try:
        with DB.Database_Manager("My_Project_DB.db") as db_rides: # Connect to the
data base
            db_rides.execute_query(query, values)
            db_rides.commit() # Save the db file after the change
    finally:
        message = f"THE_STATUS_HAS_BEEN_CHANGED"
        message = self.e_Encryption.send_message(message) # encrypt the message
before sending it
        client.send(message.encode())

elif p == "CHANGE_USER_DETAILS":
    #change the user details by the specific item to the new detail
    value = m[0]
    new_detail = m[1]
    old_detail = m[2]
    username = m[3]

    try:
        with DB.Database_Manager("My_Project_DB.db") as db_rides: # Connect to the
data base
            db_rides.execute_query(f"UPDATE Users SET {value} = ? WHERE {value} = ?
AND Username = ?;", (new_detail, old_detail, username))
            db_rides.commit() # Save the db file after the change
    finally:
        message = f"THE_DETAIL_HAS_BEEN_CHANGED"
        message = self.e_Encryption.send_message(message) # encrypt the message
before sending it
```

יובל יוגב - Drive2Game

```
client.send(message.encode())

elif p == "SEND_EMAIL_WITH_CODE":
    #get the email of the user and send email with code
    email_receiver = m[0]
    code = m[1]

    email_sender = 'yuvalwork11@gmail.com' # האימייל ממנו ישלחו ההודעות
    email_password = 'foqj albj snbn zich' # הסיסמא רק **לשליחה** דרך האימייל הזה
    subject = 'CODE FOR DRIVE2GAME' # נושא ההודעה

    em = EmailMessage() # בניית אובייקט
    em['From'] = email_sender # קובע ממי תשלח ההודעה
    em['To'] = email_receiver # קובע מי יקבל את ההודעה
    em['Subject'] = subject # קובע את נושא ההודעה
    # יצירת ההודעה, איך היא תראה - עיצוב
    em.add_alternative(f"""\
<html>
  <body style="font-family: Arial, sans-serif; background-color: #e0e0e0; padding:
20px; direction: rtl;"> <!-- יישור טקסט לימין -->
    <div style="max-width: 520px; margin: auto; background-color: white; padding:
30px; border-radius: 10px; box-shadow: 0 4px 12px rgba(0,0,0,0.1);">
      <h2 style="color: #d32f2f; text-align: center;">🚗 Drive2Game 🏀🏆</h2> <!--
--> כותרת מרכזית
      <p style="font-size: 16px; color: #333; text-align: right;">
        אנא הזן, <strong>Drive2Game</strong> -שלום! כדי להשלים את תהליך ההתחברות ל
את קוד האימות הבא:
      </p>
      <div style="font-size: 28px; font-weight: bold; color: #d32f2f; text-align: center;
margin: 20px 0;">
        {code}
      </div>
      <!-- הקוד להתחברות אדום ומרוכז באמצע -->
      <hr style="margin: 30px 0;"> <!-- קו מפריד -->
      <p style="font-size: 12px; color: #7b1fa2; text-align: center;">
        Drive2Game<br>
        <em>YOU'LL NEVER DRIVE ALONE</em>
      </p>
    </div>
  </body>
</html>
""", subtype='html')

    context = ssl.create_default_context() # הצפנת ההודעה

    # שליחת ההודעה
```

Drive2Game - יובל יוגב

```
try:
    with smtplib.SMTP_SSL('smtp.gmail.com', 465, context=context) as smtp:
        smtp.login(email_sender, email_password)
        smtp.sendmail(email_sender, email_receiver, em.as_string())
finally:
    message = f"THE_CODE_HAS_BEEN_SENT"
    message = self.e_Encryption.send_message(message) # encrypt the message
before sending it
    client.send(message.encode())

elif p == "USERS_ALREADY_RATED_SOME_USER":
    #get username and send back all the users that rated him
    username = m[0]

    try:
        with DB.Database_Manager("My_Project_DB.db") as db_users: # Connect to the
data base
            db_users.execute_query(f"SELECT [Who_rate?] FROM Users_rate WHERE
Username = ?;", (username,))
            users_already_rated = db_users.fetchall() # Collect the users that already rated
this user
    finally:
        if users_already_rated != []:
            message = f"{users_already_rated[0][0]}"
        else:
            message = "HAS_NOT_RATED_YET"
        message = self.e_Encryption.send_message(message) # encrypt the message
before sending it
        client.send(message.encode())

elif p == "SET_RATE":
    # get the new rate of the user and update
    username = m[0].split(', ')[-1]
    new_rate = m[1]
    new_rater = m[2]

    try:
        with DB.Database_Manager("My_Project_DB.db") as db_users: # Connect to the
data base
            db_users.execute_query("SELECT Total_rate, How_many_rate?, Who_rate?
FROM Users_rate WHERE Username = ?;", (username,))
            result = db_users.fetchall() # Collect the rate stars of the user

    if result == []:
        total_rate, how_many_rate, all_raters = None, None, None
    else:
        total_rate = result[0][0]
```

יובל יוגב - Drive2Game

```
how_many_rate = result[0][1]
all_raters = result[0][2]

if total_rate is None:
    total_rate = new_rate
    how_many_rate = "1"
    user_rate = new_rate

    with DB.Database_Manager("My_Project_DB.db") as db_users: # Connect to
the data base
        db_users.execute_query("INSERT INTO Users_rate VALUES (?, ?, ?, ?);",
(username, total_rate, how_many_rate, new_rater))
        db_users.commit() # Save the db file after the changes
    else:
        total_rate = f"{int(total_rate) + int(new_rate)}"
        how_many_rate = f"{int(how_many_rate) + 1}"
        user_rate = f"{int(total_rate)//int(how_many_rate)}" #מעגל למטה
        all_raters += f", {new_rater}"

    with DB.Database_Manager("My_Project_DB.db") as db_users: # Connect to
the data base
        db_users.execute_query("UPDATE Users_rate SET Total_rate = ? AND
How_many_rate? = ? AND Who_rate? = ? WHERE Username = ?;", (total_rate,
how_many_rate, all_raters, username))
        db_users.commit() # Save the db file after the changes

    with DB.Database_Manager("My_Project_DB.db") as db_users: # Connect to the
data base
        db_users.execute_query(f"UPDATE Users SET Rate_stars = ? WHERE
Username = ?;", (user_rate, username))
        db_users.commit() # Save the db file after the changes
    finally:
        message = "THE_RATE_HAS_BEEN_CHANGED"
        message = self.e_Encryption.send_message(message) # encrypt the message
before sending it
        client.send(message.encode())

    elif p == "LOGOUT":
        #remove the client from the connectting clients list & sent to the client a message
about it
        if client in self.clients:
            message = f"LOGOUT"
            message = self.e_Encryption.send_message(message) # encrypt the message
before sending it
            client.send(message.encode())

        self.clients.remove(client) # מוחק את הלקוח מהרשימה
```

Drive2Game - יובל יוגב

```
client.close() #סוגר את הסוקט בין השרת אל הלקוח  
print("Client left")
```

```
def check_the_dates(self):  
    #check the dates in the database and if they are passed, if there are so the status of  
    them would change  
    try:  
        with DB.Database_Manager("My_Project_DB.db") as db_rides: # Connect to the  
            data base  
            db_rides.execute_query(f"SELECT Id_ride, Date FROM Rides WHERE Status = 0  
ORDER BY Date;")  
            rides = db_rides.fetchall() # Collect the rides  
  
            for ride in rides:  
                if datetime.datetime.strptime(ride[1], "%d/%m/%Y").date() <  
datetime.datetime.strptime(self.date, "%d/%m/%Y").date():  
                    db_rides.execute_query(f"UPDATE Rides SET Status = 1 WHERE Id_ride =  
{ride[0]};")  
                    db_rides.commit() # Save the db file after the changes
```

```
    except Exception as e:  
        print(f"Error connecting to server: {e}")
```

```
class Client(Client_Server):  
    def __init__(self, ip):  
        Client_Server.__init__(self, ip)  
        self.client_socket = None # The client's socket for communication.  
  
    def connect(self):  
        # Initializes and connects the client to the server.  
        try:  
            self.client_socket = socket.socket() # Creates a new socket object.  
            self.client_socket.connect(  
                (self.ip, self.port)) # Connects to the server located at the specified IP address  
and port.  
        except Exception as e:  
            print(f"Error connecting to server: {e}")  
  
    def send_message(self, message):  
        # Sends a message to the server.  
        try:  
            message = self.e_Encryption.send_message(message) #encrypt the message  
before sending it  
            self.client_socket.send(message.encode()) # Encodes the string of message into  
bytes and sends it to the server through the client socket.  
        except Exception as e:
```

Drive2Game - יובל יוגב

```
print(f"Error sending message: {e}")

def get_message(self):
    # Receives a message from the server.
    try:
        message = self.client_socket.recv(4096).decode() # Get the message from the
server
        message = self.e_Encryption.get_message(message) # Decrypt the message after
get it
        return message
    except Exception as e:
        print(f"Error receiving message: {e}") # return nothing (None)

def disconnect(self):
    # Closes the client socket.
    if self.client_socket:
        self.client_socket.close()
        print("You are disconnected")
```

Encryption.py

```
import sys
```

```
from certifi import where
```

```
DEFAULT_BLOCK_SIZE = 128
```

```
BYTE_SIZE = 256
```

```
class Encryption:
```

```
    def __init__(self, pub_key_file, priv_key_file, block_size=DEFAULT_BLOCK_SIZE):
```

```
        self.pub_key_file = pub_key_file
```

```
        self.priv_key_file = priv_key_file
```

```
        self.block_size = block_size
```

```
        self.hebrew_to_english = {'א': 'a', 'ב': 'b', 'ג': 'c', 'ד': 'd', 'ה': 'e', 'ו': 'f', 'ז': 'g', 'ח': 'h',
                                'ט': 'i', 'י': 'j', 'כ': 'k', 'ל': 'l', 'מ': 'm', 'נ': 'n', 'ס': 'o', 'ע': 'p',
                                'פ': 'q', 'צ': 'r', 'ק': 's', 'ר': 't', 'ש': 'u', 'ת': 'v', 'ם': 'w', 'ן': 'x',
                                'ך': 'y', 'ז': 'z', 'א': 'A'}
```

```
        self.english_to_hebrew = {'a': 'א', 'b': 'ב', 'c': 'ג', 'd': 'ד', 'e': 'ה', 'f': 'ו', 'g': 'ז', 'h': 'ח',
                                'i': 'ט', 'j': 'י', 'k': 'כ', 'l': 'ל', 'm': 'מ', 'n': 'נ', 'o': 'ס', 'p': 'ע',
                                'q': 'פ', 'r': 'צ', 's': 'ק', 't': 'ר', 'u': 'ש', 'v': 'ת', 'w': 'ם', 'x': 'ן',
                                'y': 'ך', 'z': 'ז', 'A': 'א'}
```

```
    def send_message(self, message):
```

```
        key_size, n, e = self.read_key_file(self.pub_key_file)
```

```
        if key_size < self.block_size * 8:
```

```
            sys.exit('ERROR: Block size too large for key size.')
```

Drive2Game - יובל יוגב

```
new_message = ""
when_hebrew = []
index = 0
for letter in message:
    if not letter.isascii():
        when_hebrew.append(index)
        new_letter = self.hebrew_to_english[letter]
        new_message += new_letter
    else:
        new_message += letter
    index += 1

message = new_message

encrypted_blocks = self.encrypt_message(message, (n, e))
encrypted_content = ','.join(map(str, encrypted_blocks))
encrypted_content =
f'{len(message)}_{self.block_size}_{encrypted_content}_{','.join(map(str, when_hebrew))}'

return encrypted_content

def get_message(self, encrypted_text):
    key_size, n, d = self.read_key_file(self.priv_key_file)
    message_length, block_size, encrypted_data, when_hebrew = encrypted_text.split('_')
    message_length = int(message_length)
    block_size = int(block_size)

    when_hebrew = when_hebrew.split(',')
    hebrew_index = []
    for i in when_hebrew:
        if i != "":
            hebrew_index.append(int(i))

    if key_size < block_size * 8:
        sys.exit('ERROR: Block size too large for key size.')

    encrypted_blocks = [int(block) for block in encrypted_data.split(',')]

    message = self.decrypt_message(encrypted_blocks, message_length, (n, d),
    block_size)

    new_message = ""
    index = 0
    for letter in message:
        if index in hebrew_index:
            new_letter = self.english_to_hebrew[letter]
            new_message += new_letter
```

Drive2Game - יובל יוגב

```
    else:
        new_message += letter
    index += 1

return new_message

def read_key_file(self, filename):
    with open(filename) as f:
        content = f.read()
    key_size, n, EorD = content.split(',')
    return int(key_size), int(n), int(EorD)

def get_blocks_from_text(self, message):
    message_bytes = message.encode('ascii')
    blocks = []
    for block_start in range(0, len(message_bytes), self.block_size):
        block_int = 0
        for i in range(block_start, min(block_start + self.block_size, len(message_bytes))):
            block_int += message_bytes[i] * (BYTE_SIZE ** (i % self.block_size))
        blocks.append(block_int)
    return blocks

def get_text_from_blocks(self, blocks, message_length, block_size):
    message = []
    for block_int in blocks:
        block_message = []
        for i in range(block_size - 1, -1, -1):
            if len(message) + i < message_length:
                ascii_num = block_int // (BYTE_SIZE ** i)
                block_int %= (BYTE_SIZE ** i)
                block_message.insert(0, chr(ascii_num))
        message.extend(block_message)
    return ''.join(message)

def encrypt_message(self, message, key):
    n, e = key
    blocks = self.get_blocks_from_text(message)
    return [pow(block, e, n) for block in blocks]

def decrypt_message(self, encrypted_blocks, message_length, key, block_size):
    n, d = key
    decrypted_blocks = [pow(block, d, n) for block in encrypted_blocks]
    return self.get_text_from_blocks(decrypted_blocks, message_length, block_size)
```


יובל יוגב - Drive2Game

DB.py

```
import sqlite3
```

```
class Database_Manager:
```

```
    def __init__(self, db_path):
```

```
        self.db_path = db_path
```

```
        self.connection = None
```

```
        self.cursor = None
```

```
    def connect(self):
```

```
        try:
```

```
            self.connection = sqlite3.connect(self.db_path)
```

```
            self.cursor = self.connection.cursor()
```

```
        except sqlite3.Error as e:
```

```
            print(f"Failed to connect to {self.db_path}: {e}")
```

```
            raise
```

```
    def execute_query(self, query, params=None):
```

```
        if not self.cursor:
```

```
            raise ValueError("Database connection is not established.")
```

```
        try:
```

```
            if params:
```

```
                self.cursor.execute(query, params)
```

```
            else:
```

```
                self.cursor.execute(query)
```

```
        except sqlite3.Error as e:
```

```
            return e
```

```
    def fetchall(self):
```

```
        if not self.cursor:
```

```
            raise ValueError("Cursor is not initialized.")
```

```
        return self.cursor.fetchall()
```

```
    def commit(self):
```

```
        if self.connection:
```

```
            self.connection.commit()
```

```
    def close(self):
```

```
        if self.connection:
```

```
            self.connection.close()
```

```
            self.connection = None
```

```
            self.cursor = None
```

```
# Add these methods to support the 'with' statement
```

```
def __enter__(self):
```

```
    self.connect()
```

יובל יוגב - Drive2Game

```
return self
```

```
def __exit__(self, exc_type, exc_value, traceback):  
    self.close()
```